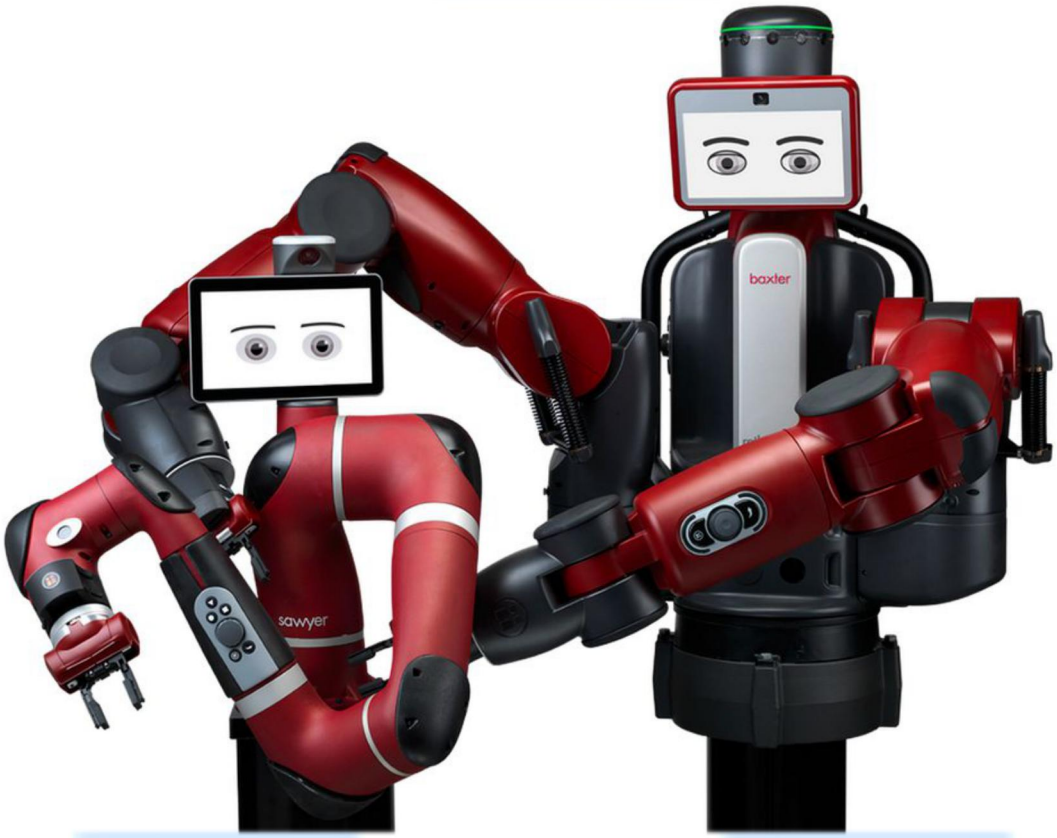
# 瑞森可研究型智能协作机器人
# BAXTER 开发指南




**湖南瑞森可机器人科技有限公司**
**2018 年版**

湖南瑞森可机器人科技有限公司
地址：长沙市岳麓区桐梓坡西路 348 号科力远综合办公楼 6 楼
电话：0731-89909712
传真：0731-89872400

瑞森可

# 前言

湖南瑞森可机器人科技有限公司是一家专业从事智能协作机器人及相关智能设备研究、开发、制造与销售，并面向全球提供智能协作机器人创新应用服务方案的高科技企业。

顺应时代发展趋势，在机器人产业迅猛发展的大浪潮下，瑞森可公司率先定义“智能协作机器人”，在制造与推广智能协作机器人 Baxter 和 Sawyer 基础上，通过引进、消化、吸收与再创新，形成了瑞森可智能协作机器人、智能移动机器人、机器人末端配件等系列产品包，并搭建瑞森可创新应用服务方案平台。同时，在国内建设智能协作机器人生产基地，创办全国首家智能协作机器人教育培训基地，并通过一流的团队、技术和服务为广大制造企业、科研院校量身定制个性化的整体解决方案，极大地推动了智能协作机器人在中国智能制造和科研教育领域中的应用。

瑞森可协作机器人是基于开源的机器人操作系统（Robot Operating System, ROS）开发的仿人形双臂 Baxter 和单臂 Sawyer 机器人，核心技术源自以美国罗德尼·布鲁克斯教授为核心的麻省理工人工智能实验室，配置了可用于先进机器人学领域研究开发的软件开发工具包（Software Development Kit, SDK），定位于为全球用户提供机器人研究、开发与应用平台。目前全球已有超过 27 个国家上 100 家实验室、大学和科研中心配置有 Baxter 或 Sawyer，其广泛应用于人机交互、机器学习、路径规划、机电一体化操控、计算机视觉等领域的科研与教学活动。

本指南为基于 Baxter 平台开展科研教学的用户提供基础的学习指南。指南主要介绍了 ROS 基础 、Baxter 工作原理与 SDK 开发、Baxter 应用案例等内容。

第一章：ROS 基础。介绍 ROS 基本命令的使用，ROS 节点编写，机器人 URDF 模型，机器人坐标变换以及一些常用的可视化工具。

第二章：MoveIt 运动规划。介绍 MoveIt 运动规划的基本原理和系统构成，运动规划接口，Baxter 运动规划配置和双臂运动规划示例。

第三章：Baxter 工作原理与 SDK 开发。介绍 Baxter 工作原理和力矩补偿、控制模式等基本概念，SDK 层次结构，应用程序编程接口及示例。

第四章：Baxter 应用案例。分析一些实际的应用案例，介绍 ROS 和 Baxter 综合应用。

附录一，Baxter 硬件参数。介绍 Baxter 机械臂及其他硬件参数。

本指南在编写过程中，参阅了国内外相应资料，在此向原作者表示感谢。

©2018，湖南瑞森可机器人科技有限公司版权所有。本指南的任何部分，包括文字、图片、图形等均归属于湖南瑞森可机器人科技有限公司，未经书面许可，任何单位和个人不得以任何方式摘录、复制、翻译、修改本指南的全部或部分。除非另有约定，本公司不对本指南提供任何明示或默示的声明或保证。

目录

第一章 ROS 基础.....1
1.1 ROS 简介.....1
1.2 ROS 基础.....3
1.2.1 工作空间.....3
1.2.3 ROS 基本概念.....10
1.2.4 创建 ROS 包.....12
1.2.5 编写 Publisher 与 Subscriber.....13
1.2.6 编写 Service 与 Client.....15
1.3 URDF.....17
1.4 TF.....19
1.5 Rqt 与 Rviz.....19
第二章 MoveIt 运动规划.....21
2.1 MoveIt 构成.....21
2.1.1 系统架构.....21
2.1.2 机器人接口.....21
2.1.3 运动规划.....22
2.1.4 规划场景.....24
2.1.5 运动学求解与碰撞检测.....25
2.1.6 轨迹处理.....26
2.2 MoveIt 接口.....26
2.2.1 配置 Baxter 运动规划包.....26
2.2.2 move_group 接口.....30
2.2.3 planning scene 接口.....33
第三章 Baxter 工作原理与 SDK 开发.....37
3.1 Baxter 基本工作原理.....37
3.1.1 机械臂控制概述.....37
3.1.2 机械臂控制模式.....37
3.1.3 碰撞检测.....40
3.1.4 重力补偿.....42
3.1.5 零重力模式.....43
3.2 Baxter 的 SDK.....44
3.3 Baxter 应用程序接口.....44
3.3.1 可用应用程序接口.....44
3.3.2 机械臂接口.....46
3.3.3 夹持器接口.....46
3.3.4 导航按钮接口.....46
3.3.5 使能/禁用机器人接口.....46
3.3.6 相机接口.....47
3.3.7 模拟IO 接口.....47
3.3.8 数字量IO 接口.....47
3.3.9 头部接口.....48
3.3.10 其他设置.....48
3.4 建立 Baxter 工作站.....49
3.4.1 硬件需求.....49
3.4.2 创建 Baxter 开发工作空间.....49
3.4.3 安装 Baxter 的 SDK.....49
3.4.4 配置 Baxter 通信/ROS 工作空间.....50
3.4.5 检查开发环境配置是否正确.....51

3.4.6 Hello Baxter.....51
3.5 SDK 示例解析.....54
3.5.1 键盘控制夹持器.....54
3.5.2 摇动头部.....58
3.5.3 双臂随动.....60
3.5.4 语音控制.....62
第四章 Baxter 应用案例.....66
4.1 双臂协同及运动学跟踪.....66
4.1.1 对偶四元数.....66
4.1.2 任务协作操作.....67
4.1.3 Baxter 跟踪控制器.....67
4.1.4 下载与安装.....67
4.1.5 运行示例.....67
4.1.6 使用跟踪功能.....67
4.1.7 ROS 服务及主题.....68
4.2 视觉抓取乒乓球.....68
4.2.1 相机与工作空间的关系.....68
4.2.2 相机标定参数.....69
4.2.3 像素坐标到工作空间坐标的转换.....69
4.2.4 确定托盘在工作空间中的位置.....70
4.2.5 确定托盘方向及插值放置位置.....71
4.2.6 确定球的位置.....72
4.2.7 下载和安装.....72
4.3 基于Kinect 的动作跟随.....72
4.4 脸部识别与跟踪.....78
4.5 Baxter 下四连棋游戏.....81
附录一 Baxter 硬件参数.....87

# 第一章 ROS 基础

本章主要介绍机器人操作系统 ROS 相关的一些基本知识与操作指令。

## 1.1 ROS 简介

ROS 是一个开源的机器人操作系统，需运行在计算机操作系统之上（例如 Linux 系统），因此也认为是一类用于机器人的开源的元操作系统。ROS 为机器人软件开发者提供了大量的库和工具，包括硬件抽象，设备驱动，计算机视觉，消息传递，软件包管理等。ROS 起源于美国斯坦福大学人工智能实验室，前期主要由 Willow Garage 公司负责维护，现在转交 OSRF（Open Source Robotics Foundation, OSRF）负责运营，全球的大学和机器人研究机构都积极参与其中。

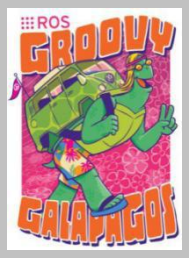



**图 1-1 使用 ROS 的部分研究机构**

从 2009 年 Willow Garage 开源 ROS 依赖，ROS 已经发行了 10 多个版本。目前常用的是 Indigo 和 Kinetic，本教材以 Indigo 为例。

**表 1-1 ROS 版本列表**

| 版本 | 发布日期 | 海报 | 图标 | EOL 日期 |
|---|---|---|---|---|
| ROS Melodic Morenia | May 23rd, 2018 |  |  | May, 2023 (Bionic EOL) |
| ROS Lunar Loggerhead | May 23rd, 2017 |  |  | May, 2019 |

| | | | | |
|---|---|---|---|---|
| ROS Kinetic Kame **(Recommended)** | May 23rd, 2016 |  |  | May, 2021 |
| ROS Jade Turtle | May 23rd, 2015 |  |  | May, 2017 |
| ROS Indigo Igloo | July 22nd, 2014 |  |  | April, 2019 (Trusty EOL) |
| ROS Hydro Medusa | Sep 4th, 2013 |  |  | May, 2015 |
| ROS GroovyGalapagos | December 31,2012 |  |  | July, 2014 |
| ROS Fuerte Turtle | April 23, 2012 |  |  | -- |
| ROS Electric Emys | August 30, 2011 |  |  | -- |
| ROS Diamondback | March 2, 2011 |  |  | -- |

| | | | | |
|---|---|---|---|---|
| ROS C Turtle | August 2, 2010 |  |  | -- |
| ROS Box Turtle | March 2, 2010 |   |  | -- |

## 1.2 ROS 基础

### 1.2.1 工作空间

Packages，软件包，是 ROS 应用程序代码的组织单元，每个软件包都可以包含程序库、可执行文件、脚本或者其它手动创建的东西。

程序代码是分布在众多 ROS 软件包当中，当使用命令行工具（比如 ls 和 cd）来浏览时会非常繁琐，因此 ROS 提供了专门的命令工具来简化这些操作。

**1) rospack 命令**

rospack 允许你获取软件包的有关信息。在本指南中，我们只涉及到 rospack 中 find 参数选项，该选项可以返回软件包的路径信息。

用法：

```
#rospack find[包名称]
```

示例：

```
$ rospack find roscpp
```

应输出：

```
YOUR_INSTALL_PATH/share/roscpp
```

如果你是在 UbuntuLinux 操作系统上通过 apt 来安装 ROS，你应该会准确地看到：

```
/opt/ros/indigo/share/roscpp
```

**2) roscd 命令**

roscd 是 rosbash 命令集中的一部分，它允许你直接切换(cd)工作目录到某个软件包或者软件包集当中。

用法：

```
#roscd[本地包名称[/子目录]]
```

示例：

```
$ roscd roscpp
```

为了验证我们已经切换到了 roscpp 软件包目录下，现在我们可以使用 Unix 命令 pwd 来 输出当前工作目录：

```
$ pwd
```

你应该会看到：

```
YOUR_INSTALL_PATH/share/roscpp
```

你可以看到 YOUR_INSTALL_PATH/share/roscpp 和之前使用 rospackfind 得到的路径名称是一样的。

注意，就像 ROS 系统中的其它工具一样，roscd 只能切换到那些路径已经包含在 ROS_PACKAGE_PATH 环境变量中的软件包，要查看 ROS_PACKAGE_PATH 中包含的路径可以输 入：

```
$ echo $ROS_PACKAGE_PATH
```

你的 ROS_PACKAGE_PATH 环境变量应该包含那些保存有 ROS 软件包的路径，并且每个路径之间用冒号分隔开来。一个典型的 ROS_PACKAGE_PATH 环境变量如下：

```
/opt/ros/indigo/base/install/share:/opt/ros/indigo/base/install/stacks
```

跟其他路径环境变量类似，你可以在 ROS_PACKAGE_PATH 中添加更多其它路径，每条路径使用冒号':'分隔。

**3) 子目录**

使用 roscd 也可以切换到一个软件包或软件包集的子目录中。 执行：

```
$ roscd roscpp/cmake
$ pwd
```

应该会看到：

```
YOUR_INSTALL_PATH/share/roscpp/cmake
```

**4) roscdlog**

使用 roscdlog 可以切换到 ROS 保存日记文件的目录下。需要注意的是，如果你没有执行过任何 ROS 程序，系统会报错说该目录不存在。

如果你已经运行过 ROS 程序，那么可以尝试：

```
$ roscd log
```

**5) 使用 rosls**

rosls 是 rosbash 命令集中的一部分，它允许你直接按软件包的名称而不是绝对路径执行 ls 命令（罗列目录）。

用法：

```
#rosls[本地包名称[/子目录]]
```

示例：

```
$ rosls roscpp_tutorials
```

应输出：

```
cmake package.xml srv
```

**6) Tab 自动完成输入**

当要输入一个完整的软件包名称时会变得比较繁琐。在之前的例子中 roscpp_tutorials 是个相当长的名称，幸运的是，一些 ROS 工具支持 TAB 自动完成输入的功能。

输入：

```
$ roscd roscpp_tut<<<现在请按 TAB 键>>>
```

当按 TAB 键后，命令行中应该会自动补充剩余部分：

```
$ roscd roscpp_tutorials/
```

这应该有用，因为roscpp_tutorials 是当前唯一一个名称以roscpp_tut 作为开头的ROS 软件包。

现在尝试输入：

```
$ roscd tur<<<现在请按 TAB 键>>>
```

按 TAB 键后，命令应该会尽可能地自动补充完整：

```
$ roscd turtle
```

但是，在这种情况下有多个软件包是以 turtle 开头，当再次按 TAB 键后应该会列出所有以 turtle 开头的 ROS 软件包：

```
turtle_actionlib/turtlesim/turtle_tf/
```

这时在命令行中你应该仍然只看到：

```
$ roscd turtle
```

现在在 turtle 后面输入 s 然后按TAB 键：

```
$ roscd turtles<<<请按 TAB 键>>>
```

因为只有一个软件包的名称以 turtles 开头，所以你应该会看到：

```
$ roscd turtlesim/
```

ROS 命令工具的的命名方式：

```
rospack = ros + pack(age)
roscd = ros + cd
rosls = ros + ls
```

这种命名方式在许多ROS 命令工具中都会用到。

### 1.2.2 ROS 基本命令

| 文件系统操作 | |
|---|---|
| **rospack** | 检查ROS 包 |
| **rospack profile** | 修复路径和pluginlib 问题 |
| **roscd** | 切换目录 |
| **rospd/rosd** | ROS 的Pushd 等价 |
| **rosls** | 列出包的信息 |
| **rosed** | 打开ROS 文件 |
| **roscp** | 文件复制 |
| **rosdep** | 安装系统依赖 |
| **roswtf** | 显示正在运行的ROS 系统的警告和错误 |
| **catkin_create_pkg** | 创建包 |
| **wstool** | 管理工作空间的软件库 |
| **catkin_make** | 编译工作空间代码 |
| **rqt_dep** | 显示软件包结构和依赖 |

用法：

```
$ rospack find [package]
$ roscd [package[/subdir]]
$ rospd [package[/subdir] | +N | -N]
$ rosd
$ rosls [package[/subdir]]
$ rosed [package] [file]
$ roscp [package] [file] [destination]
$ rosdep install [package]
$ roswtf or roswtf [file]
$ catkin_create_pkg [package name] [depend1]..[dependN]
$ wstool [init | set | update]
$ catkin_make
$ rqt_dep [options]
```

| 节点启动 | |
|---|---|
| **roscore** | 启动ros 主节点及参数服务等 |
| **rosrun** | 运行可执行文件 |
| **roslaunch** | 启动launch 文件 |

用法：

```
roscore
$ roscore
rosrun
$ rosrun package_name executable_name
如：
$ rosrun turtlesim turtlesim.node
```

```
roslaunch
$ roslaunch package_name file_name.launch
$ roslaunch -p 1234 package_name file_name.launch
$ roslaunch --local package_name file_name.launch
```

| 日志工具rosbag | |
|---|---|
| **rosbag record** | 记录特定topic 的数据到bag 文件 |
| **rosbag play** | 回放bag 文件 |
| **rosbag compress** | 压缩一个或多个bag 文件 |
| **rosbag decompress** | 解压缩bag 文件 |
| **rosbag filter** | 过滤bag 文件内容 |

用法：

```
记录：$ rosbag record topic1 topic2
回放：$ rosbag play -a demo log.bag
多个同时回放：$ rosbag play demo1.bag demo2.bag
```

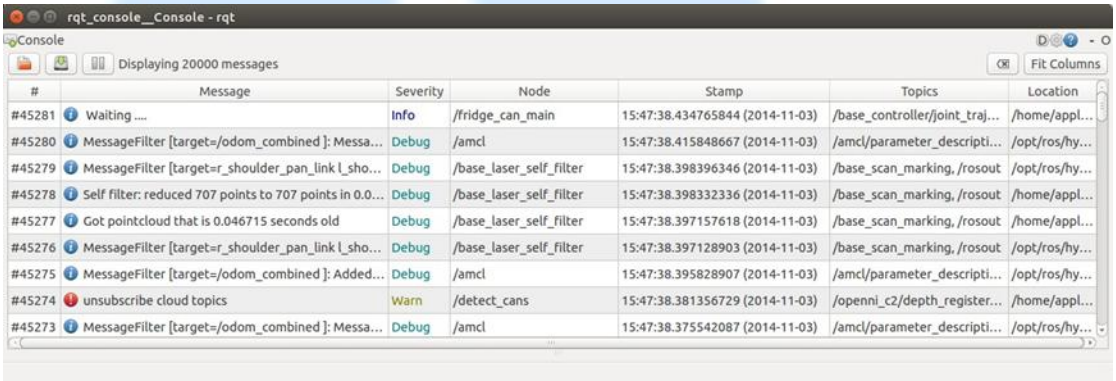
| 节点主题自检 rosmsg/rossrv, rosnode, rostopic rosparam rosservice | |
|---|---|
| **rosmsg show** | 显示消息定义 |
| **rosmsg list** | 显示全部消息名字 |
| **rosmsg md5** | 显示消息MD5 值 |
| **rosmsg package** | 列出包中的所有消息 |
| **rosmsg packages** | 列出包含消息的所有包 |
| **rosnode ping** | 测试节点连接 |
| **rosnode list** | 列出所有活动节点 |
| **rosnode info** | 输出节点信息 |
| **rosnode machine** | 列出某机器人上运行的节点 |
| **rosnode kill** | 终止一个节点 |
| **rostopic bw** | topic 带宽 |
| **rostopic echo** | 显示topic 内容 |
| **rostopic find** | 根据类型寻找topic |
| **rostopic hz** | topic 发布频率 |
| **rostopic info** | topic 信息 |
| **rostopic list** | 所有topic 列表 |
| **rostopic pub** | 向topic 发布消息 |
| **rostopic type** | topic 类型 |
| **rosparam set** | 设置参数值 |
| **rosparam get** | 获取参数值 |
| **rosparam load** | 装载参数文件 |
| **rosparam dump** | 将参数写入到文件 |
| **rosparam delete** | 删除参数 |

| | |
|---|---|
| **rosparam list** | 所有参数列表 |
| **roservice list** | 列出所有可用服务 |
| **roservice node** | 打印提供服务的节点 |
| **roservice call** | 调用服务 |
| **roservice args** | 列出服务所需参数 |
| **roservice type** | 打印服务类型 |
| **roservice uri** | 打印服务ROSRPC uri |
| **roservice find** | 根据服务类型找到服务 |

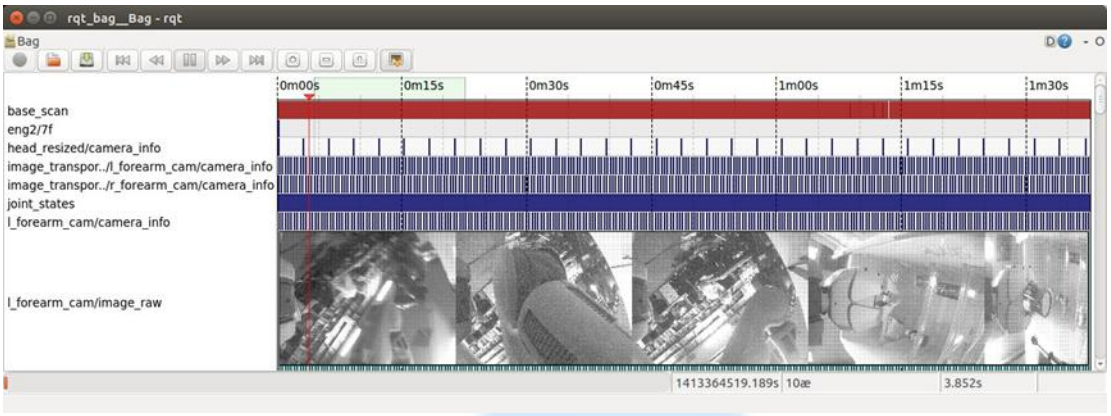
| **图形界面工具** | |
|---|---|
| **rqt_console** | 显示rosout 上的消息 |
| **rqt_bag** | 记录、回放bag 文件 |
| **rqt_logger_level** | 设置日志等级，如警告、错误 |
| **rqt_topic** | 查看发布的topics |
| **rqt_msg/srv** | 显示消息或服务定义 |
| **rqt_publisher** | 向topic 发送消息 |
| **rqt_service_caller** | 调用服务 |
| **rqt_graph** | 节点关联图 |
| **rqt_dep** | 软件包依赖关系 |
| **rqt_reconfigure** | 参数动态配置 |
| **rqt_shell** | 指令输入窗口 |
| **tf_echo** | 打印坐标系间的坐标变换 |
| **view_frames** | 查看系统坐标变换树 |
| **rqt_plot** | 画出topic 数据图形 |
| **rqt_image_view** | 显示图像 |

示例：

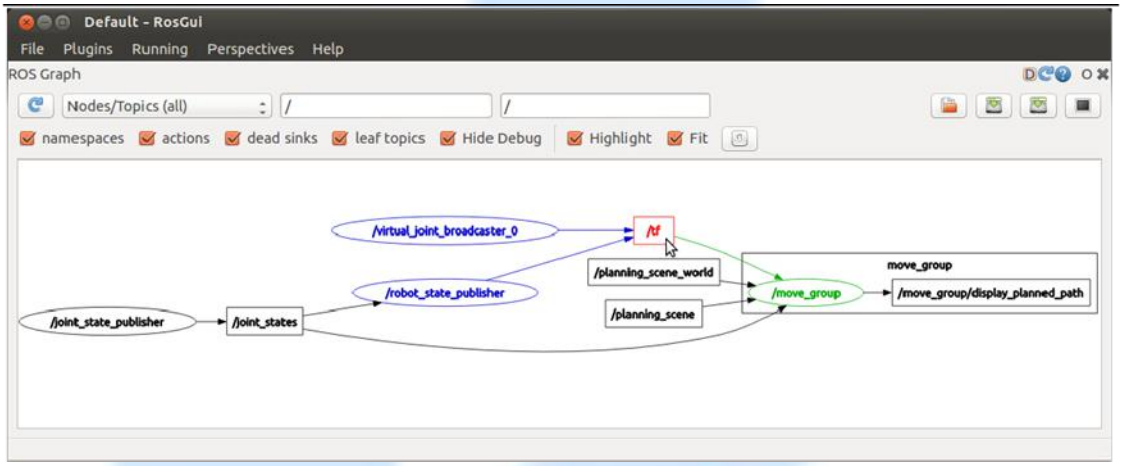
**rqt_console：**



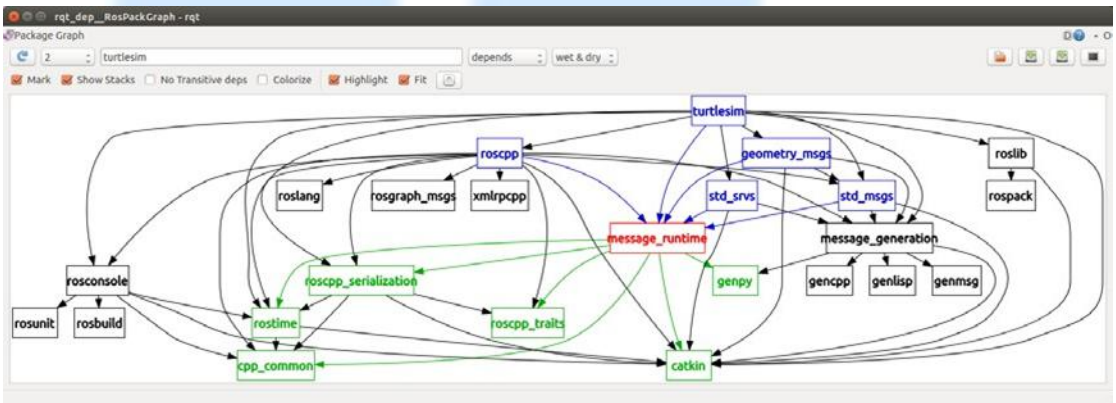
**rqt_bag：**



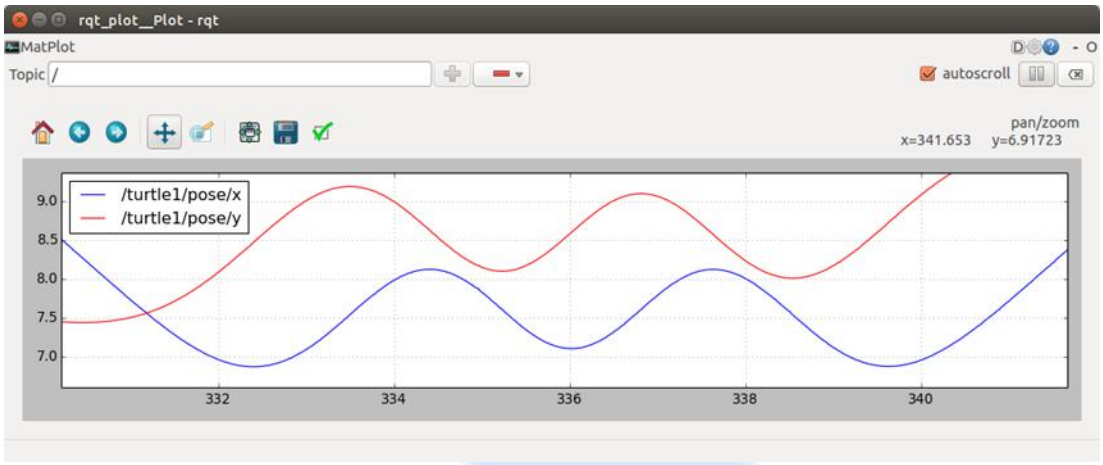

**rqt_graph：**



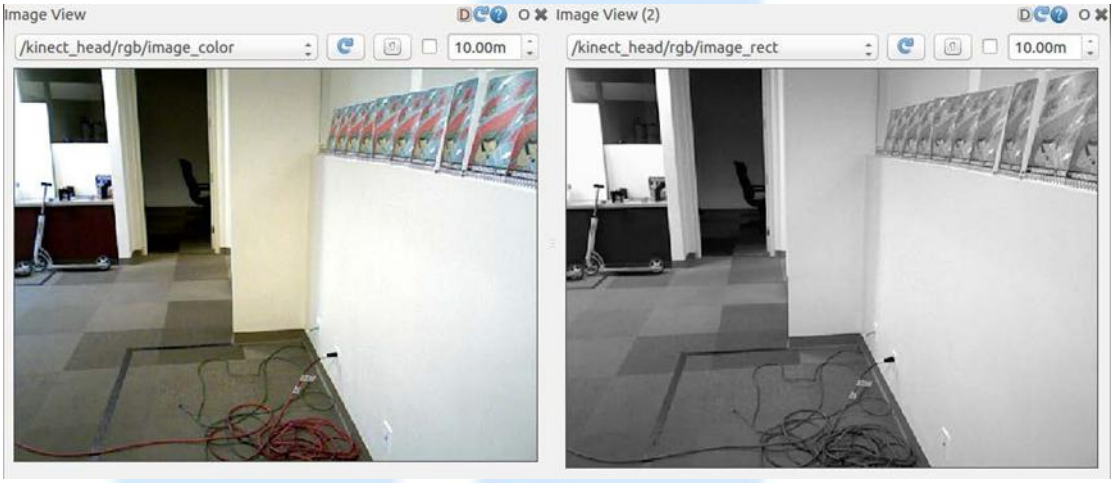

**rqt_dep：**



**rqt_plot：**




**rqt_image_view：**




### 1.2.3 ROS 基本概念

讲述 ROS 基本概念与概念之间的联系。

**1）node 与topic**

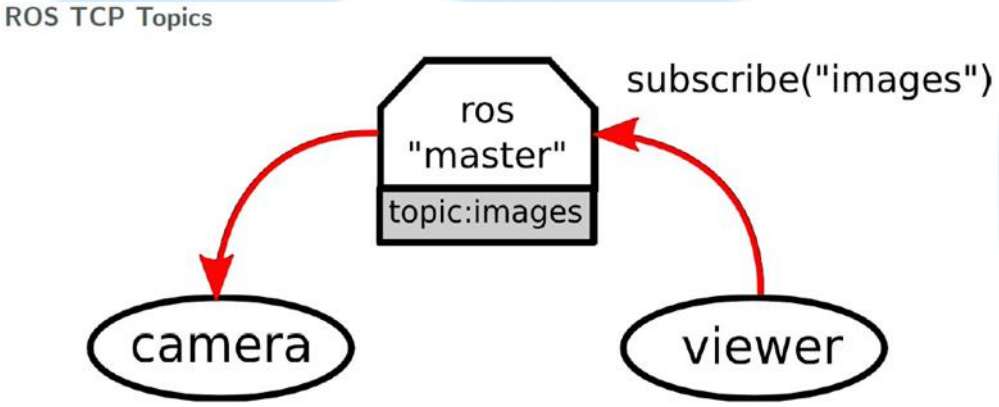



**图 1-2 ROS 节点通讯图**

**Nodes**：node 是ROS 的基本可执行单元，如图 1-2 中的 camera，viewer；

**Topics**：topic 即主题，各 node 可以向 topic 发送消息，或监听 topic 消息；

**Messages**：topic 消息类型，如 Float64，Vector；

**Master：为 node 提供名称服务，所以 node 都要向 master 注册；**

**当 nodes 通过 master 注册后可以直接用 TCP 协议通讯，如下图所示。**

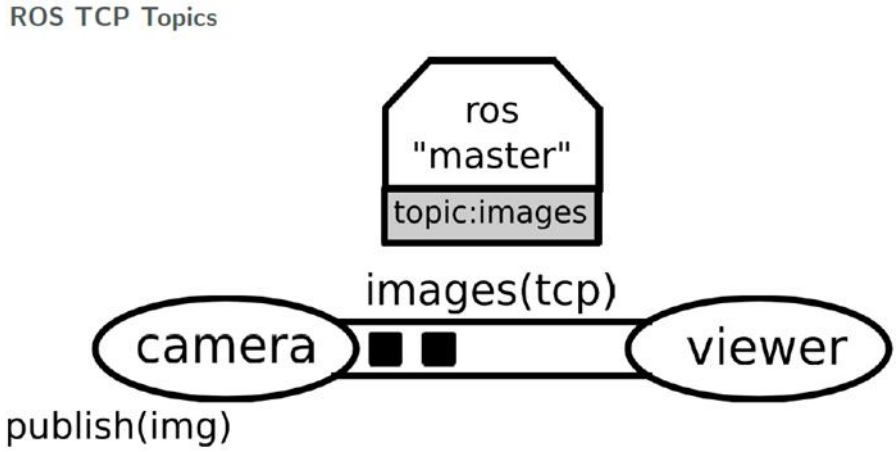



**图 1-3 node 直接通讯**

每个 topic 可对应多个 publisher 和 subscriber，如图1-4，camera node 发布 imgtopi，viewer node 和 viewer_too node 均监听于 imgtopic。

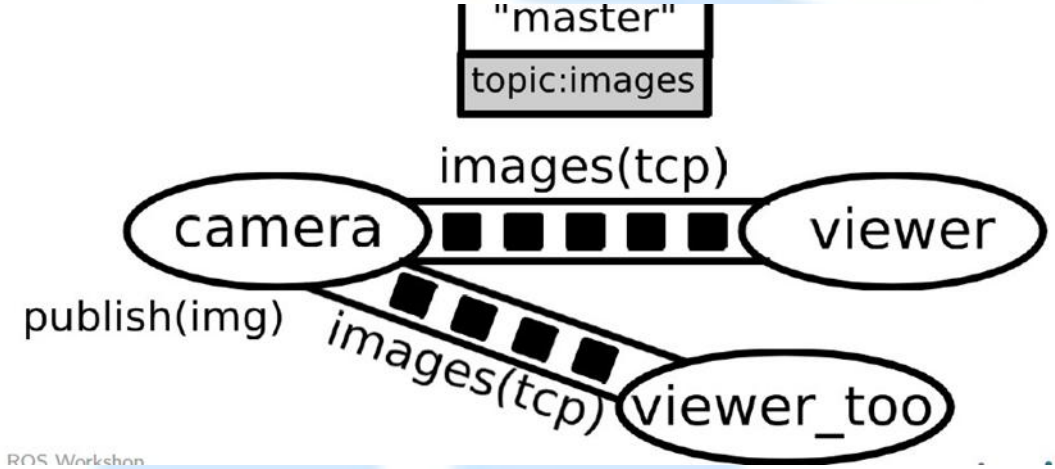



**图 1-4 一个topic 对应多个publisher 和subscriber**

**2）service，action，与 parameter server**

Nodes 之间也可以通过 Service 进行通讯。Service server 提供服务，Service client 通过 XML RPC 调用Service server 提供的服务，如图1-5 所示。

Action 和 Service 很相似，对于有时间要求的任务，比如移动机械臂到某个位置，我们想知道机械臂移动到某个位置，执行的结果。Action 可以提供反馈和执行结果，相当于实时的增强型Service。

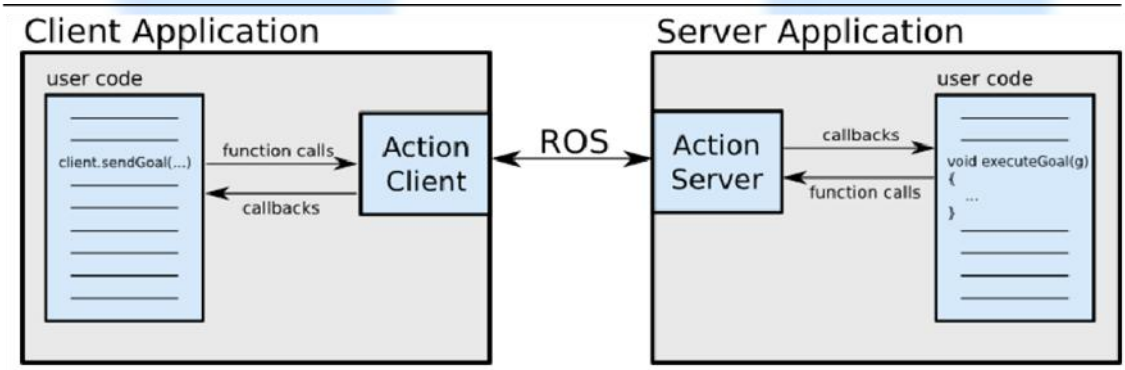



**图 1-5 action 工作方式**

### 1.2.4 创建 ROS 包

我们可以使用(catkin_create_pkg)工具来自动创建 ROS package，roscreate-pkg 易于使用，能够避免很多错误，但所创建 package 只是一个文件夹外加一个 XML 文件。

现在，我们要创建一个名为 foobar 的package。请转至 catkin workspace，并确认你已经刷新了 setup 文件。

```
catkin_ws_top $ mkdir -p src/foobar
catkin_ws_top $ cd src/foobar
```

我们要做的第一件事就是添加配置文件。 package.xml 文件使得像 rospack 这类工具能够检测出我们所创建的 package 的依赖项。

在 foobar/package.xml 文件里添加如下语句:

```
<package>

    <name>foobar</name>
    <version>1.2.4</version>
    <description>
    This package provides foo capability.
    </description>

    <maintainer email="foobar@foo.bar.willowgarage.com">PR-foobar</main tainer>
    <license>BSD</license>
    <buildtool_depend>catkin</buildtool_depend>
    <build_depend>roscpp</build_depend>
    <build_depend>std_msgs</build_depend>
    <run_depend>roscpp</run_depend>
    <run_depend>std_msgs</run_depend>

</package>
```

同样，你可以参考 catkin tutorial 以获得更多关于 catkin/package.xml 的信息。

现在，你的 package 里已经包含配置文件(package.xml)，ROS 能够找到它。试一试，执行如下指令：

```
rospack find foobar
```

如果 ROS 配置正确，你将会看到类似如下的信息：

/home/user/ros/catkin_ws_top/src/foobar

这就是 ROS 如何在后台寻找 package 的机理。 应该注意到我们刚才所创建的 package 依赖于 roscpp 和 std_msgs。而 catkin 恰恰是利用这些依赖项来配置所创建的 package。

至此，我们还需要一个 CMakeLists.txt 文件，这样 catkin_make 才能够利用 CMake 强大的跨平台特性来编译所创建的 package。

在 foobar/CMakeLists.txt 文件里输入：

```
cmake_minimum_required(VERSION 2.8.3)
project(foobar)
find_package(catkin REQUIRED roscpp std_msgs)
catkin_package()
```

这就是你想在 ROS 下用 catkin 编译 package 所要做的全部工作。当然了，如果想要让它正真的编译一些东西，你还需要学习关于 CMake 宏的一些知识。

请参考 CMakeLists.txt 以获得 更多信息。 回顾初级教程如(Creating Package 等)来修改你的 package.xml 和 CMakeLists.txt 文件。、

### 1.2.5 编写 Publisher 与 Subscriber

节点是 ROS 里的可执行单元，可以连接到 ROS 网络中去，这里我们创建一个消息发布节点（“talker”），它将持续的向 ROS 系统发布消息。

同时创建一个名为 listener 的 Subscriber 节点接收 talker 节点发布的字符串消息，并在屏幕上打印。

**1) Publisher 代码**

```
#!/usr/bin/env python
# license removed for brevity
import rospy
from std_msgs.msg import String

def talker():
    pub = rospy.Publisher('chatter', String, queue_size=10)
    rospy.init_node('talker', anonymous=True)
    rate = rospy.Rate(10) # 10hz
    while not rospy.is_shutdown():
        hello_str = "hello world %s" % rospy.get_time()
        rospy.loginfo(hello_str)
        pub.publish(hello_str)
        rate.sleep()

if __name__==’__main__’:
    try:
        talker()
    except rospy.ROSInterruptException:
        Pass
```

**2) Publisher 代码解释**

python 说明：

```
1 #!/usr/bin/env python
```

每一个 python 节点文件的顶部都应该放上这么一段，声明用哪个 Python 解释器执行这个脚本。

```
3 import rospy
4 from std_msgs.msg import String
```

导入rospy 包，从std_msgs 包中引入String 类型的消息。

```
7  pub=rospy.Publisher('chatter',String,queue_size=10)
8  rospy.init_node('talker',anonymous=True)
```

创建消息发布器，初始化ROS 节点。

```
9 rate=rospy.Rate(10)#10hz
```

设定消息发布频率为10Hz。

```
10 while not rospy.is_shutdown():
11      hello_str="helloworld%s"%rospy.get_time()
12      rospy.loginfo(hello_str)
13      pub.publish(hello_str)
14      rate.sleep()
```

主循环，当ROS 没有关闭时，持续发布‘helloworld’的字符串消息。

```
Msg = String()
msg.data =str
String(data = str)
```

构造消息的实例。

```
17 try:
18     talker()
19 except rospy.ROSInterruptException:
20     pass
```

异常捕获，Ctrl+C 结束程序。

**3) Subscriber 代码**

```
1       #!/usr/bin/env python
2       import rospy
3       from std_msgs.msg import String
4
5       def callback(data):
6           rospy.loginfo(rospy.get_caller_id() + "I heard %s", data.data)
7
8       def listener():
9
10          # In ROS, nodes are uniquely named. If two nodes with the same
11          # node are launched, the previous one is kicked off. The
12          # anonymous=True flag means that rospy will choose a unique
13          # name for our 'listener' node so that multiple listeners can
14          # run simultaneously.
15          rospy.init_node('listener', anonymous=True)
16
17          rospy.Subscriber("chatter", String, callback)
```

```
18
19          # spin() simply keeps python from exiting until this node is stopped
20          rospy.spin()
21
22      if __name__ == '__main__':
23          listener()
```

为代码添加可执行权限：

```
$ chmod +x listener.py
```

**5）Subscriber 代码解释**

初始化 ROS 节点：

```
15 rospy.init_node('listener', anonymous=True)
16
17 rospy.Subscriber("chatter", String,callback)
18
19# spin() simply keeps python from exiting until this node is stop ped
20 rospy.spin()
```

声明节点要监听的topic 名字及消息类型，设置回调函数，当新消息到来时自动调用回调函数，对传入的消息做相应处理。

rospy.spin()保持节点处于运行状态，不同于roscpp 中的spin（），Python 中的subscriber 有自己的线程，spin（）不影响回调函数。

## 1.2.6 编写 Service 与 Client

这里我们将创建一个名为（“add_two_ints_server”）的 Service，它接收传递过来的两个整数，返回两数之和。

**1) Service 代码**

```
1       #!/usr/bin/env python
2
3       from beginner_tutorials.srv import *
4       import rospy
5
6       def handle_add_two_ints(req):
7           print "Returning [%s + %s = %s]"%(req.a, req.b, (req.a + req.b))
8           return AddTwoIntsResponse(req.a + req.b)
9
10      def add_two_ints_server():
11          rospy.init_node('add_two_ints_server')
12          s = rospy.Service('add_two_ints', AddTwoInts, handle_add_two_ints)
13          print "Ready to add two ints."
14          rospy.spin()
15
16      if __name__ == "__main__":
```

```
17          add_two_ints_server()
```

**2) Service 代码解释**

同样我们首先通过rospy 初始化节点。

```
12  s = rospy.Service('add_two_ints', AddTwoInts, handle_add_two_ints)
```

然后声明一个新的 service，名字为“add_two_ints”，类型是 AddTwoInts。然后把服务请求中的参数传递到 handle_add_two_ints，该函数求处理请求后返回请求处理结果。同样，这里的rospy.spin()只是将程序挂起，没有实际作用。

**3) Client 代码**

```
1       #!/usr/bin/env python
2
3       import sys
4       import rospy
5       from beginner_tutorials.srv import *
6
7       def add_two_ints_client(x, y):
8           rospy.wait_for_service('add_two_ints')
9           try:
10              add_two_ints = rospy.ServiceProxy('add_two_ints', AddTwoInts)
11              resp1 = add_two_ints(x, y)
12              return resp1.sum
13          except rospy.ServiceException, e:
14              print "Service call failed: %s"%e
15
16      def usage():
17          return "%s [x y]"%sys.argv[0]
18
19      if  __name__ == " __main__  ":
20          if len(sys.argv) == 3:
21              x = int(sys.argv[1])
22              y = int(sys.argv[2])
23          else:
24              print usage()
25              sys.exit(1)
26          print "Requesting %s+%s"%(x, y)
27          print "%s + %s = %s"%(x, y, add_two_ints_client(x, y))
```

**4) Client 代码解释**

Client 没必要初始化ROS 节点，我们通过wait_for_service 等待Service 端服务启动，服务启动之前，此函数将一直处于等待状态。

```
8 rospy.wait_for_service('add_two_ints')
```

然后通过ServiceProxy 创建Client 对象。

```
10 add_two_ints = rospy.ServiceProxy('add_two_ints',AddTwoInts)
```

最后直接调用该对象，并将两个整数作为参数，返回值存到resp1 中。

```
11 resp1 = add_two_ints(x,y)
12return resp1.sum
```

## 1.3 URDF

URDF（Unified Robot Description Format），统一机器人描述格式，是一种 XML 语言写成的，用于描述机器人连杆和关节关节的文件。连杆部分有，可视化、碰撞属性，用于显示机器人三维模型并用于碰撞检测，关节部分定义关节类型，转角或平动范围限制，父连杆子连杆关系等。

**1）连杆定义**

```
<link name="my_link">
  <inertial>
    <origin xyz="0 0 0.5" rpy="0 0 0"/>
    <mass value="1"/>
    <inertia ixx="100"  ixy="0"  ixz="0"  iyy="100"  iyz="0"  izz="100" />
  </inertial>

  <visual>
    <origin xyz="0 0 0" rpy="0 0 0" />
    <geometry>
      <box size="1 1 1" />
    </geometry>
    <material name="Cyan">
      <color rgba="0 1.0 1.0 1.0"/>
    </material>
  </visual>

  <collision>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <cylinder radius="1" length="0.5"/>
    </geometry>
  </collision>
</link>
```

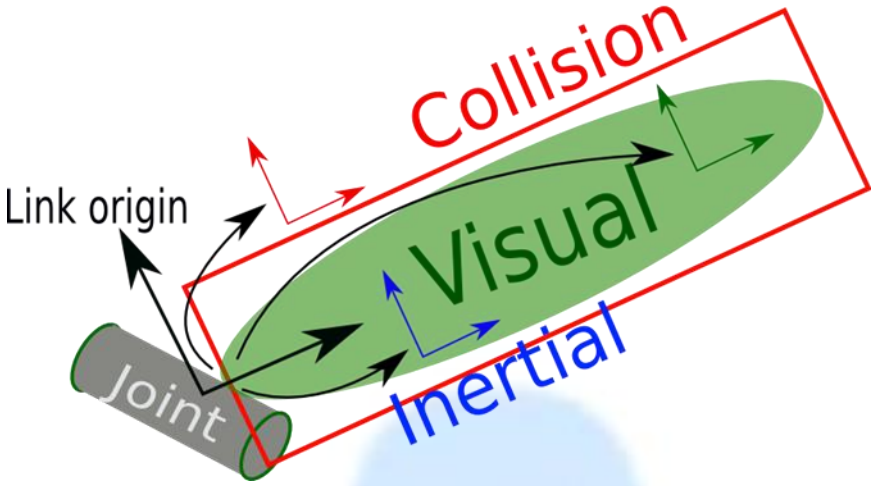



**图 1-6 连杆**

上述XML 定义了一个连杆，<inertial>标签定义了该连杆的质量和转动惯量，<visual>定义了物体的可视化属性，这里是长、宽、高均为 1m 的立方体，显示颜色是Cyan，<collision>定义了机器人的碰撞属性，用于碰撞检测，这里是直径为1m，半径为0.5m 的圆柱。

**2）关节定义**

```
<joint name="my_joint" type="floating">
    <origin xyz="0 0 1" rpy="0 0 3.1416"/>
    <parent link="link1"/>
    <child link="link2"/>

    <calibration rising="0.0"/>
    <dynamics damping="0.0" friction="0.0"/>
    <limit effort="30" velocity="1.0" lower="-2.2" upper="0.7" />
    <safety_controller k_velocity="10" k_position="15" soft_lower_limit="-2.0"
    soft_upper_limit="0.5" />
</joint>
```

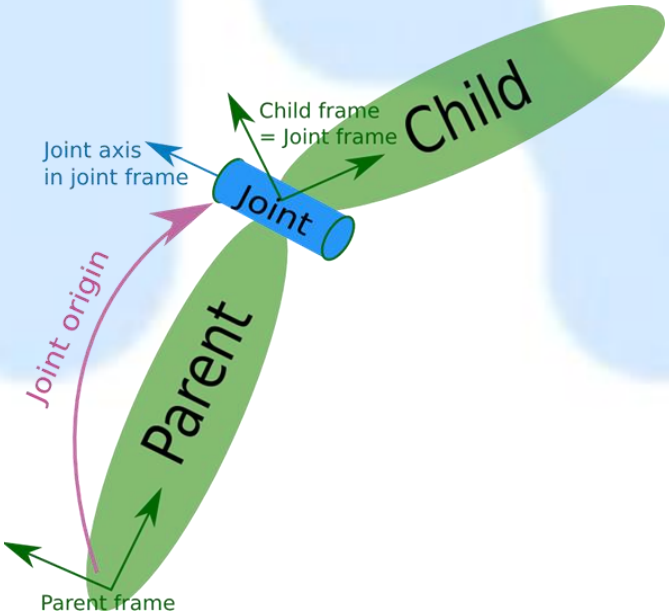



**图 1-7 浮动关节**

上述 XML 定义了一个浮动关节，该关节的父连杆是 link1，子连杆是 link2，同时也定义了阻尼系数，速度、力矩等限制。常见的关节类型，比如机械臂，是 revolute，表示旋转关节，有转角限制；如移动平台，是 continuous，表明轮子可以连续旋转，

更详细的 XML 定义，可以参考：http://wiki.ros.org/urdf/XML。

## 1.4 TF

TF（transform），用于表示和处理机器人不同连杆之间的坐标变换关系。它内部维护一个坐标变换树，可以根据关节转角信更新机器人不同部分的坐标变换关系。

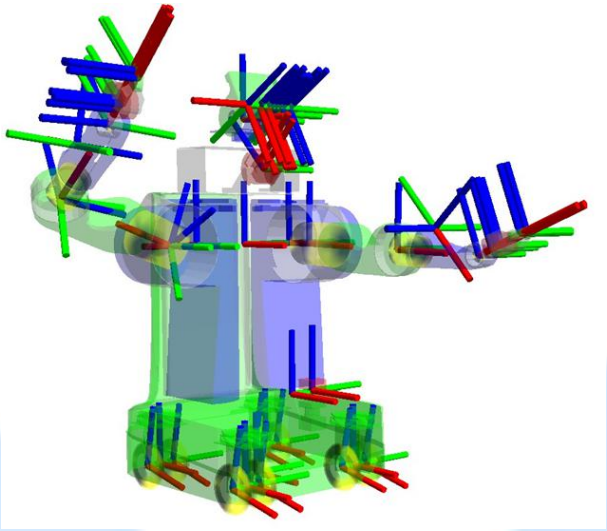


图 1-8 TF 模型

## 1.5 Rqt 与 Rviz

Rqt 和Rviz 都是ROS 中常用的可视化工具，rqt 是基于QT 的ROS 图形化界面，有很多实用的工具，比如发布消息、控制机器人运动、监控机器人状态、绘图等。

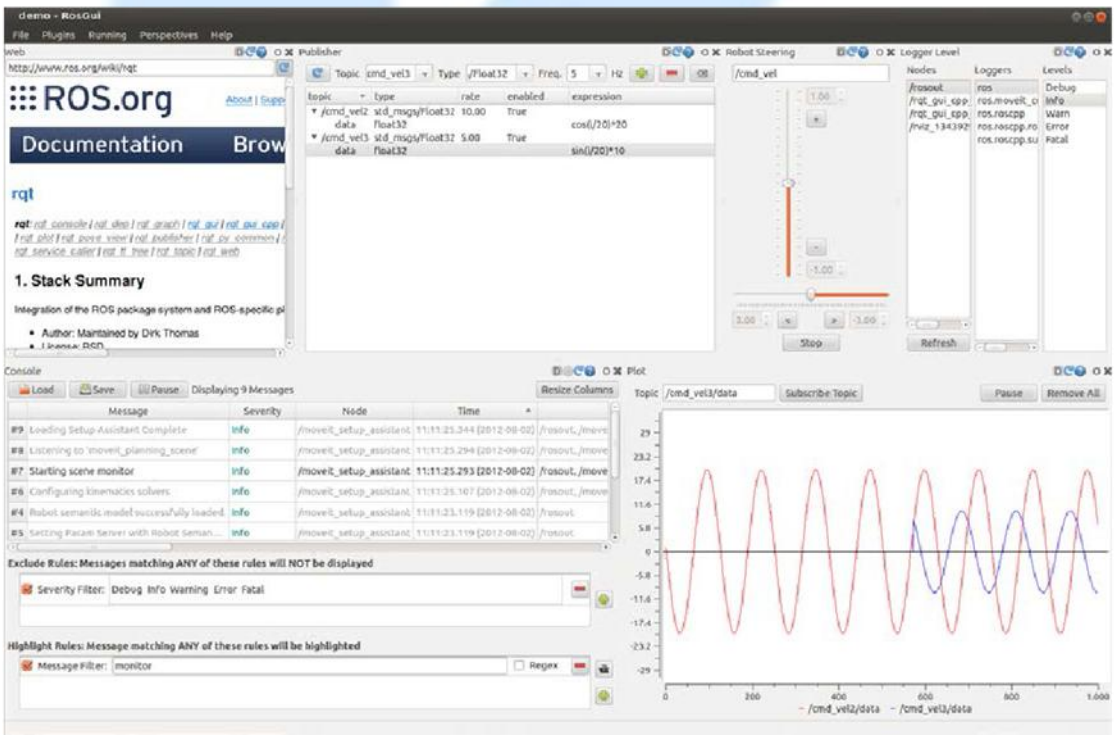



图 1-9 rqt 界面

Rviz 用于显示ROS 系统的各种参数或消息，比如显示机器人模型、相机图像、坐标轴坐标变换等等。

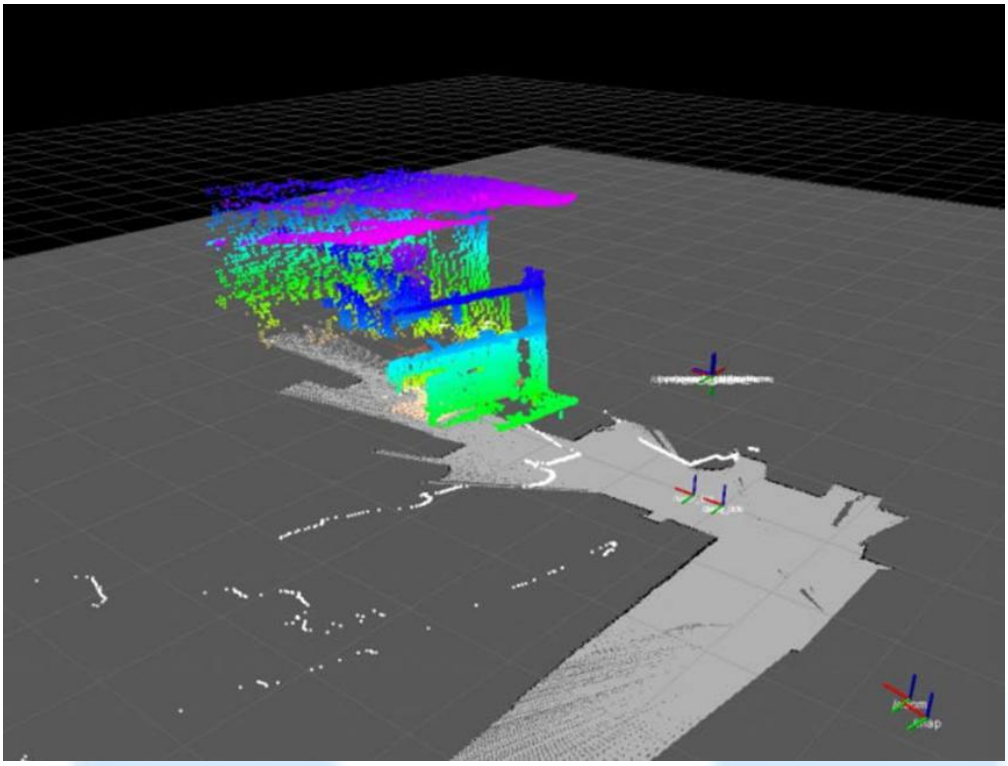


**图 1-10 Rviz 界面**

# 第二章 MoveIt 运动规划

## 2.1 MoveIt 构成

### 2.1.1 系统架构

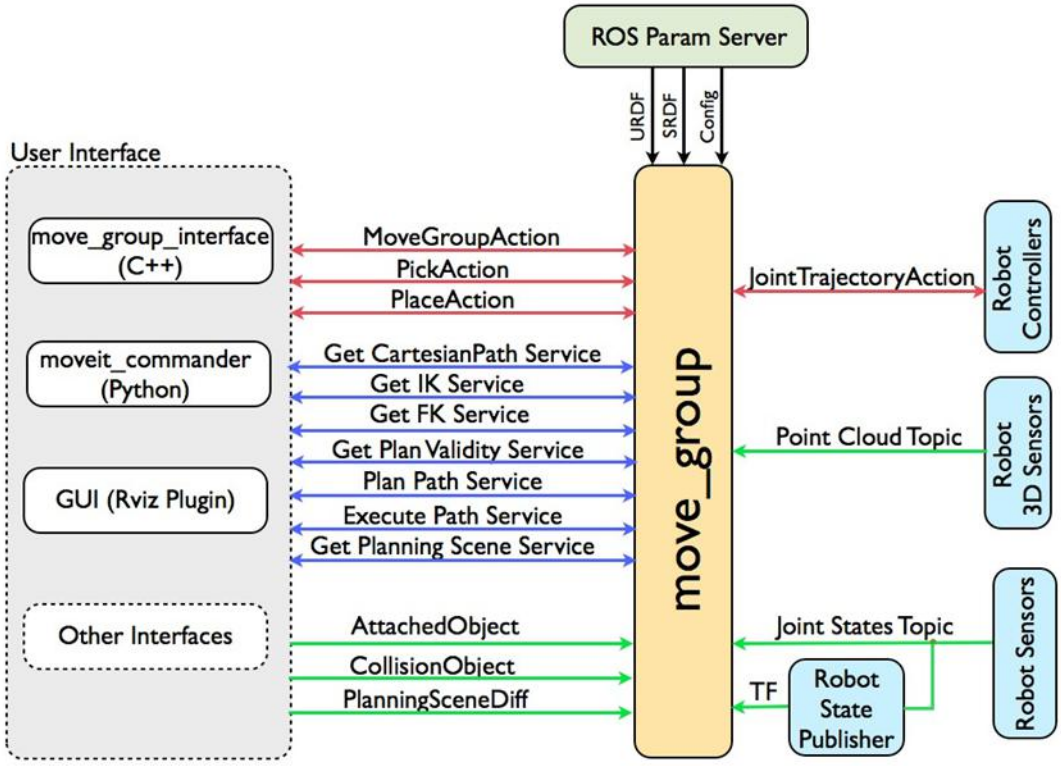



**图 2-1 系统架构图**

**1）move_group 节点**

上图是一个高层级的系统架构图，move_group 节点作为一个集成节点，通过加载不同的插件实现各种各样的功能。

**2）用户接口**

用户可以通过三种方式和move_group 交互，C++，Python 及图形界面。

**3）配置**

move_group 作为ros 节点需要各种配置参数才能工作，这些参数包括：

1. URDF-move_group 寻找 robot_description 以获得机器人模型；

2. SRDF-move_group 通过寻找 robot_semantic_description 参数获得机器人运动链，碰撞检测矩阵等参数；

3. MoveIt 配置：MoveIt 通过参数服务器获得机器人转角限制，运动学描述，运动规划，感知等参数，这些配置文件默认位于 MoveIt 配置包的 config 目录下。

### 2.1.2 机器人接口

move_group 通过 topic、action 和机器人交互，它需要获取机器人的当前状态信息，比如关节位置、点云等机器人传感器数据，也会向机器人的运动控制器发送轨迹指令。

**1）关节状态信息**

move_group 监听 joint_states 以获取当前机器人的状态信息，比如由此获得关节转角。move_group 可以同时监听多个发布到此 topic 的发布器，可能每个发布器只发布机器人部分关节的信息。

**2）坐标变换信息**

move_group 通过 ros 的 TF 库获得机器人的坐标变换信息。据此 move_group 可以获得机器人各部分的姿态。注意，MoveIt 只监听 TF，因此要保证 robot_state_publisher 处于运行状态。

**3）控制器接口**

move_group 通过 FollowJointTrajectoryAction 接口和机器人控制器进行交互，运行在机器人上的 action server 提供此接口，move_group 本身不提供此接口。move_group 启动时会初始化一个 action client 和机器人上的 action server 进行通讯。

**4）规划场景**

move_group 使用规划场景监视器维护一个规划场景，它不仅表示当前的环境状态，也包含机器人当前的状态信息。机器人的状态信息里包括机器人当前携带了什么物体，这些物体被认为是和机器人是一个整体，规划场景部分会详细介绍这些功能。

**5）可扩展能力**

move_group 被设计为很容易扩展，单个功能，比如 pick place，运动学，运动规划等实际上是以插件的形式提供的，这些插件共用一个基类。插件可以通过一系列的 ROS 参数及插件库进行配置。绝大部分用户不必去配置 move_group 插件，它们通常在 launch 文件中自动配置好了。

### 2.1.3 运动规划

运动规划的具体内容为：

**1）运动规划插件**

MoveIt 通过插件接口和运动规划器交互，这样 MoveIt 很容易使用不同类型的规划器，使之更有扩展性。同运动规划器交互的接口是通过 ros action 或 service 实现的，默认的规划器是 OMPL。

**2）运动规划请求**

运动规划请求声明了您想要规划器做些什么，比如说，您可以请求规划器将机械臂移动到不 同的位置（在关节空间或笛卡尔空间）。默认会执行碰撞检测功能（含自身碰撞检测）。您也可以把一个物体附加到末端执行器上或机器人的其他部位，比如当机器人拿起一个物体时，这样运动规划时，也会把该物体考虑进去。您也可以声明一些运动约束条件，MoveIt 提供的内置的规划约束是运动学约束，包括：

**位置约束**-把连杆位置约束在特定区域内；

**方位约束**-约束连杆的roll，pitch，yaw 等绕轴的旋转；

**可视性约束**-约束连杆上的一个点处于特定传感器的视锥范围内；

**关节约束**-约束关节转角最小最大值；

**用户声明的约束**-用户自定义的其他约束。

### 3）运动规划结果

运动规划将会产生对应于规划请求的运动轨迹，该轨迹可以让机器人运动到指定位置。注意的是，由move_group 产生的轨迹不仅仅是一条路径，move_group 会根据速度和加速度限制产生满足关节约束的轨迹。

### 4）运动规划流程

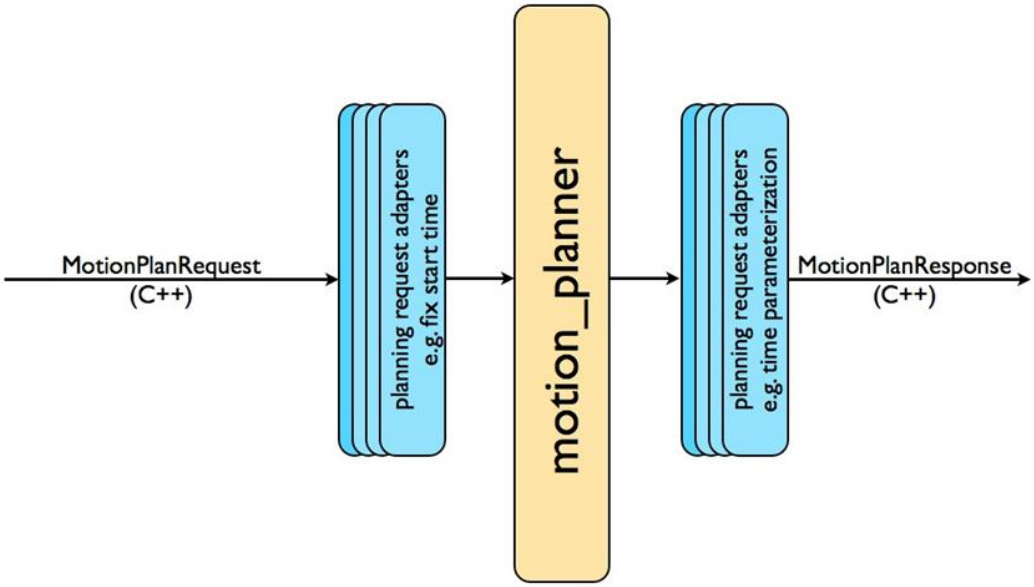



**图 2-2 运动规划流程**

上图表示了运动规划器和规划请求适配器的系统工作。规划请求适配器可以对规划请求进行预处理，也可以对规划响应进行后期处理。预处理在有些情况下是很有用的，比如机器人的初始状态略微超出了关节转角限制。有些操作也需要后期处理，比如把规划的路径转变成基于时间参数的轨迹。MoveIt 提供了一系列的规划适配器用于完成一些特定功能。

**起始状态适配器**。此适配器让机器人起始状态处于关节限制的范围之内。比如当机器人的关节限制没有配置好，机器人一个或多个关节超出关节限制，此时运动规划器将无法求解。此适配器可以将起始状态设置在关节转角限制范围内，当然这不是非常正确的方法，比如，某些情况下关节真的超出限制很多。有一个参数可以设置偏离关节限制达到多少时，才采用此适配器进行修复。

**工作空间适配器**。声明机器人的工作空间，默认是 10m x 10m x 10m 的立方体空间。此工作空间只在规划请求中不含这些限制时声明。

**起始状态碰撞适配器**。此适配器用于机器人起始状态处于碰撞中时，当机器人的起始状态处于碰撞时，可以通过 jiggle_factor 参数让机器人起始状态进行微小移动，另外一个参数可以声明在放弃之前的随机扰动次数。

**起始状态路径约束适配器**。此适配器工作在起始状态不遵循路径约束，它将会规划一条机器人当前状态到满足路径约束位置的路径。新的位置将作为规划的起始状态。

**时间参数化**。运动规划器通常只产生运动学上的路径，这个路径一般不符合速度和加速度约束。此适配器将根据速度和加速度的约束对规划的路径进行时间参数化。

### 5）OMPL

OMPL（Open Motion Planning Library）是一个开源的运动规划库，主要实现了一些随机规划算法。MoveIt 直接集成了 OMPL 并将它作为默认的运动规划器。OMPL 中的规划器是抽象的，它没有机器人的概念。MoveIt 将它配置为后端，用它解决机器人问题。

### 2.1.4 规划场景

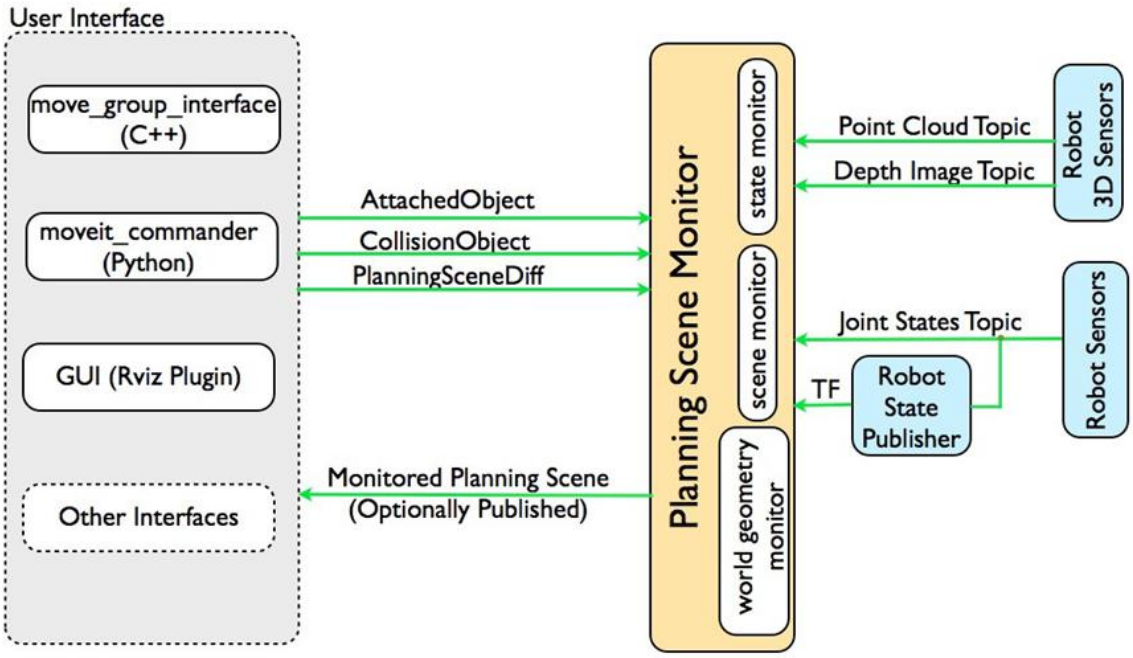



**图 2-3 规划场景图**

规划场景用于表示机器人所处的环境及机器人自身状态，它由 move_group 内部的场景监视器维护管理，场景监视器主要监听：

状态信息：joint_states topic

传感器信息：使用下述的空间几何监视器

空间几何信息：来自于用户输入

**1）空间几何监视器**

空间几何监视器根据机器人上的传感器数据和用户输入信息建立机器人周围的空间几何模型。它用下述的 occupancy_map_monitor 建立机器人当前环境的三维模型，并通过 planning_scene 完善该模型。

**2）三维感知**

MoveIt 的三维感知是由 occupancy_map_monitor 处理的，occupancy_map_monitor 使用插件 的机制处理不同类型的传感器数据，MoveIt 默认支持两种数据输入：

点云：由 point_cloud_occupancy_map 插件处理；

深度图像：由 depth_image_occupancy_map 插件处理。

您也可以自己添加插件。

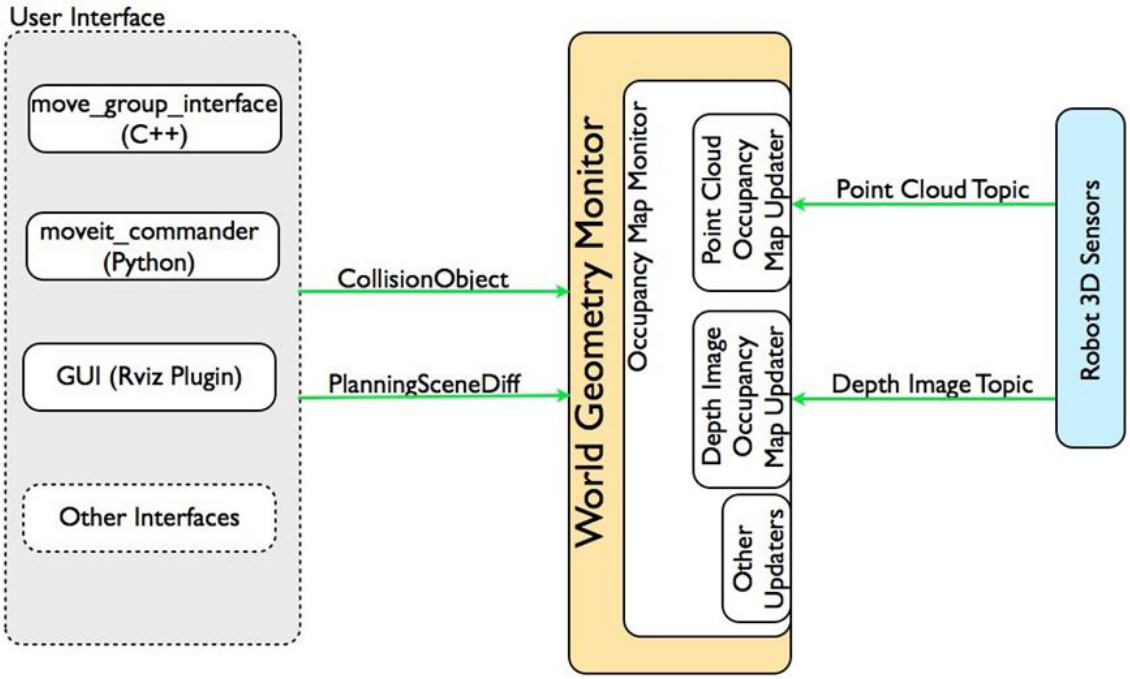



**图 2-3 三维感知**

### 2.1.5 运动学求解与碰撞检测

**1）运动学插件**

MoveIt 运动学求解也是使用插件机制，可以让用户自己写运动学解算算法。正运动学及雅克比集成在 RobotState 类中，默认的逆运动学求解插件是基于雅克比的数值求解器。它在生成运动规划包时已经自动配置好了。

**2）IKFast 插件**

通常，我们会实现自己的运动学求解器，比如 PR2 就有自己的运动学求解器。一个常用的做法是使用 IKFast 软件包产生自己机器人运动学求解器的 C++代码。

**3）碰撞检测**

MoveIt 的碰撞检测功能集成在 planning scene 的 CollisionWorld 中，默认情况下用户不必关心碰撞检测。MoveIt 的碰撞检测功能是通过 FCL 实现的。

**4）碰撞物体**

MoveIt 支持几种不同类型的物体，包括：

网格

基本形状，比如圆球，圆柱，立方体

Octomap，Octomap 对象的物体可以直接用于碰撞检测

**5）FCL**

FCL（Flexible Collision Library）是 MoveIt 所用的主要的碰撞检测库。

**6）碰撞矩阵（ACM）**

碰撞检测是一项很耗费电脑资源的任务，通常占用了运动规划 90%左右的计算开销。ACM 通过二进制编码找到需要进行碰撞检测的两个物体，在 ACM 中如果两个物体的值为 1，表示这两个物体间不需要进行碰撞检测，可能因为这两个物体离的很远不可能发生碰撞。

### 2.1.6 轨迹处理

规划器通常只产生所谓的路径，此路径不包含时间信息。MoveIt 内有轨迹处理例程，可以根据最大速度、最大加速度的限制，将路径转变成包含时间信息的轨迹。关节限制的信息在 joint_linits.yaml 中声明。

## 2.2 MoveIt 接口

本节介绍 MoveIt 的相关接口。

### 2.2.1 配置 Baxter 运动规划包

MoveIt 提供了一个图像界面，您可以通过此图形界面配置您机器人的运动链等，以和 MoveIt 交互。它最主要的功能是为机器人生成一个 SRDF（Semantic Robot Description Format）文件。另外，它也会生成诸如控制器管理器，运动学配置的配置文件和一系列的 launch 文件。我们将以 Baxter 为例，学习如何配置运动规划包。

**1）启动设置助手**

```
$ roslaunch  moveit_setup_assistant  setup_assistant.launch
```

设置助手启动时有两个选项，配置已有的 MoveIt 包和建立新的 MoveIt 包。这里我们选择建立新的 MoveIt 包。

建立新的规划包时要加载 URDF 文件，选中 Baxter 的 URDF 文件所在路径，比如../baxter_description/urdf/baxter.urdf，然后加载。

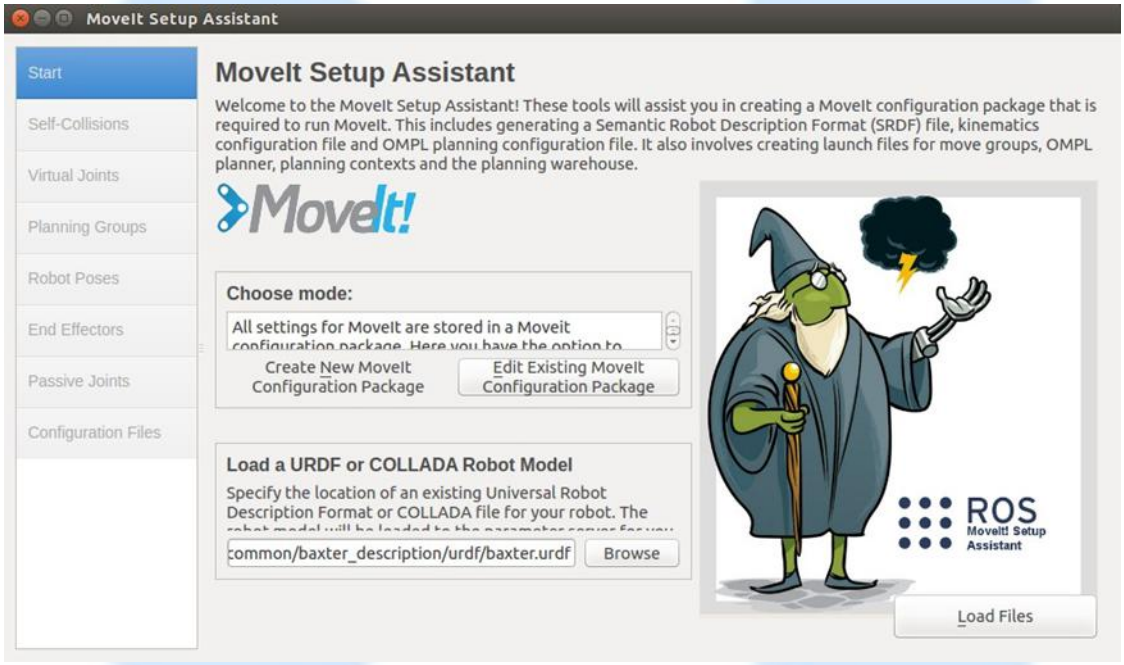



**图 2-4 加载 MoveIt**

装载成功后会显示 Baxter 的三维模型：

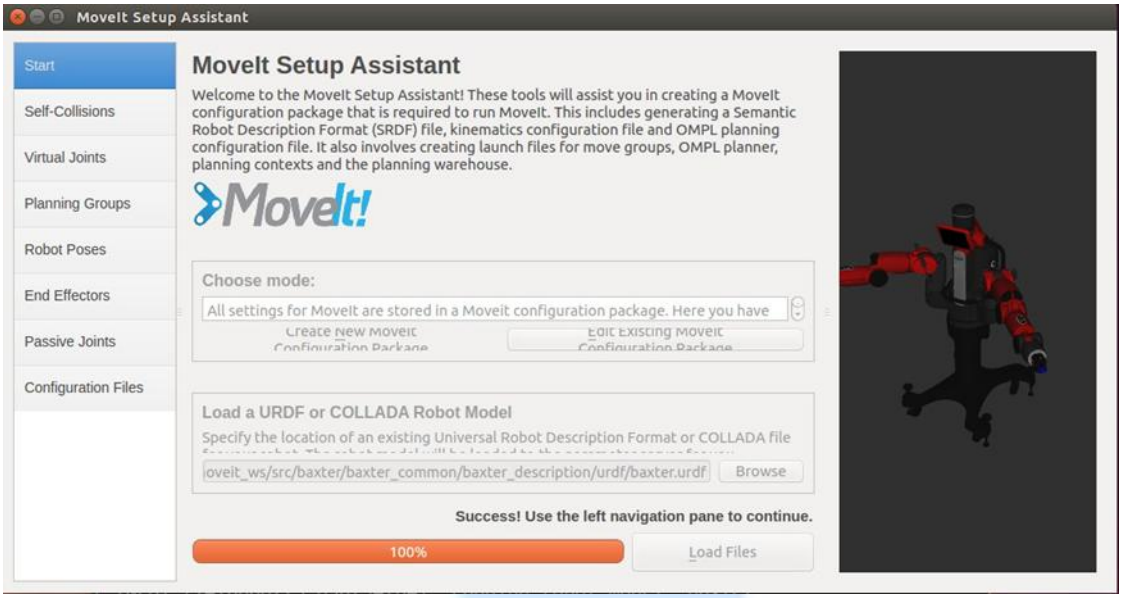



**图 2-5 显示 Baxter 三维模型**

### 2) 生成自我碰撞检测矩阵

为了降低碰撞检测的开销，可以预先生成机器人不同连杆之间的碰撞关系，一些不会发生碰撞的连杆可以禁用碰撞检测。那些总是碰撞、从不碰撞、相邻的连杆之间的碰撞检测通常会被取消。采样密度声明了机器人随机自我碰撞检测的采样次数，更高的密度需要较长的计算时间，但能更好的给出碰撞检测结果。默认的是采样 10000 次，碰撞检测是并行的。

选中 Self_Collisions 面板，选择采样密度，点击 Regenerate Default Collision Matrix，等待几十 秒，则可以生成碰撞检测矩阵。

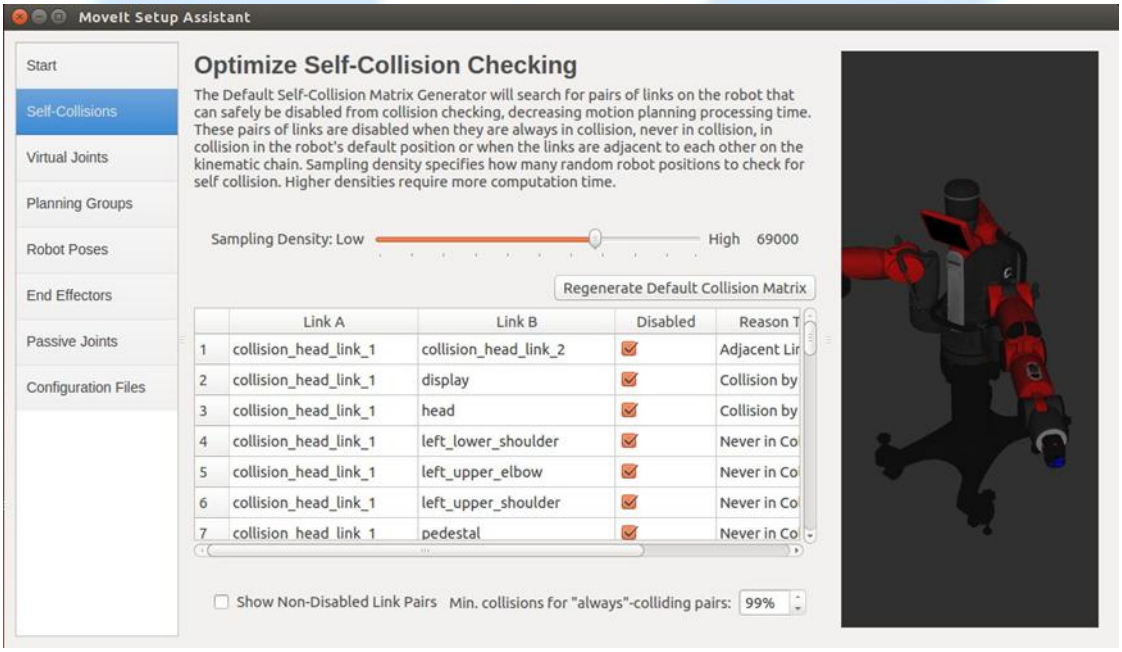



**图 2-6 生成碰撞检测矩阵**

### 添加虚拟关节

虚拟关节表示机器人和世界的关系，比如带移动平台的机器人，可以在地面运动，它和地面之间可以设置一个平面类型的虚拟关节。Baxter 的虚拟关节可以设置为固定，或其他类型，不会对 Baxter 产生实质影响，这里添加了一个浮动虚拟关节。

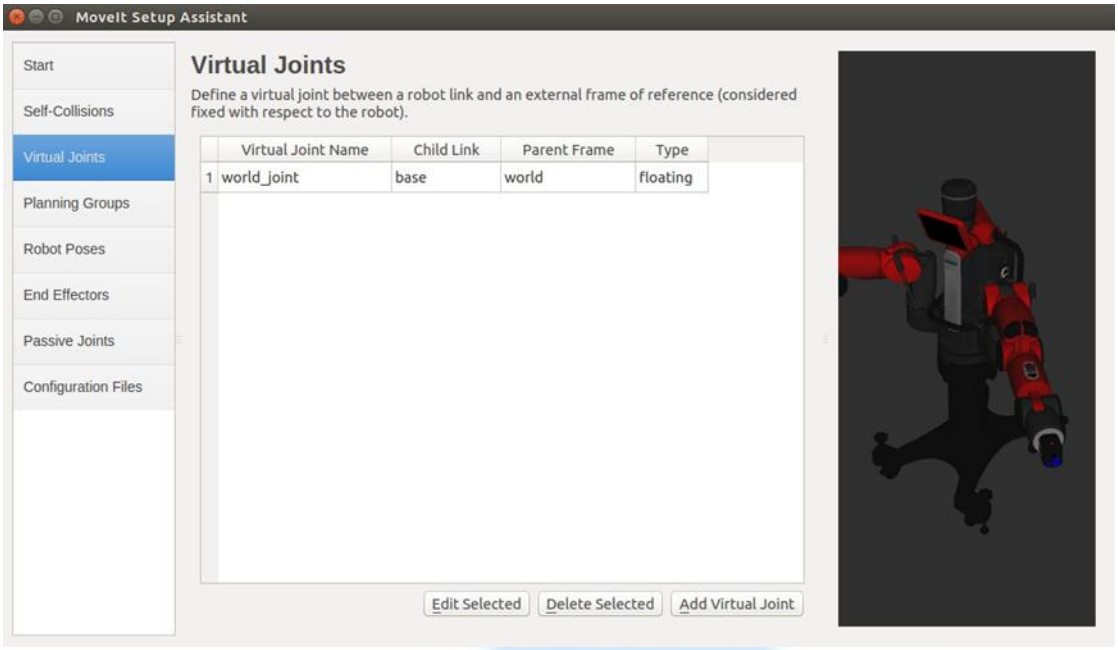



**图 2-7 添加浮动虚拟关节**

### 配置运动规划组

运动规划组是运动规划时的所需要用到的运动链，可以是一组关节，也可以配置为从机械臂第一个连杆到最后一个连杆，也可以是一些子规划组。运动规划组配置时一般要选择机械臂相应的关节，给规划组配置运动求解器、规划时间、规划次数等。

我们为 Baxter 设置了三个规划组，分别是 left_arm，right_arm，both_arms。

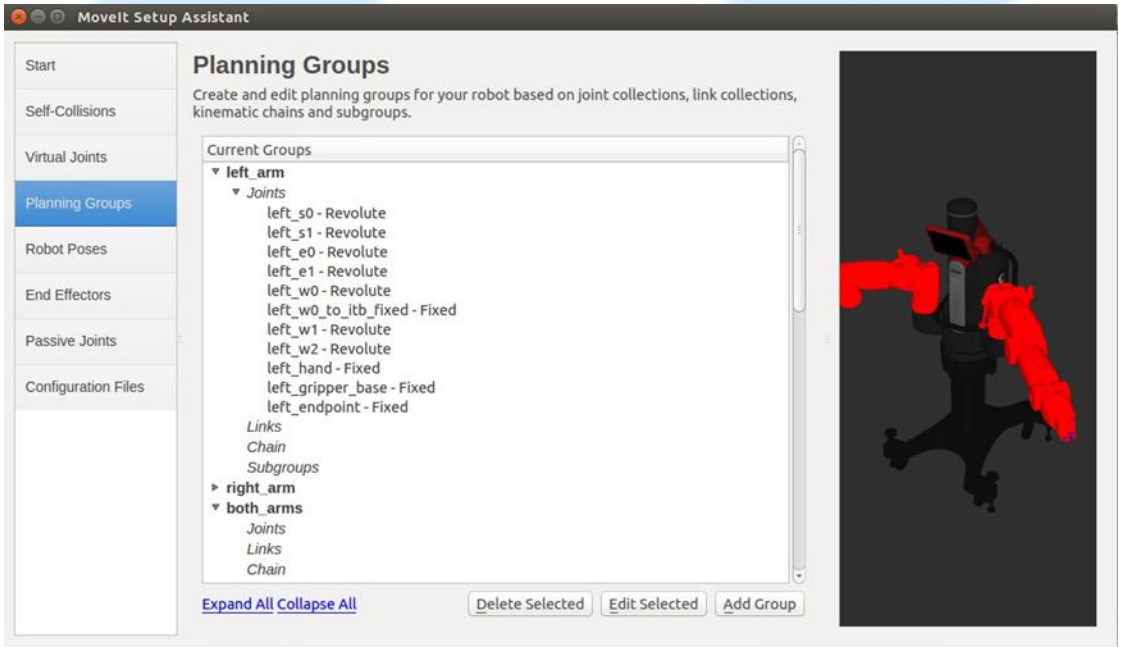



**图 2-8 设置运动规划组**

### 5）定义机器人姿态

可以给机器人预先定义一些姿态，方便运动规划时直接调用，比如为 Baxter 的左臂设置中间姿态。

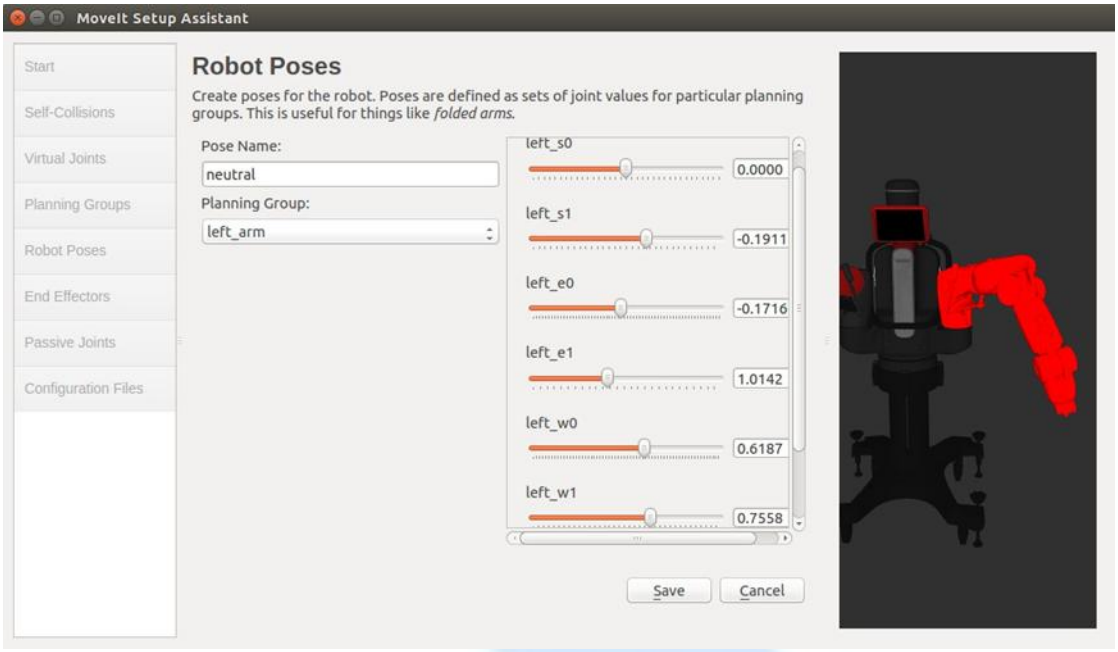



**图 2-9 定义机器人姿态**

**6）生成配置文件**

Baxter 机器人没有被动关节，因此我们直接生成规划配置文件。选择您的工作空间，填入规划包的名字，一般是 robotname_moveit_config 点击 Generate Package 即可为我们的机器人生成一系列的配置和启动文件。

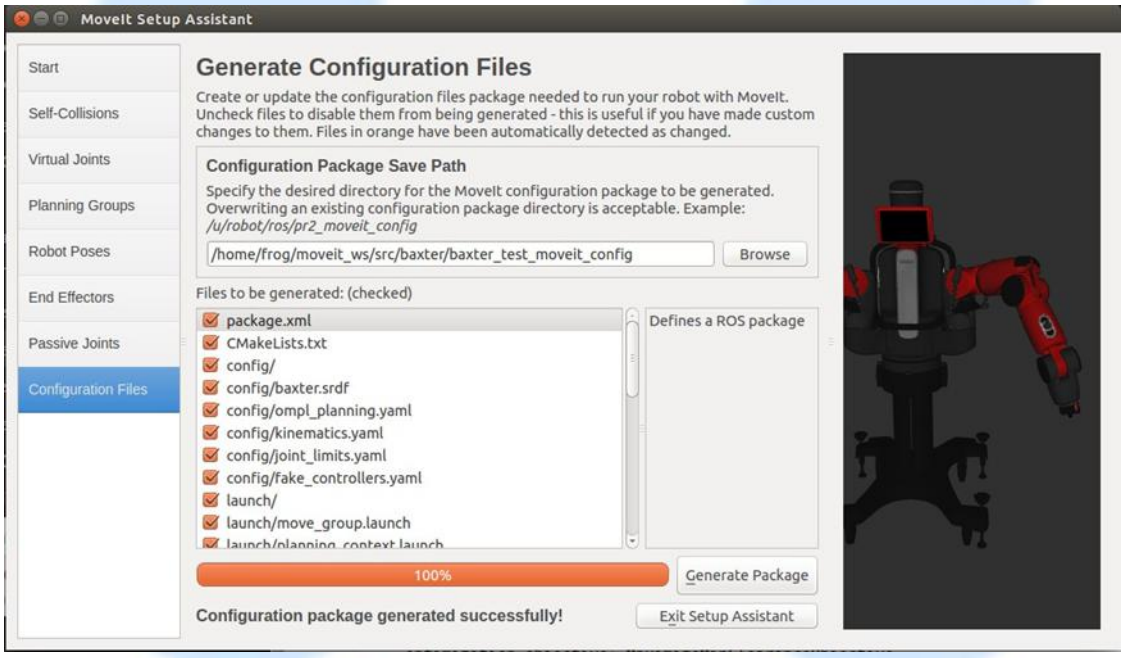



**图 2-10 生成配置和启动文件**

**验证规划包**

启动：

```
$ roslaunch baxter_moveit_config demo.launch
```

如下图所示，您将看到 Baxter 机械臂末端有彩色的圆圈和箭头，代表机械臂末端的平动

和转动自由度，至此，Baxter 运动规划包已经配置完成了。

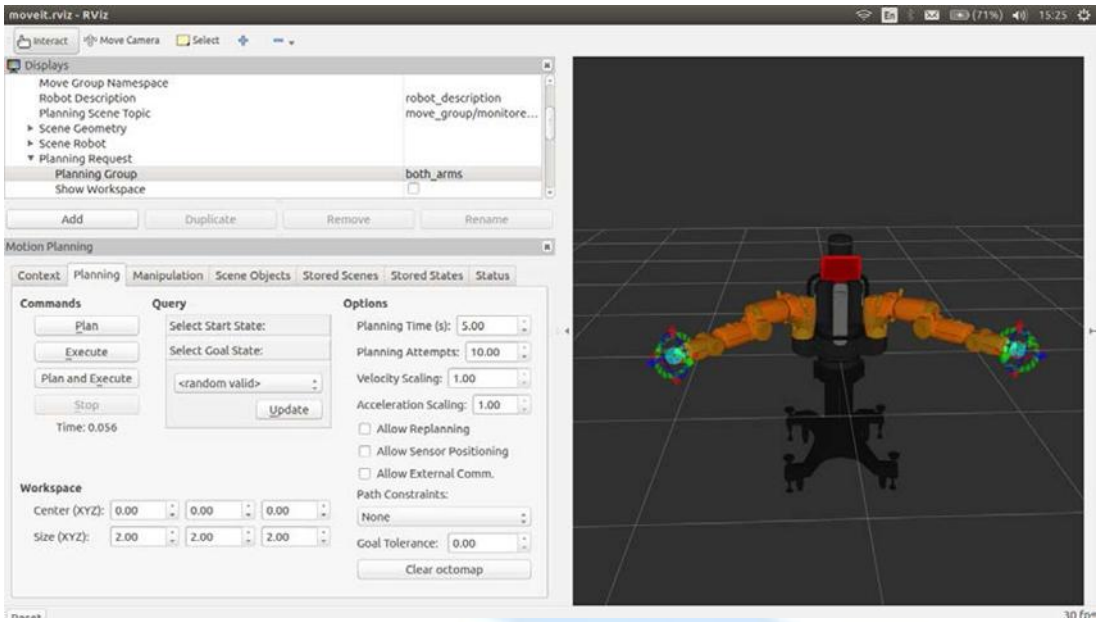



**图 2-11 运动规划包配置完成**

### 2.2.2 move_group 接口

和 MoveIt 交互式主要是通过 move_group 的 C++、Python API 或 ROS action，service 以及 topic。move_group 接口提供了大量针对运动规划的操作，比如设置关节目标，设置机械臂末端位置，创建运动规划，移动机器人等。

这里以Python 接口为例，介绍如何使用MoveIt 的功能。

**1) 导入 Python 模块**

```
import sys
import copy
import rospy
import moveit_commander
import moveit_msgs.msg
import geometry_msgs.msg
```

**2) 初始化 moveit_commander 及 rospy**

```
print "============ Starting tutorial setup"
moveit_commander.roscpp_initialize(sys.argv)
rospy.init_node('move_group_python_interface_tutorial', anonymous=True)
```

**3) 实例化一个 RobotCommander 对象，用于和机器人交互**

```
robot = moveit_commander.RobotCommander()
```

**4) 实例化一个 PlanningSceneInterface 对象，它代表了机器人周围的环境**

```
scene = moveit_commander.PlanningSceneInterface()
```

**5) 实例化一个 MoveGroupCommander 对象**

该对象代表了一个运动规划组，在这里我们实例化左臂。此接口可用于在左臂上进行路径规

划，并执行规划的路径。

```
group = moveit_commander.MoveGroupCommander("left_arm")
```

**6) 在 Rviz 中显示规划的路径**

```
display_trajectory_publisher = rospy.Publisher('/move_group/display_planned_path',
                                               moveit_msgs.msg.DisplayTrajectory)
print "============ Waiting for RVIZ..."
rospy.sleep(10)
print "============ Starting tutorial "
```

**7) 获得基本信息**

获得机器人参考坐标系：

```
print "============ Reference frame: %s" % group.get_planning_frame()
```

打印终端效应器的名字:

```
print "============ Reference frame: %s" %group.get_end_effector_link()
```

获得机器人所有的规划组：

```
print "============ Robot Groups:"
print robot.get_group_names()
```

获得机器人当前状态：

```
print "============ Printing robot state"
print robot.get_current_state()
print "============"
```

**8) 规划到目标点**

设置目标点：

```
print "============ Generating plan 1"
pose_target = geometry_msgs.msg.Pose()
pose_target.orientation.w = 1.0
pose_target.position.x = 0.7
pose_target.position.y = -0.05
pose_target.position.z = 1.1
group.set_pose_target(pose_target)
```

执行规划命令。只请求规划路径，不实际执行，规划成功后会在Rviz 中显示出来：

```
plan1 = group.plan()
print "============ Waiting while RVIZ displays plan1..."
rospy.sleep(5)
```

发布规划好的路径，以方便在Rviz 中查看：

```
print "============ Visualizing plan1"
display_trajectory = moveit_msgs.msg.DisplayTrajectory()

display_trajectory.trajectory_start = robot.get_current_state()
display_trajectory.trajectory.append(plan1) display_trajectory_publisher.publish(display_trajectory);

print "============ Waiting while plan1 is visualized (again)..."
 rospy.sleep(5)
```

**9) 将机器人移动到目标点**

这个过程和上面的过程类似，我们调用的是go()函数，这个函数有个wait 参数，设置为True 时，程序会阻塞指导运动完成为止，失败则直接返回。

```
# Uncomment below line when working with a real robot
# group.go(wait=True)
```

**10) 关节空间运动规划**

除了指定机械臂的末端位置外，也可以设置关节转角值。

首先清除之前设置的目标值：

```
group.clear_pose_targets()
```

获得当前关节转角值：

```
group_variable_values = group.get_current_joint_values()
print "============ Joint values: ", group_variable_values
```

修改其中的一个关节值，执行规划命令：

```
group_variable_values[0] = 1.0
group.set_joint_value_target(group_variable_values)

plan2 = group.plan()

print "============ Waiting while RVIZ displays plan2..."
 rospy.sleep(5)
```

**11) 笛卡尔路径规划**

可以声明一系列的路点让机器人沿这些路点形成的路径运动：

```
waypoints = []

# start with the current pose
waypoints.append(group.get_current_pose().pose)

# first orient gripper and move forward (+x)
wpose = geometry_msgs.msg.Pose()
wpose.orientation.w = 1.0
wpose.position.x = waypoints[0].position.x + 0.1
wpose.position.y = waypoints[0].position.y
wpose.position.z = waypoints[0].position.z
waypoints.append(copy.deepcopy(wpose))

# second move down
wpose.position.z -= 0.10
waypoints.append(copy.deepcopy(wpose))

# third move to the side
wpose.position.y += 0.05
waypoints.append(copy.deepcopy(wpose))
```

如果想让机械臂末端沿路径以1cm 的间距插值，我们可以设置jump_threshold 的值0.01，这里设置为0.0，禁用它。

```
(plan3, fraction) = group.compute_cartesian_path(
                                   waypoints,          # waypoints to follow
                                   0.01,               # eef_step
                                   0.0)                # jump_threshold

print "============= Waiting while RVIZ displays plan3..."
rospy.sleep(5)
```

### 2.2.3 planning scene 接口

这节我们介绍如何通过planning scene 接口向规划场景中添加或移除物体。

**1) 设定 Publisher**

可以通过ROS 特定的diffs topic 和规划场景交互，一个规划场景的diff 指的是当前规划场景和新的规划场景之间的差异。

```
ros::Publisher planning_scene_diff_publisher =
node_handle.advertise<moveit_msgs::PlanningScene>("planning_scene", 1);
while(planning_scene_diff_publisher.getNumSubscribers() < 1)
{
      ros::WallDuration sleep_t(0.5);
      sleep_t.sleep();
}
```

**2) 定义要附加的物体**

定义要添加到规划场景或要从当前场景中移除的物体，以及要附加到机器人上的物体。

```
moveit_msgs::AttachedCollisionObject attached_object;
attached_object.link_name = "r_wrist_roll_link";
/* The header must contain a valid TF frame*/
attached_object.object.header.frame_id = "r_wrist_roll_link";
/* The id of the object */
attached_object.object.id = "box";

/* A default pose */
 geometry_msgs::Pose pose;
pose.orientation.w = 1.0;

/* Define a box to be attached */
shape_msgs::SolidPrimitive primitive;
 primitive.type = primitive.BOX;
primitive.dimensions.resize(3);
primitive.dimensions[0] = 0.1;
primitive.dimensions[1] = 0.1;
primitive.dimensions[2] = 0.1;

attached_object.object.primitives.push_back(primitive);
attached_object.object.primitive_poses.push_back(pose);
```

上面代码中，为了将物体附件到机器人上，需要加一个ADD 的操作:

```
attached_object.object.operation = attached_object.object.ADD;
```

**3) 将物体添加到场景中**

通过将物体添加到场景中的物体碰撞集中实现添加物体，只使用 attached_object 消息的 object 域。

```
ROS_INFO("Adding the object into the world at the location of the right wrist.");
moveit_msgs::PlanningScene planning_scene;
planning_scene.world.collision_objects.push_back(attached_object.object);
planning_scene.is_diff = true;
planning_scene_diff_publisher.publish(planning_scene);
sleep_time.sleep();
```

**4) 场景的同步及异步更新**

move_group 使用场景 diff 时，有两种不同的机制：

通过 rosservice 发送 diff 请求时，场景会阻塞知道 diff 已经应用到场景了（同步更新）；

通过 topic 发送 diff 消息，消息会继续发送而不管 diff 是否已经应用到场景了（异步更新）。

大部分的教程使用的是后一个机制（发布 diff 消息后，设置一定的休眠时间），可以用下面 的调用服务的方法较好的替代 topic 的方式。

```
ros::ServiceClient planning_scene_diff_client =node_handle.serviceClient<moveit_msgs::
                                    ApplyPlanningScene>("apply_planning_scene");
planning_scene_diff_client.waitForExistence();
moveit_msgs::ApplyPlanningScene srv;
srv.request.scene = planning_scene;
planning_scene_diff_client.call(srv);
```

**5）将物体附加到机器人上**

当机器人拿起一个物体时，我们需要把该物体附加到机器人上，这样执行运动规划时，规划 器会把该物体作为机器人的一部分考虑进去。

将物体附加到机器人上有两个步骤，一是把它从环境中移除，再是把它附加到机器人上。

```
/* First, define the REMOVE object message*/
moveit_msgs::CollisionObject remove_object;
remove_object.id = "box";
remove_object.header.frame_id = "odom_combined";
remove_object.operation = remove_object.REMOVE;
```

要先清除diff 消息中的已附加物体和碰撞物体。

```
/* Carry out the REMOVE + ATTACH operation */
ROS_INFO("Attaching the object to the right wrist and removing it from the world.");
planning_scene.world.collision_objects.clear();
planning_scene.world.collision_objects.push_back(remove_object);
planning_scene.robot_state.attached_collision_objects.push_back(attached_object);
planning_scene_diff_publisher.publish(planning_scene);sleep_time.sleep();
```

**6) 解除机器人附加的物体**

解除附加到机器人上的物体时同样也需要两步，一是从机器人上解除附加，再是把物体插入到环境中。

```
/* First, define the DETACH object message*/
moveit_msgs::AttachedCollisionObject detach_object;
detach_object.object.id = "box";
detach_object.link_name = "r_wrist_roll_link";
detach_object.object.operation = attached_object.object.REMOVE;
```

同样要先清除diff 消息中已附加的物体和碰撞物体。

```
/* Carry out the DETACH + ADD operation */
ROS_INFO("Detaching the object from the robot and returning it to the world.");
planning_scene.robot_state.attached_collision_objects.clear();
planning_scene.robot_state.attached_collision_objects.push_back(detach_object);
planning_scene.world.collision_objects.clear();
planning_scene.world.collision_objects.push_back(attached_object.object);
planning_scene_diff_publisher.publish(planning_scene);
```

```
sleep_time.sleep();
```

**7）将物体从规划场景中移除**

将物体从规划场景中移除时，只需要用到之前定义的移除物体的消息，同样注意，清除 diff 消息中含已附加物体和碰撞物体的部分，以免误操作这些物体。

```
ROS_INFO("Removing the object from the world.");
planning_scene.robot_state.attached_collision_objects.clear();
planning_scene.world.collision_objects.clear();
planning_scene.world.collision_objects.push_back(remove_object);
planning_scene_diff_publisher.publish(planning_scene);
```

此章将向您介Baxter 的基本工作原理及基于SDK 的编程基础。

# 第三章 Baxter 工作原理与 SDK 开发

## 3.1 Baxter 基本工作原理

此节主要对 Baxter 的工作原理进行详细介绍。

### 3.1.1 机械臂控制概述

Baxter 机械臂有四个控制层级：

1）用户层：用户可以通过 SSH 或 Python API 的方式向机械臂发送控制指令；

2）关节指令监听器：监听 ROS 主题（运行在 Baxter 内部 gentoo linux 上）；

3）实时电机控制循环：运行于 Baxter 内部 gentoo 上，具有最高控制优先级；

4）关节控制器：机械臂关节电机控制器。

第 2~4 层级运行于 Baxter 内部 PC 或关节控制器上，用户不可修改。Baxter 机械臂关节有三种控制模式：位置模式、速度模式、力矩模式。每种模式的控制器都是电机控制器插件的子集。默认情况下，Baxter 监听和发布 ROS 消息的频率是 100Hz，实际测试表明，在 800Hz 的控制频率下 Baxter 仍能正常工作。

从接收用户 Python 代码的命令到执行命令的流程是，ROS 网络从 topic 或 action 接受指令并存入消息序列，由实时电机控制循环（1kHz）异步检索并发往电机控制器（JCB）。JCB 的执行命令的时间片是 1ms。在下一个 1ms 的时间片中，关节位置及弹簧形变被读取，随后传递到关节控制发布器，并发布到ROS 网络中去。

控制命令时序分析如下：

1）1.6ms，用户通过 ROS 发布消息到关节控制器；

2）1ms，关节控制器到电机控制器，异步实时循环（1Khz）；

3）1ms，电机控制器将命令传递到 JCB 并执行；

4）1ms，JCB 读取关节状态并反馈到电机控制器；

5）1ms，电机控制器报告给关节控制器消息发布器；

6）1.6ms，关节消息发布器发布消息到 ROS 网络。

整个命令消息发送、状态反馈的过程耗时约 7.2ms。

实时电机控制器的执行时间约 1ms，平均实际执行时间约 700us，有时执行时间会超出 100us， 执行超时不会传播，也不会影响系统的实时性，因为下个控制周期耗时将是 700us。

可以通过 ros 的诊断机制查看电机控制器的状态，"rostopic echo/diagnostics" ，找到 "RealtimeControlLoop" 部分。

实时电机控制器有一些列的插件，比如碰撞检测、重力补偿、弹簧回滞补偿等。在这些插件之上，当压按腕部的感应按钮时，机械臂会进入零重力模式，允许机械臂进行反向驱动。释放感应开关后，电机控制器返回到之前的控制模式。

### 3.1.2 机械臂控制模式

关节控制模式定义了 Baxter 关节的控制方式，控制命令格式定义在 baxter_core_msgs 包中 JointCommand.msg 消息内，可以通过 Ros 的 rosmsg show 命令查看命令的格式。

控制机械臂时，我们可以向相应的 rostopic 发送 JointCommand 类型的消息：

```
robot/limb/<left/right>/joint_command
```

默认控制命令发布频率应大于5Hz，可通过下面的topic查看命令超时时间：

```
/robot/limb/<left/right>/joint_command_timeout
```

控制命令消息格式：

```
int32 mode
float64[] command
string[] names

int32 POSITION_MODE=1
int32 VELOCITY_MODE=2
 int32 TORQUE_MODE=3
int32 RAW_POSITION_MODE=4
```

其中：

Mode：控制模式，1-位置控制，2-速度控制，3-力矩控制，4-原始位置控制；

Command：命令值，顺序对应于 name 中的关节名顺序；

Name：关节名。

**1）位置控制模式**

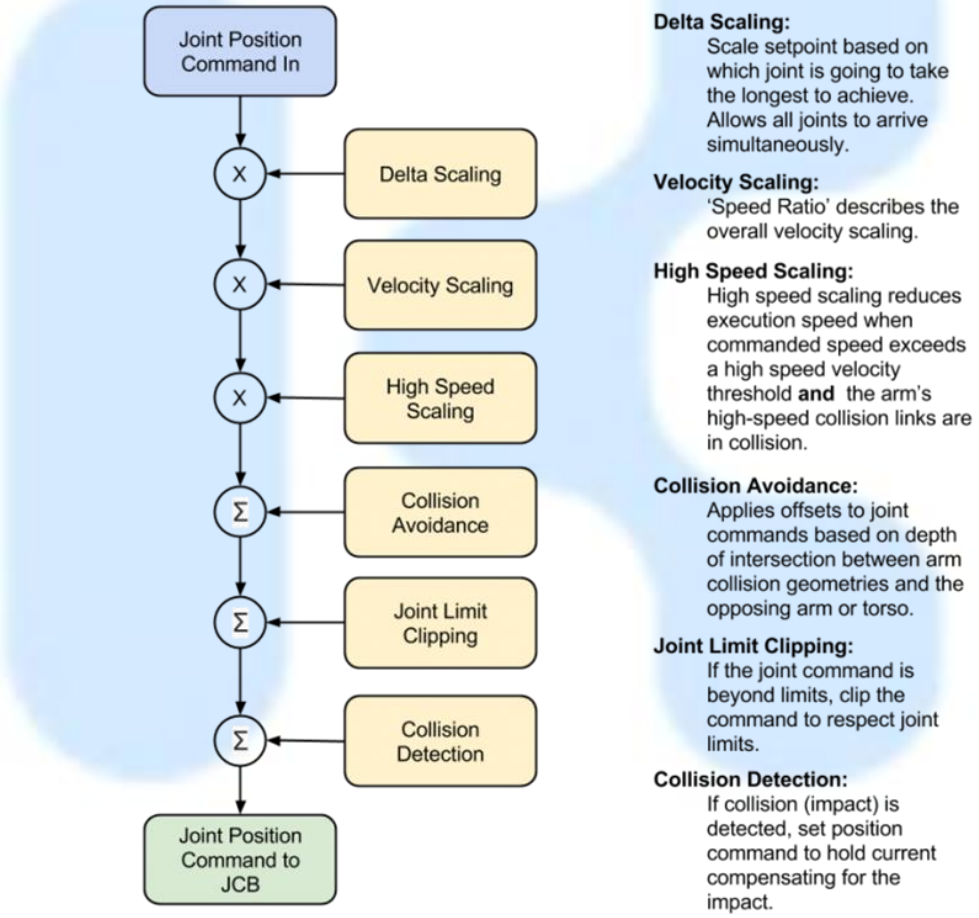



**图 3-1 位置控制模式图**

2）原始位置控制模式

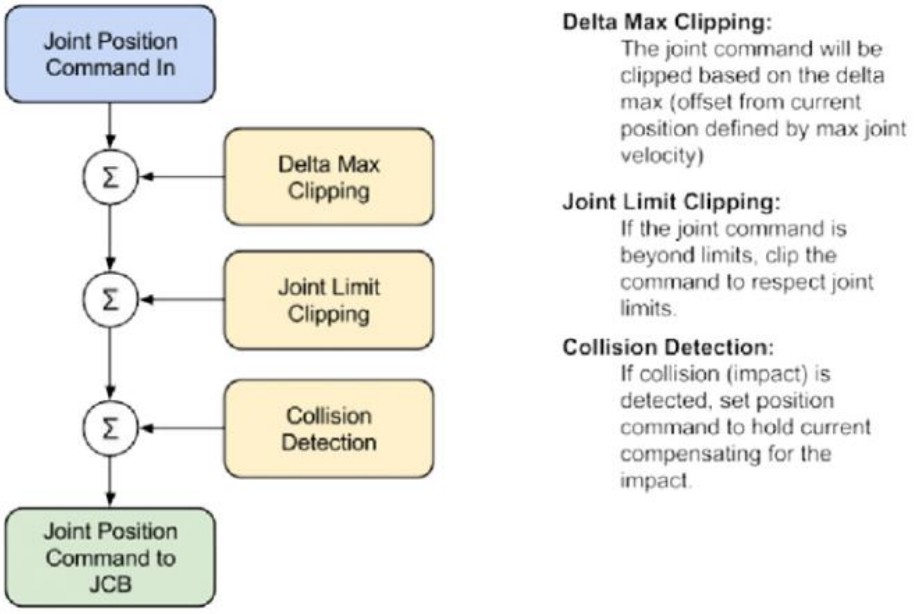



图 3-2 原始位置控制模式图

3）速度控制模式

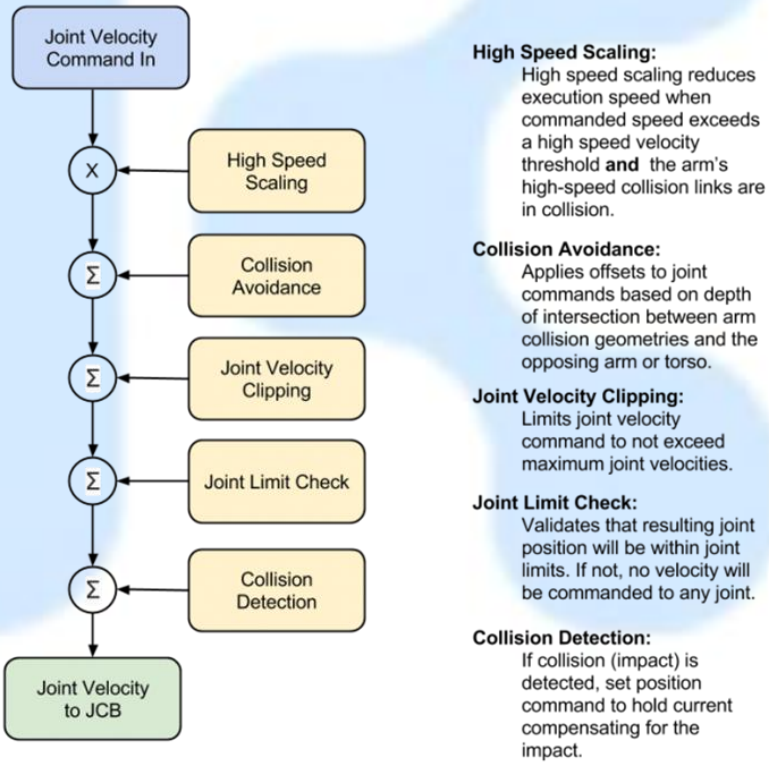



图 3-3 速度控制模式图

4）力矩控制模式

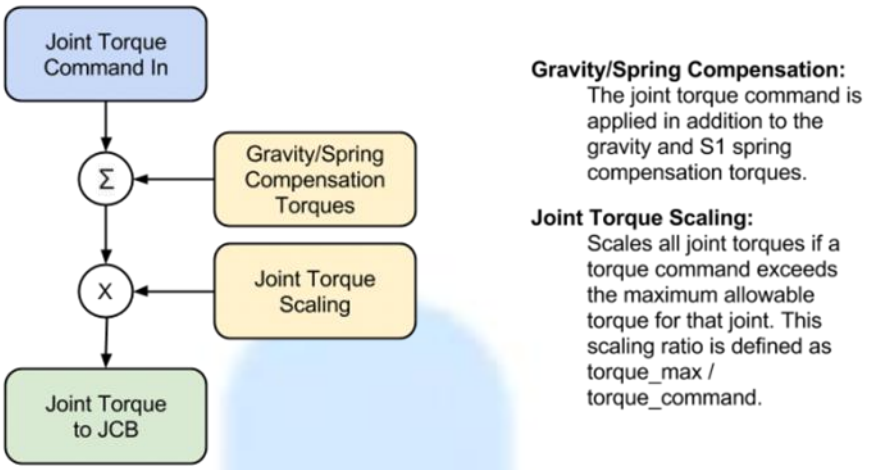



**图 3-4 力矩控制模式图**

如上图所示，在力矩控制模式，重力及弹簧补偿的力矩也被考虑进去了，可以通过向/robot/limb/right/suppress_gravity_compensation 发布频率大于5Hz 的空消息禁用重力补偿。

### 3.1.3 碰撞检测

Baxter 有三个层级的碰撞模型。

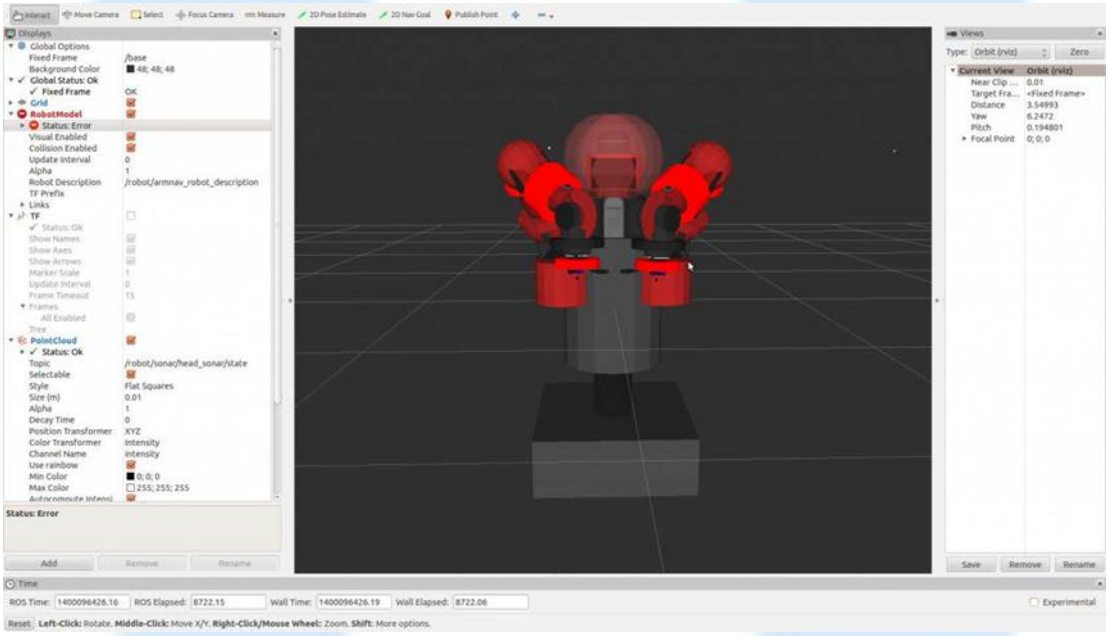


**图 3-5 Baxter 碰撞模型**

如上图所示，Baxter 每个连杆都有它自己的碰撞模型块，碰撞模型比机器人实际模型要大一些，当不同的碰撞模型将要接触时，避障功能将被触发。碰撞模型块可以通过 RVIZ 查看，相应的机器人描述文件是/robot/armnav_robot_description。

另外一类碰撞模型包含两种碰撞检测类型，碰撞和挤压。当关节力矩突然发生变化时，有可能是机械臂碰到了障碍物，比如碰到了一个人。碰撞后机械臂会停下 2 秒，然后在运动。挤压发生在机器人碰到静止的障碍物，比如碰到一堵墙，关节力矩在增加，但机械臂不能运动，当关节力矩超过一个阈值时，机械臂将立即停止运动。机器人的日志系统会记录这些碰撞：

```
2014-05-14T14:45:37.883-
0400INFO(/collision/right/joint_collision_estimator_right,JointCollisionEstimator.cpp:374)impact
:1.380773>0.850000,largestcontributor:right_w0,withvalue:0.651761
```

上述日志表示机器人碰到了障碍物，关节 right_w0 的力矩大于 0.85。同样挤压的消息也可以通过日志系统查看：

```
2014-05-14T14:48:31.723-
0400INFO(/collision/right/joint_collision_estimator_right,JointCollisionEstimator.cpp:395)squish
on Cartesian dimension0-16.177,16.000
```

碰撞消息也可以在rqt_console 中查看：

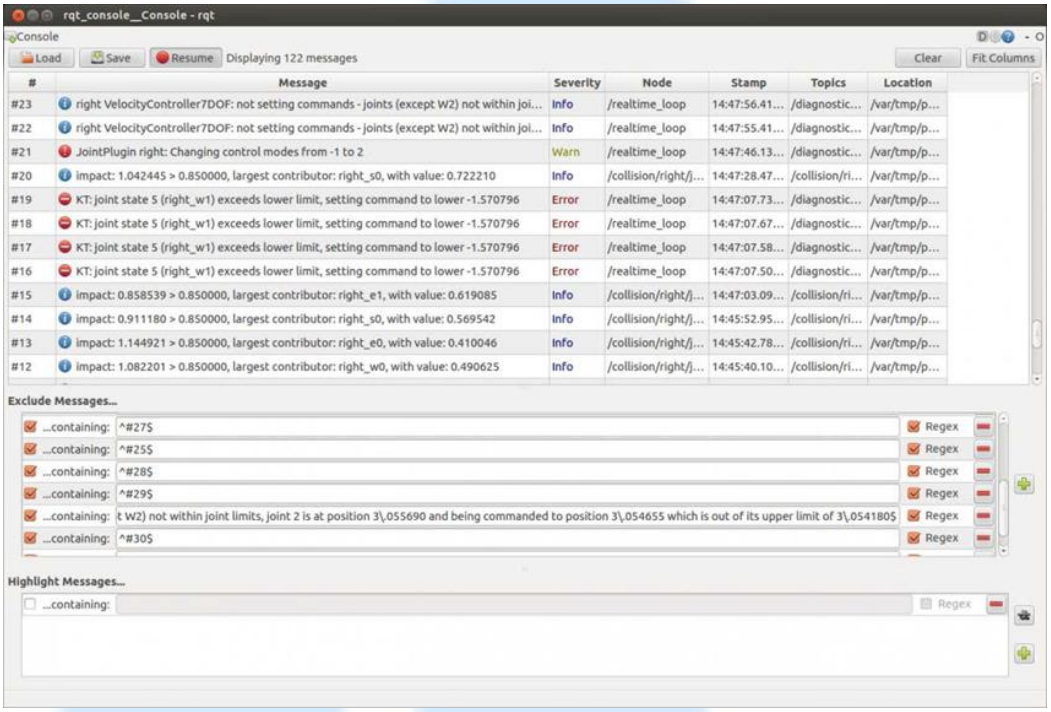



**图 3-6 rqt_console 碰撞消息**

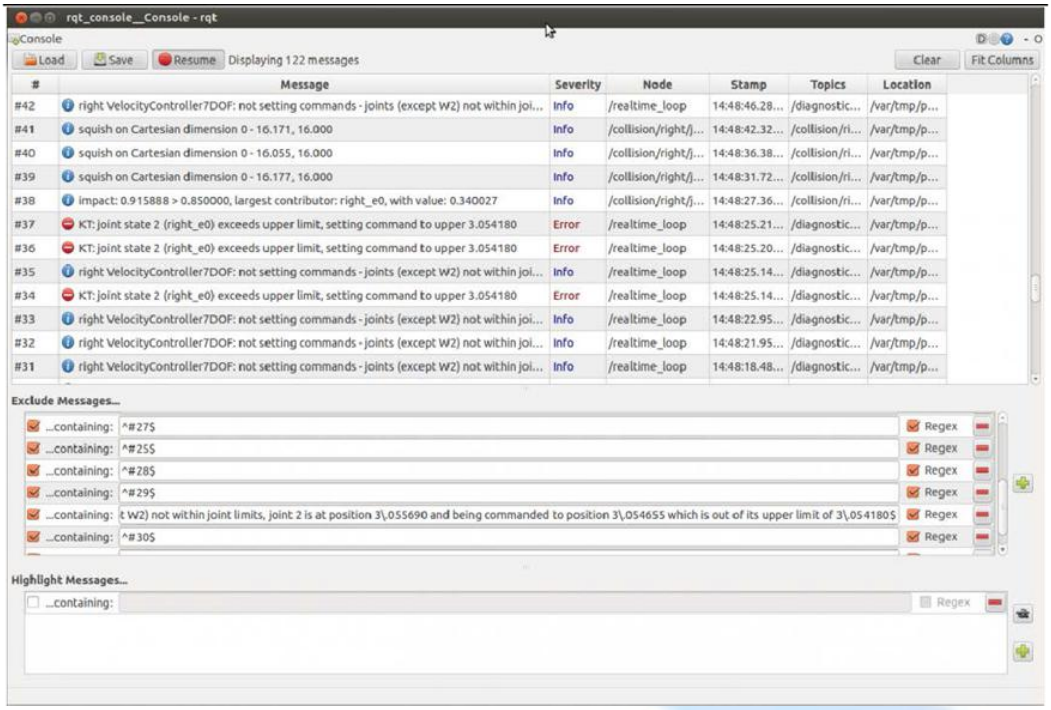



**图 3-7 rqt_console 碰撞消息**

最后一个碰撞模型是，速度缩小。此功能常用避免两个机械臂高速运动时的碰撞，此时碰撞模型块比原始的要大，机械臂在笛卡尔空间中的速度超过 0.2m/s 时，此碰撞被触发，速度被减小并进入位置控制模式。

可以向/robot/limb/<side>/suppress_collision_avoidance 发送空消息禁用碰撞检测功能，比如禁用左臂碰撞检测功能：

```
$ rostopic pub /robot/limb/left/suppress_collision_avoidancestd_msgs/Empty-r10
```

在此消息发布期间，左臂的碰撞检测功能将一直处于被禁用状态。

### 3.1.4 重力补偿

为了抵消重对机械臂产生的影响，Baxter 控制器会对机械臂进行重力补偿。此功能是机械臂的底层基本功能，一旦机器人处于使能状态，该功能即被激活。Baxter 的重力补偿功能使用 KDL 进行正逆动力学解算，S1 关节的弹簧也需要补偿力矩，它将配合 KDL 使用内部的弹簧模型完成关节力矩补偿。不管关节控制器是否处于激活状态，解算的重力补偿力矩都会通过一个单独的通道直接传递到 JCB。

可以通过 robot/limb/<side>/gravity_compensation_torques 查看重力补偿力矩：

```
$ rostopic echo robot/limb/<left/right>/gravity_compensation_torques
```

也可以禁用重力补偿功能：

```
$ rostopic pub –r 10 /robot/limb/<side>/suppress_gravity_compensationstd_msgs/Empty
```

重力补偿消息的格式如下：

```
commanded_position:实时控制循环的位置命令
commanded_velocity:实时控制循环的速度命令
```

commanded_effort:实时控制循环的力矩命令，在力矩控制模式下此命令是您设定的力矩命令，在位置控制模式下它是为了达到该位置所需施加的力

actual_position:实时控制循环测得的关节位置
actual_velocity:实时控制循环测得的关节位速度
actual_effort:实时控制循环根据关节弹簧形变测得的关节力矩
gravity_model_effort:重力补偿力矩
hysteresis_model_effort:弹簧补偿力矩
crosstalk_model_effort:关节串扰补偿
hystState:弹簧迟滞模型参数

### 3.1.5 零重力模式

零重力模式有时候易和重力补偿相混淆，默认情况下，只要机器人被使能，重力补偿功能就会处于激活状态。在零重力模式下，关节控制器被禁用，因此您可以自由的拖动机械臂，此时由重力产生的力矩由电机来补偿抵消，关节控制器由于处于禁用状态将不产生力矩命令，机械臂相当于处于失重状态。

通过按压腕部的感应按钮使机器人进入零重力模式：

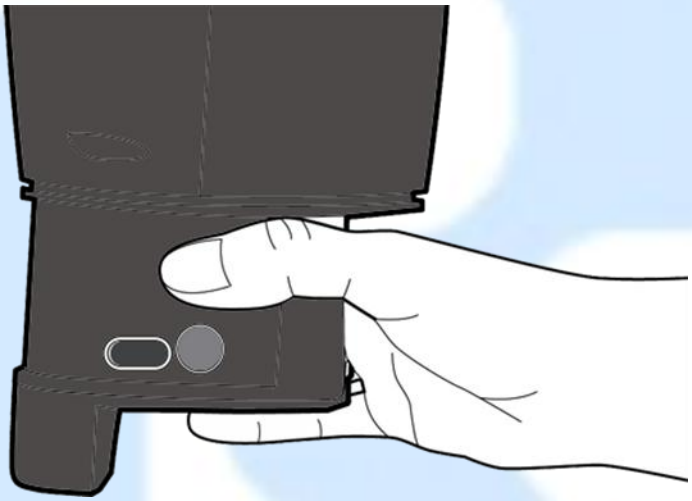


**图 3-8 零重力模式**

不同于重力补偿模式，零重力模式不能被直接禁用，可以通过禁用腕部交互来禁用零重力模式。同样，也是向相应 rostopic 发送频率大于 5Hz 的空消息，如：

```
$ rostopic pub -r 10 /robot/limb/<side>/suppress_cuff_interactionstd_msgs/Empty
```

## 3.2 Baxter 的 SDK

主要介绍 SDK 的布局以及工作站的建立。

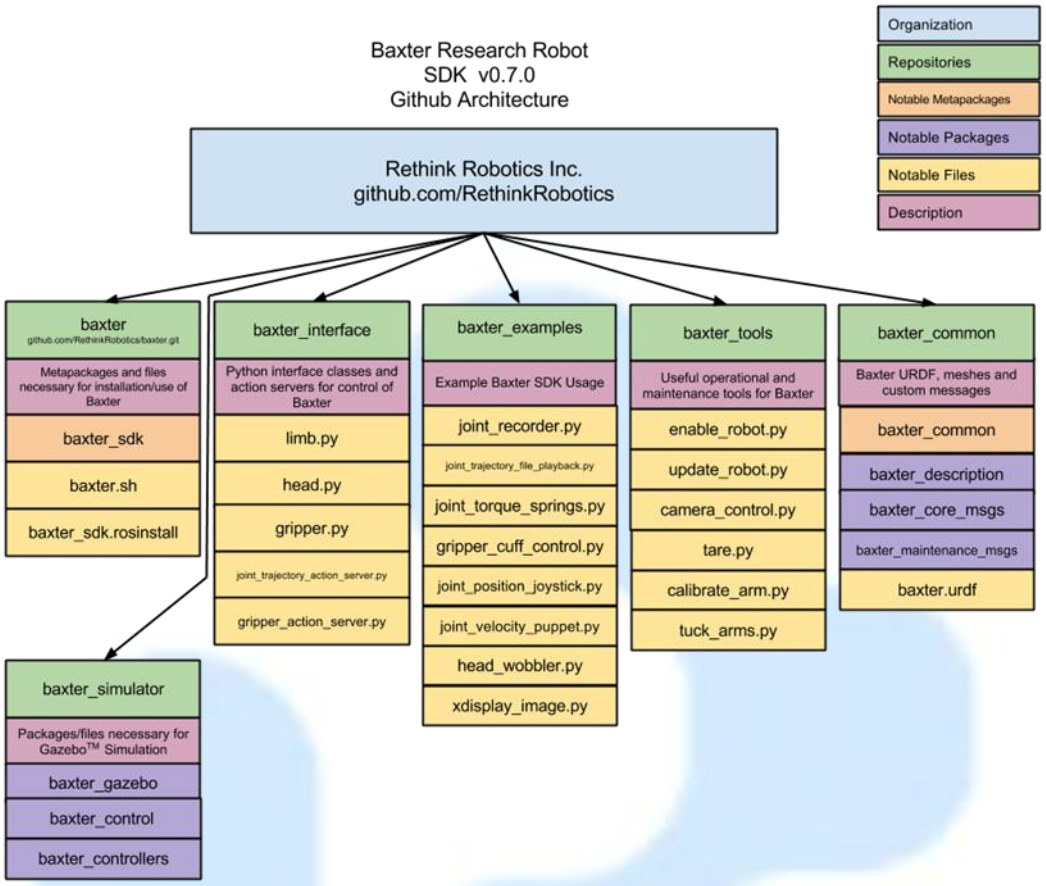



**图 3-9 baxter 的 sdk 布局**

**baxter**

该软件包包含用于安装 baxter_sdk 所需的元数据包和文件，包括 baxter_sdk 元数据包及 SDK 包。也包括创建这个软件包所需要的 rosinstall 文件，baxter.sh 及用于设置 ROS 环境变量的 bash 脚本，旨在帮助用户轻松地设置自己的 ROS 环境并尽快开始使用 baxter。

**baxter_interface**

用于控制 baxter 的 Python 接口类和 Action 服务器。这些 Python 模块直接封装了与 Baxter 通讯所需要的 ROS 通讯功能，使用户能够简单的对 Baxter 进行编程而不必事先熟悉 ROS。此软件包也包括用于控制机械臂运动轨迹和夹持器动作的 Action 服务器。这些标准的 Action 服务器可以和其他的 ROS 工具，比如和 MoveIt！协同工作。

**baxter_tools**

此软件包提供了一系列有用的用于的操作和维护 Baxter 的工具软件。这些脚本可用于日常应用（如机器人开启/关闭，摄像头控制）和维护任务（去皮、校准、更新机器人软件）。

## 3.3 Baxter 应用程序接口

Baxter 应用程序接口，提供了用于和 Baxter 机器人交互的 PythonAPI，它由一系列的封装了 ROS 通讯的类组成，可以直接通过此 PythonAPI 控制机器人不同的接口。

### 3.3.1 可用应用程序接口

模块层级：

baxter_control

baxter_control.pid

baxter_dataflow

baxter_dataflow.signals

baxter_dataflow.wait_for

baxter_dataflow.wait_for'

baxter_dataflow.weakrefset

baxter_interface

baxter_interface.analog_io

baxter_interface.camera

baxter_interface.cfg

baxter_interface.cfg.GripperActionServerConfig

baxter_interface.cfg.HeadActionServerConfig

baxter_interface.cfg.PositionFFJointTrajectoryActionServerConfig

baxter_interface.cfg.PositionJointTrajectoryActionServerConfig

baxter_interface.cfg.VelocityJointTrajectoryActionServerConfig

baxter_interface.digital_io

baxter_interface.gripper

baxter_interface.head

baxter_interface.limb

baxter_interface.navigator

baxter_interface.robot_enable

baxter_interface.robust_controller

baxter_interface.settings

详细函数和变量可参考：http://api.rethinkrobotics.com/baxter_interface/html/frames.html。

### 3.3.2 机械臂接口

此接口针对机械臂操作，可通过下述方式初始化：

```
from baxter_interface import Limb
right_arm=Limb('right')
left_arm=Limb('left')
```

该接口包含下述功能：
查询关节状态；
切换控制模式；
发送关节命令（位置、速度、力矩）。

### 3.3.3 夹持器接口

此接口针对夹持器操作：

```
from baxter_interface import Gripper
right_gripper = Gripper('right')
left_gripper = Gripper('left')
```

主要功能如下：
向夹持器发送打开或闭合命令；
查询夹持器状态和属性；
夹持器插拔式时做出相应反应；
夹持器标定；
控制夹持器的各种动作（速度、移动力、抓取力、死区、真空阈值等）。

### 3.3.4 导航按钮接口

此接口对应于Baxter 的导航按钮的接口：

```
from baxter_interface import Navigator
right_arm_navigator = Navigator('right')
left_arm_navigator = Navigator('left')
right_torso_navigator = Navigator('torso_right')
left_torso_navigator = Navigator('torso_left')
```

主要功能如下：
查询转轮的状态；
对转轮和按钮动作做出相应反应；
控制导航按钮的灯。

### 3.3.5 使能/禁用机器人接口

此接口用于使能和禁用机器人，同时用于验证机器人软件版本：

```
from baxter_interface import RobotEnable
baxter = RobotEnable()
```

主要功能如下：
使能、禁用、停止、复位机器人；

获得机器人当前工作状态；
验证机器人和工作站软件版本是否相符。

### 3.3.6 相机接口

此相机接口用于和Baxter 机器人的相机进行交互：

```
from baxter_interface import CameraController
left_hand_camera = CameraController('left_hand_camera')
right_hand_camera = CameraController('right_hand_camera')
head_camera = CameraController('head_camera')
```

主要用于：
打开或关闭相机；
设置相机分辨率；
可用分辨率：
(1280,800)
(960,600)
(640,400)
(480,300)
(384,240)
(320,200)
获得相机设置参数（帧速、曝光度、增益、白平衡等）。

### 3.3.7 模拟IO 接口

此模拟IO 接口包含Baxter 所有模拟IO 接口：

```
from baxter_interface import AnalogIO
<componentname> =  AnalogIO(<componentid>)
```

可用模拟IO 组件包括：

```
left_hand_range,   right_hand_range,
left_itb_wheel,   right_itb_wheel,
torso_left_itb_wheel,   torso_right_itb_wheel ,
left_vacuum_sensor_analog,   right_vacuum_sensor_analog,
torso_fan
```

模拟 IO 口可用选项：
_on_io_state:状态改变时执行动作
state：或状态
is_output:检查端口是否具有输出功能
set_output：设置端口为输出

### 3.3.8 数字量IO 接口

此数字量IO 接口包含Baxter 所有数字量IO 接口：

```
from baxter_interface import DigitalIO
<componentname> = DigitalIO(<componentid>)
```

可用数字量IO 组件包括：

**表 3-1 数字量 IO 组件表**

| | |
|---|---|
| **AllDigitalComponentIDs:** | |
| **Outputs：** | |
| NavInner Lights: | left_itb_light_inner,right_itb_light_inner,torso_left_itb_light_inner,torso_right_itb_light_inner |
| NavOuter Lights | left_itb_light_outer,right_itb_light_outer,torso_left_itb_light_outer,torso_right_itb_light_outer |
| Configure Valves: | left_blow,right_blow,left_suck,right_suck |
| Actuate Pneumatics: | left_pneumatic,right_pneumatic |
| Camera Power: | left_hand_camera_power,right_hand_camera_power,torso_camera_power |
| Inputs： | |
| (Analog)RawIR RangeValues: | left_hand_range,right_hand_range |
| BackShoulderButto ns: | left_shoulder_button,right_shoulder_button |
| Cuff(Squeeze)Sens or: | left_lower_cuff,right_lower_cuff |
| CuffOKButton: | left_lower_button,right_lower_button |
| CuffGraspButton: | left_upper_button,right_upper_button |
| SafetyMat'isattache d': | torso_process_sense0 |
| SafetyMat'isNOTact uated': | torso_safety_stop |

数字量 IO 口可用选项：
_on_io_state:状态改变时执行动作
State:或状态
is_output:检查端口是否具有输出功能
set_output：设置端口为输出

### 3.3.9 头部接口

此接口用于控制Baxter 头部运动：

```
from baxter_interface import Head
<componentname> = Head()
```

可用命令：
_on_head_state: 对用于状态改变
pan,nodding,panning: 状态值
set_pan,command_nod：设置摇头、点头;

### 3.3.10 其他设置

SDK 包含机器人参数设置：
机械臂关节转角精度;
头部平动关节旋转精度;
SDK 版本;
夹持器版本和 SDK 版本兼容设置。

## 3.4 建立 Baxter 工作站

学习研究版的 Baxter 需要下载许多 SDK，因此可单独创建一个工作区间。

### 3.4.1 硬件需求

- 1 个 Baxter Research Robot
- 1 台 PC 机或笔记本，用来安装 Ubuntu 系统
- 1 个无线路由器
- 1 根网线

### 3.4.2 创建 Baxter 开发工作空间

确定已经安装好 Ubuntu 系统和配置好 ROS 操作系统之后，我们便可以继续下面的学习了。

**（1）创建 ROS 工作空间**

打开 Gnome Terminal, 可以通过鼠标点击左上角图标或者搜索“terminal”,亦可通过组合快捷键“Ctrl + Alt + T” 打开：

```
$ mkdir -p ~/ros_ws/src
# ros_ws (short for ROS Workspace)
```

其中~/ros_ws/src 为 ROS 开发环境所在路径地址，~代表用户 Home 目录路径。此目录用户可以自定义，推荐与官方 wiki 上写的一样，这样之后看 wiki 也方便些，注意 $ 符号后面的为 terminal 终端所应输入的命令，#符号后为提示语，请不要输入。

**（2）Source ROS**

```
$ source /opt/ros/indigo/setup.bash
```

注意：在每次打开终端时，都先要运行上述命令才能运行 ros 相关命令，为了避免这一繁琐过程，可以事先在.bashrc 文件（该目录在当前系统的 Home 目录下）中添加这条命令，这样当你每次登录后，系统自动帮你执行这些命令配置好开发环境。

**（3）编译与安装**

这一步要在创建的工作空间 ros_ws 中进行，所以先要改变工作目录路径，进入 ros_ws。

```
$ cd ~/ros_ws
$ catkin_make
$ catkin_make install
```

### 3.4.3 安装 Baxter 的 SDK

首先安装 python 依赖包，这一步要在 Home 目录（即~路径）下进行。

**(1) 安装依赖包**

```
$ sudo apt-get update
$ sudo apt-get install git-core python-argparse python-wstool python-vcstools python-rosdep ros-indigo-control-msgs ros-indigo-joystick-drivers
```

```
```

**(2) 安装 Baxter SDK**

使用 wstool 工作空间工具，会自动在 Baxter Github 中检查所有需要的源，并安装在 ROS 工作空间（即 ros_ws）中。

```
$ cd ~/ros_ws/src
$ wstool init .
$ wstool merge
https://raw.githubusercontent.com/RethinkRobotics/baxter/master/baxter_sdk.rosinstall
$ wstool update
```

这一过程安装时间比较长，如果电脑没有翻墙的话，很有可能安装不成功，所以进行此过程的时候最好先使用 VPN 连接！

Source ROS

"'You must use ROS Indigo to use RSDK 1.2.0. Use command below."'

# ROS Indigo

```
$ source /opt/ros/indigo/setup.bash
```

**(3) 编译与安装**

同样，这一步也需要在 ros_ws 路径下进行，因此先要进入 ros_ws 路径。

```
$ cd ~/ros_ws
$ catkin_make
$ catkin_make install
```

### 3.4.4 配置 Baxter 通信/ROS 工作空间

这一步骤描述了 ROS 开发环境的配置与设置。这一步假设已经设置好网络，将 Baxter 与 PC 通过无线路由器在局域网内相连。

**（1）下载 baxter.sh**

这一步主要是下载 Baxter 的配置文件，并改变这个文件的用户权限，方便后续编辑。

```
$ wget https://github.com/RethinkRobotics/baxter/raw/master/baxter.sh
$ chmod u+x baxter.sh
```

**（2）自定义 baxter.sh**

对 baxter.sh 脚本进行修改，填入 PC 机与 Baxter 机器人的一些信息。打开 baxter.sh，以 gedit 编辑器为例：

```
$ cd ~/ros_ws
$ gedit baxter.sh
```

**（3）编辑 baxter_hostname**

Baxter 的 hostname 默认为它的序列号，序列号可以在 Baxter 背部电源按钮旁边铭牌上找到。另外，也可以通过连接键盘，并同时按下 Ctrl+Alt+F3 找到。

```
# Specify Baxter's hostname
**baxter_hostname="baxter_hostname.local"**
```

例如，Baxter 的序列号为：011506P0014，则修改如下：

```
baxter_hostname="011506P0014.local"
```

**（4）编辑 your_ip**

将 your_ip 一栏修改为本机的 IP 地址。推荐将本机 IP 设为固定 IP，这样就不用每次开关机以后都要重新进行修改了。例如我在做实验的过程中将本机的 IP 地址设为：192.168.1.102，于是修改如下：

```
your_ip="192.168.1.102"
```

使用此种模式需要屏蔽屏蔽 your_hostname, 在其前加上“#”即可。

**（5）编辑 your_hostname**

除了编辑 your_ip, 你也可以将 your_hostname 一栏修改为本机的 hostname 地址和“.local”）。推荐设置为这种模式，这样无论 IP 如何改变，your_hostname 会指向新的 IP。例如我在做实验的过程中本机 hostname 为 cothinkTab，于是修改如下：

```
your_hostname="cothinkTab.local"
```

使用此种模式需要屏蔽屏蔽 your_ip, 在其前加上“#”即可。

**（6）修改 ros_version**

检查 baxter.sh 文件中，ros_version 一栏是否为 indigo（默认），如不是，修改如下：

```
ros_version=“indigo”
```

**（7）保存并关闭 baxter.sh 文件**

做完以上修改之后，请保存并关闭 baxter.sh 文件。

**（8）初始化 SDK 环境**

这一步骤很简单，在 Terminal 中运行如下代码即可：

```
$ cd ~/ros_ws
$ ./baxter.sh
```

### 3.4.5 检查开发环境配置是否正确

通过命令：

```
$ env | grep ROS
```

运行上述代码之后，会返回一些与 ROS 相关的配置信息，官方 wiki 上说关注以下几项就好：

ROS_MASTER_URI - This should now contain your robot's hostname.

ROS_IP - This should contain your workstation's IP address.

or

ROS_HOSTNAME - If not using the workstation's IP address, the ROS_HOSTNAME field should contain your PC's hostname. Otherwise, this field should not be available.

至此，工作空间配置就全部完成了，下面将运行一些简单的程序，验证上面的配置全部正确。

### 3.4.6 Hello Baxter

设计第一个 Baxter 的 ros 程序。

**（1）设置 ROS 环境**

如果之前设置过 ROS 环境，那么在~/ros_ws 文件目录下应该会有一个 devel 文件夹，如果

已经有了，可以跳过下面的代码（当然重复运行一次下面的代码也是没问题的），如果还没有此文件夹，那必须运行一遍下面的代码：

```
# Move to root of our catkin workspace
$ cd ~/ros_ws
$ source /opt/ros/indigo/setup.bash
$ catkin_make
```

接下来再 source 一下 ROS 环境设置脚本，也就是 baxter.sh

```
# Source baxter.sh script
$ . baxter.sh
```

**（2）检查 ROS 连接**

这一步，我们会检查 Baxter 机器人与我们的 PC 通信是否正常，也是我们用 PC 去控制 Baxter 的关键。

**（3）检查 PC 与 ROS Master(Baxter)的连接**

这一步是检查 PC 能否给 ROS Master 发送数据。

还记得前文设置的 baxter_hostname 么？我们只要检查下是否能 ping 通即可。代码如下：

```
$ ping 011506P0014.local
```

如果显示有数据通信，就说明与 ROS Master 的连接正常，继续下一步。

**（4）检查 Baxter 与开发工作空间 PC 的连接**

这一步是想检查 PC 能否收到 ROS Master 发过去的命令。

首先用 SSH 远程登录 ROS Master，在 ROS Master 上 ping 我们的 PC 机，看是否有通信。代码如下：

```
$ ssh ruser@<our ROS Master>

# Password:
rethink

# Example:
$ ssh ruser@011506P0014.local

# Now that we have SSH'd into the robot, verify that we are able to ping back to the development PC
ruser@011506P0014.local:~$ ping <ROS_IP/ROS_HOSTNAME>

# Examples:
# ROS_IP
ruser@011506P0014.local:~$ ping 192.168.1.102
```

如果能 ping 通，说明连接正常，可以退出远程登录了。

```
ruser@011506P0014.local:~$ exit
```

**（5）查看 ROS Topic**

如果双向通信没有问题了，我们就可以查看目前正在发生的 ROS Topic 了（关于 ROS Topic，请查看 ROS 操作系统教程）

```
$ rostopic list
```

以上代码可以查看所有的 ROS Topic，如果要查看特定的 Topic，示例如下：

```
$ rostopic echo /robot/joint_states
```

**（6）使能 Baxter**

操作 Baxter 机器人之前，先要使能机器人。在 baxter_tools SDK 工具包中，提供了“enable/disable/reset/stop”机器人的 API。使能机器人的代码如下：

```
$ rosrun baxter_tools enable_robot.py -e
```

**（7）运行一个示例程序**

Rethink Robotics 官方给我们提供了很多示例程序，大多是提供了 Python 与 C++的调用接口，我们来运行 joint_velocity_wobbler.py 看看效果，更多关于这个示例的信息，可以查看 Joint Velocity Wobbler Example Page。

```
$ rosrun baxter_examples joint_velocity_wobbler.py
```

**（8）自己编写一个程序来运行 Baxter**

大多数编程语言的第一示例程序都是 hello world 之类的，本程序也不例外，我们将编写一个 Hello Baxter 程序，这个程序实现的功能是让 Baxter 机器人向我们挥手。

首先，当然还是使能 Baxter 机器人：

```
$ rosrun baxter_tools enable_robot.py -e
```

接着打开 gedit 编辑器，输入一下代码，并保存为 hello_baxter.py。

```
# Import the necessary Python modules

# rospy - ROS Python API
import rospy

# baxter_interface - Baxter Python API
import baxter_interface

# initialize our ROS node, registering it with the Master
rospy.init_node('Hello_Baxter')

# create an instance of baxter_interface's Limb class
limb = baxter_interface.Limb('right')

# get the right limb's current joint angles
angles = limb.joint_angles()

# print the current joint angles
print angles

# reassign new joint angles (all zeros) which we will later command to the limb
angles['right_s0']=0.0
angles['right_s1']=0.0
angles['right_e0']=0.0
```

```
angles['right_e1']=0.0
angles['right_w0']=0.0
angles['right_w1']=0.0
angles['right_w2']=0.0

# print the joint angle command
print angles

# move the right arm to those joint angles
limb.move_to_joint_positions(angles)

# Baxter wants to say hello, let's wave the arm

# store the first wave position
wave_1 = {'right_s0': -0.459, 'right_s1': -0.202, 'right_e0': 1.807, 'right_e1': 1.714, 'right_w0': -0.906, 'right_w1': -1.545, 'right_w2': -0.276}

# store the second wave position
wave_2 = {'right_s0': -0.395, 'right_s1': -0.202, 'right_e0': 1.831, 'right_e1': 1.981, 'right_w0': -1.979, 'right_w1': -1.100, 'right_w2': -0.448}

# wave three times
for _move in range(3):
    limb.move_to_joint_positions(wave_1)
    limb.move_to_joint_positions(wave_2)

# quit
quit()
```

在 Terminal 中运行编辑好的 python 程序：

```
$ python hello_baxter.py
```

是否看到 Baxter 在向你挥手？恭喜你，你已经完成了控制 Baxter 的第一个程序！

## 3.5 SDK 示例解析

对 Baxter 的 SDK 里比较典型的几个示例进行详细介绍。

### 3.5.1 键盘控制夹持器

通过键盘控制 Baxter 的左右双臂的夹持器，比如控制夹持器的打开和关闭，进行夹抓的 calibrate 等。

```
#!/usr/bin/env python

import argparse
import rospy

import baxter_interface
import baxter_external_devices
```

```
from baxter_interface import CHECK_VERSION

def map_keyboard():
    #初始化机器人应用程序接口
    print("Getting robot state... ")
    #使能机器人时检查SDK版本和机器人版本是否一一致
    rs = baxter_interface.RobotEnable(CHECK_VERSION)
    init_state = rs.state().enabled
    #初始化左右夹持器
    left = baxter_interface.Gripper('left', CHECK_VERSION)
    right = baxter_interface.Gripper('right', CHECK_VERSION)

    #退出程序时禁用机器人
    def clean_shutdown():
        if not init_state:
            print("Disabling robot...")
            rs.disable()
        print("Exiting example.")

    # ROS退出时调用clean_shutdown
    rospy.on_shutdown(clean_shutdown)

    #夹持器不支持的命令
    def capability_warning(gripper, cmd):
        msg = ("%s %s - not capable of '%s' command" % (gripper.name, gripper.type(), cmd))
        rospy.logwarn(msg)

    #电动夹持器按偏移量相对移动
    def offset_position(gripper, offset):
        if gripper.type() != 'electric':
            capability_warning(gripper, 'command_position')
            return
        current = gripper.position()
        gripper.command_position(current + offset)

    #电动夹持器按偏移量设置夹持力
    def offset_holding(gripper, offset):
        if gripper.type() != 'electric':
            capability_warning(gripper, 'set_holding_force')
            return
        current = gripper.parameters()['holding_force']
        gripper.set_holding_force(current + offset)

    #电动夹持器按偏移量设置移动时的夹持力
    def offset_moving(gripper, offset):
        if gripper.type() != 'electric':
            capability_warning(gripper, 'set_moving_force')
```

```
            return
        current = gripper.parameters()['moving_force']
        gripper.set_moving_force(current + offset)

    # 电动夹持器按偏移量设置移动时的速度
    def offset_velocity(gripper, offset):
        if gripper.type() != 'electric':
            capability_warning(gripper, 'set_velocity')
            return
        current = gripper.parameters()['velocity']
        gripper.set_velocity(current + offset)

    # 电动夹持器按偏移量设置死区
    def offset_dead_band(gripper, offset):
        if gripper.type() != 'electric':
            capability_warning(gripper, 'set_dead_band')
            return
        current = gripper.parameters()['dead_zone']
        gripper.set_dead_band(current + offset)

    #按键值和功能绑定，如按下‘w’，左臂夹持器打开
    bindings = {
        #key: (function, args, description)
        'r': (left.reboot, [], "left: reboot"),
        'R': (right.reboot, [], "right: reboot"),
        'c': (left.calibrate, [], "left: calibrate"),
        'C': (right.calibrate, [], "right: calibrate"),
        'q': (left.close, [], "left: close"),
        'Q': (right.close, [], "right: close"),
        'w': (left.open, [], "left: open"),
        'W': (right.open, [], "right: open"),
        '[': (left.set_velocity, [100.0], "left:set 100% velocity"),
        '{': (right.set_velocity, [100.0], "right:set 100%velocity"),
        ']': (left.set_velocity, [30.0], "left:set 30% velocity"),
        '}': (right.set_velocity, [30.0], "right:set 30% velocity"),
        's': (left.stop, [], "left: stop"),
        'S': (right.stop, [], "right: stop"),
        'z': (offset_dead_band, [left, -1.0], "left:decrease dead band"),
        'Z': (offset_dead_band, [right, -1.0], "right:decrease dead band"),
        'x': (offset_dead_band, [left, 1.0], "left:increase dead band"),
        'X': (offset_dead_band, [right, 1.0], "right:increase dead band"),
        'f': (offset_moving, [left, -5.0], "left:decrease moving force"),
        'F': (offset_moving, [right, -5.0], "right:decrease moving force"),
        'g': (offset_moving, [left, 5.0], "left:increase moving force"),
        'G': (offset_moving, [right, 5.0], "right:increase moving force"),
        'h': (offset_holding, [left, -5.0], "left:decrease holding force"),
        'H': (offset_holding, [right, -5.0], "right:decrease holding force"),
```

```
            'j': (offset_holding, [left, 5.0], "left:increase holding force"),
            'J': (offset_holding, [right, 5.0], "right:increase holding force"),
            'v': (offset_velocity, [left, -5.0], "left:decrease velocity"),
            'V': (offset_velocity, [right, -5.0], "right:decrease velocity"),
            'b': (offset_velocity, [left, 5.0], "left:increase velocity"),
            'B': (offset_velocity, [right, 5.0], "right:increase velocity"),
            'u': (offset_position, [left, -15.0], "left:decrease position"),
            'U': (offset_position, [right, -15.0], "right:decrease position"),
            'i': (offset_position, [left, 15.0], "left:increase position"),
            'I': (offset_position, [right, 15.0], "right:increase position"),
        }

        done = False
        print("Enabling robot... ")
        rs.enable()
        print("Controlling grippers. Press ? for help, Esc to quit.")

        #程序主循环，根据读取的键值，执行相应命令
        while not done and not rospy.is_shutdown():
            c = baxter_external_devices.getch()
            if c:
            if c in ['\x1b', '\x03']:
                done = True
            elif c in bindings:
                cmd = bindings[c]
                cmd[0](*cmd[1])
                print("command: %s" % (cmd[2],))
            else:
            print("key bindings: ")
            print(" Esc:Quit")
            print("  ?:Help")
            for key, val in sorted(bindings.items(),key=lambda x: x[1][2]):
                print("      %s: %s" % (key, val[2]))
        # force shutdown call if caught by key handler
        rospy.signal_shutdown("Example finished.")


def main():
        """RSDK夹持器示例程序: 键盘控制 使用您工作站的键盘控制baxter机器人夹持器的运
动
        通过调整不同的参数，可以控制机器人不同的运动，比如标定、速度、力等
        """

        #输入‘？’查看键值和命令的对应关系
        epilog = """See help inside the example with the '?' key for key bindings. """
        arg_fmt = argparse.RawDescriptionHelpFormatter
        #构造参数分析器
```

```
    parser = argparse.ArgumentParser(formatter_class=arg_fmt,
                                     description=main. doc   , epilog=epilog)
    #分析系统传入的参数
    parser.parse_args(rospy.myargv()[1:])

    print("Initializing node... ")
     rospy.init_node("rsdk_gripper_keyboard")

     map_keyboard()

if   __name__== ' __main__   ':
     main()
```

### 3.5.2 摇动头部

控制 Baxter 的头部，使头部进行左右摆动等。

```
#!/usr/bin/env python

import argparse
import random
import rospy
import baxter_interface
from baxter_interface import CHECK_VERSION

class Wobbler(object):
    def  _init_  (self):
        """
        机器人摇头示例程序
        """
        self._done = False
        self._head = baxter_interface.Head()

        #验证机器人是否处于使能状态
        print("Getting robot state... ")
        self._rs = baxter_interface.RobotEnable(CHECK_VERSION)
        self._init_state = self._rs.state().enabled
        print("Enabling robot... ")
        self._rs.enable()
        print("Running. Ctrl-c to quit")

    def clean_shutdown(self):
        """
        退出程序时将机器人头部置于中间位置并维持初始状态
        """
        print("\nExiting example...")
        if self._done:
            self.set_neutral()
```

```
        if not self._init_state and self._rs.state().enabled:
            print("Disabling robot...")
            self._rs.disable()

    def set_neutral(self):
        """
        设置机器人头部处于中间位置，此时pan关节转角为0
        """
        self._head.set_pan(0.0)

    def wobble(self):
        self.set_neutral()
        """
        执行摇头动作
        """
        self._head.command_nod()
        # pan关节转角命令发布频率
        command_rate = rospy.Rate(1)
        #控制频率
        control_rate = rospy.Rate(100)
        start = rospy.get_time()
        #在-1.5弧度和1.5弧度间随机取转动角度
        while not rospy.is_shutdown() and (rospy.get_time() - start<10.0):
            angle = random.uniform(-1.5, 1.5)
            while (not rospy.is_shutdown() and not (abs(self._head.pan() - angle) <=
                                baxter_interface.HEAD_PAN_ANGLE_TOLERANCE)):
                #设置转动角度和速度
                self._head.set_pan(angle, speed=30, timeout=0)
                control_rate.sleep()
            command_rate.sleep()

        self._done = True
        rospy.signal_shutdown("Example finished.")

def main():
    """RSDK 头部示例:摇动
    点头，并随机将头部从一侧移动到另一侧
    """
    arg_fmt = argparse.RawDescriptionHelpFormatter
    parser = argparse.ArgumentParser(formatter_class=arg_fmt,description=main._doc_ )
    parser.parse_args(rospy.myargv()[1:])

    print("Initializing node... ")
    rospy.init_node("rsdk_head_wobbler")

    wobbler = Wobbler()
    rospy.on_shutdown(wobbler.clean_shutdown)
```

```
    print("Wobbling... ")
    wobbler.wobble()
    print("Done.")

if __name__ == '__main__':
    main()
```

### 3.5.3 双臂随动

此示例用于控制 Baxter 的双臂随动，即当你控制 Baxter 的一只手臂进行移动时，将这个运动镜像到另外一个手臂上，使另外一个手臂跟随着做同样的动作。

```
#!/usr/bin/env python
import argparse
import sys
import rospy
from std_msgs.msg import (
    UInt16,
)
import baxter_interface
from baxter_interface import CHECK_VERSION
class Puppeteer(object):
    def _init_ (self, limb, amplification=1.0):
        """
        双臂随动示例
        @param limb: 模仿另一个臂的臂
        @param amplification: 机械臂运动的缩放参数
        """
        puppet_arm = {"left": "right", "right": "left"}
        # 被控制的臂
        self._control_limb = limb
        # 模仿运动的臂
        self._puppet_limb = puppet_arm[limb]
        # 初始化机械臂接口
        self._control_arm =baxter_interface.limb.Limb(self._control_limb)
        self._puppet_arm =baxter_interface.limb.Limb(self._puppet_limb)
        self._amp = amplification

        # 使能机器人
        print("Getting robot state... ")
        self._rs = baxter_interface.RobotEnable(CHECK_VERSION)
        self._init_state = self._rs.state().enabled
        print("Enabling robot... ")
        self._rs.enable()

    # 复位机器人控制模式
```

```
def _reset_control_modes(self):
    rate = rospy.Rate(100)
    for _ in xrange(100):
        if rospy.is_shutdown():
            return False
        self._control_arm.exit_control_mode()
        self._puppet_arm.exit_control_mode() rate.sleep()
    return True

def set_neutral(self):
    """
    将双臂都移动到中间位置
    """
    print("Moving to neutral pose...")
    self._control_arm.move_to_neutral()
    self._puppet_arm.move_to_neutral()

def clean_shutdown(self):
    print("\nExiting example...")
    #退出程序时复位控制模式，将机械臂移动到中间位置，禁用机器人
    self._reset_control_modes()
    self.set_neutral()
    if not self._init_state:
        print("Disabling robot...")
        self._rs.disable()
    return True

def puppet(self):
    """
    双臂随动
    """
    self.set_neutral() rate = rospy.Rate(100)

    #获取机械臂关节名
    control_joint_names = self._control_arm.joint_names()
    puppet_joint_names = self._puppet_arm.joint_names()

    #按压机械臂腕部感应按钮，移动机械臂
    print ("Puppeting:\n"
           "Grab %s cuff and move arm.\n"
           "Press Ctrl-C to stop...") % (self._control_limb,)
    while not rospy.is_shutdown():
        cmd = {}
        for idx, name in enumerate(puppet_joint_names):
            v = self._control_arm.joint_velocity(control_joint_names[idx])
            if name[-2:] in ('s0', 'e0', 'w0', 'w2'):
                v = -v
```

```
                cmd[name] = v * self._amp
            #获取被拖动的机械臂的关节速度，将其发送到模仿机械臂
            self._puppet_arm.set_joint_velocities(cmd) rate.sleep()

def main():
    """RSDK 关节速度示例: 模仿 将一个机械臂的运动镜像到另外一个机械臂上，使用关
    节速度控制模式运行此程序时，将模仿臂的参数传递到类的构造函数中去，使另外一
    个臂处于零重力模式 并拖动其运动模仿臂将跟随被拖动的臂运动
    """
    max_gain = 3.0
    min_gain = 0.1

    arg_fmt =argparse.RawDescriptionHelpFormatter
    parser =argparse.ArgumentParser(formatter_class=arg_fmt,description=main. _doc_  )
    required = parser.add_argument_group('required arguments')

    #声明模仿臂
    required.add_argument("-l", "--limb", required=True, choices=['left','right'], help=
                          "specify the puppeteer limb (the control limb)")
    parser.add_argument("-a", "--amplification", type=float, default=1.0, help=("amplification
                        to apply to the puppeted arm [%g, %g]"% (min_gain, max_gain))  )
    #设置最小最大缩放倍数，比如被控制臂的运动速度为1，
    #缩放系数为0.5，模仿臂将以 0.5的速度运动
    args =parser.parse_args(rospy.myargv()[1:])
    if (args.amplification < min_gain or max_gain <args.amplification):
        print("Exiting: Amplification must be between: [%g, %g]" % (min_gain, max_gain))
        return 1

    print("Initializing node... ")
    rospy.init_node("rsdk_joint_velocity_puppet")

    #传递模仿臂和缩放系数参数
    puppeteer = Puppeteer(args.limb, args.amplification)
    rospy.on_shutdown(puppeteer.clean_shutdown)
    puppeteer.puppet()

    print("Done.")
    return 0

if   __name__== ' __main__   ':
    sys.exit(main())
```

### 3.5.4 语音控制

通过语音控制机器人动作，并且向头部显示器发送相应图片。

```
#!/usr/bin/env python
 """
```

```
通过语音控制机器人动作，需要pocketsphinx包进行语音识别 参考：
http://www.ros.org/wiki/pocketsphinx
"""
import rospy
from geometry_msgs.msg import Twist
from std_msgs.msg import String
from math import copysign
from sound_play.libsoundplay import SoundClient
import os import sys import argparse
import rospy
import cv2
import cv_bridge
from sensor_msgs.msg import (Image, )
#图片及示例程序的路径
MEDIA_PATH ='/home/frog/moveit_ws/src/baxter/sysu_baxter/demos/GreetDemo'
EXAMPLES_PATH ='/home/frog/moveit_ws/src/baxter/baxter_examples/scripts'

def send_image(path):
    """
    向机器人头部显示器发布图片
    @param path: 图片存放路径
    """
    #使用opencv读取图片
    img = cv2.imread(path)
    #将图片转为ROS消息并发布
    msg = cv_bridge.CvBridge().cv2_to_imgmsg(img, encoding="bgr8")
    pub = rospy.Publisher('/robot/xdisplay', Image, latch=True,queue_size=1)
    pub.publish(msg)
    #休眠1秒等待图像被显示出来
    rospy.sleep(1)

def talkback(self, word):
    #在屏幕上输出并识别出来的语音
    rospy.loginfo(word)
    #以给定语音发音出被识别到的词语
    self.soundhandle.say(word, self.voice)

class VoiceNav:
    def _init_ (self):
        rospy.init_node('voice_nav')
        rospy.on_shutdown(self.cleanup)
        # 不用以过高频率运行此程序，设为5Hz
        self.rate = rospy.get_param("~rate", 5)
        r = rospy.Rate(self.rate)

        #确定语音控制是否暂停的标志
        self.paused = False
```

```
    #创建语音客户端
    self.voice = rospy.get_param("~voice", "voice_kal_diphone")
    self.soundhandle = SoundClient()

    # Initialize the Twist  message we will publish.
    self.cmd_vel = Twist()
    #初始化运动控制topic，baxter不用此topic
    self.cmd_vel_pub = rospy.Publisher('cmd_vel', Twist, queue_size=5)

    # 监听/recognizer/output获得语音指令
    rospy.Subscriber('/recognizer/output', String, self.speech_callback)

    #关键字到命令的映射
    self.keywords_to_command = {'hello': ['hello', 'hello baxter', 'hi', 'come'], 'stop'],
                                'old': ['old', 'how old','how'],
                                'where': ['where', 'where are','from'],
                                'good': ['good', 'good bye', 'bye',
                                'sport': ['what is', 'sport'],
                                'difficult': ['difficult']}

    #开始接收语音指令
    rospy.loginfo("Ready to receive voice commands")

    # 持续发布运动指令，baxter不用
    while not rospy.is_shutdown():
      self.cmd_vel_pub.publish(self.cmd_vel)
       r.sleep()

def get_command(self, data):
    #将语音词语映射到命令
    for (command, keywords) inself.keywords_to_command.iteritems():
       for word in keywords:
          if data.find(word) > -1:
             return command

def speech_callback(self, msg):
    #语音回调函数
    command = self.get_command(msg.data)

    #在屏幕上输出识别到的指令
    rospy.loginfo("Command: " + str(command))

    #如果语音指令是‘hello’，则显示打招呼的图片，并执行握手的动作
    if command == 'hello':
          back = 'hello'
          talkback(self, back)
```

```
            send_image('/home/frog/moveit_ws/src/baxter/sysu_baxter/demos/GreetDe
mo/pic/1.png' )
            pypath = "python " + EXAMPLES_PATH + '/joint_position_file_playback.py' + " -f "
                     + MEDIA_PATH + "/pro/shake_hand.dat"
             print pypath
            os.system(pypath)

        # 如果语音指令是‘old’，则显示‘old’的图片，并说'i am one yearold'
        elif command == 'old':
            back = 'i am one year old'
            talkback(self, back)
           send_image('/home/frog/moveit_ws/src/baxter/sysu_baxter/demos/GreetDe  mo/pic/old.jpg')

        #如果语音指令是‘good’，则显示‘goodbye’的图片，并执行问候动作
        elif command == 'good':
            back = 'bye bye'
            talkback(self, back)
            send_image('/home/frog/moveit_ws/src/baxter/sysu_baxter/demos/GreetDemo/
                       pic/goodbye.jpg')
           pypath = "python " + EXAMPLES_PATH + '/joint_position_file_playback.py' + " -f " +
                    MEDIA_PATH + "/pro/greet.dat"
            os.system(pypath)

        # 如果语音指令是‘sport’，则显示篮球的图片，并说'my favorite sportis basketball'
        elif command == 'sport':
            back = 'my favorite sport is basketball' talkback(self, back)
           send_image('/home/frog/moveit_ws/src/baxter/sysu_baxter/demos/GreetDemo
            /pic/basketball.jpg')

        else:
            return

    def cleanup(self):
        # 停止机器人运动，baxter不用
        twist = Twist()
       self.cmd_vel_pub.publish(twist)
        rospy.sleep(1)

if __name__=="__main__  ":
    try:
        VoiceNav()
        rospy.spin()
    except rospy.ROSInterruptException:
        rospy.loginfo("Voice control terminated.")
```

# 第四章 Baxter 应用案例

本章介绍基于研究型 Baxter 的部分应用示例，涉及到运动学规划、计算机视觉、摇杆操控等方面。通过本章的学习，可以更好的了解基于 baxter 开展机器人软件开发的一些方向，并为后续人工智能或深度学习的研究开发提供一定的基础。

## 4.1 双臂协同及运动学跟踪

此方案基于 CDTS（协作双臂任务空间项目）项目，用于多机械臂协同操作，提供 ROS 软件包，可运行在 Baxter 科研版机器人上。此方案也提供了针对单个臂的运动学控制，具有运动学跟踪能力，即当 Baxter 一个机械臂运动时，另外一个机械臂跟踪此机械臂的运动。机械臂运动过程中，末端执行器将保持初始相对位置不变。



**图 4-1 Baxter 双臂协同持球**

### 4.1.1 对偶四元数

对偶四元数（DualQuaternions）是对偶数和四元数在多维空间中的有机结合，可以理解为元素为四元数的对偶数，同样可以理解为元素为对偶数的四元数。较之四元数只能表示 3D 旋转，对偶数只能表示平移，对偶四元数的优越性体现在它继承了二者的共同特性，从而能统一的表示旋转与平移。它的三维就空间描述可以由下图 4-2 表示。

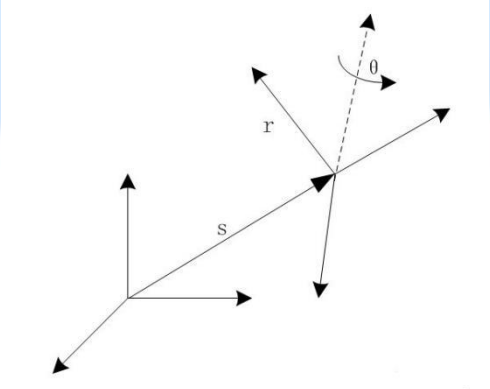



**图 4-2 对偶四元数**

### 4.1.2 任务协作操作

协同操作使得使机器人能够完成更为复杂的任务，协同操作时左右末段执行器的姿态如图 4-3 所示。

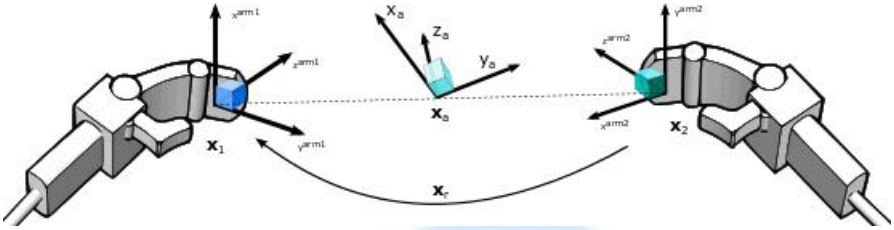


**图 4-3 末端执行器协同操作姿态**

### 4.1.3 Baxter 跟踪控制器

Baxter 跟踪控制器执行跟踪功能，当一个机械臂运动时，另外一个机械臂跟踪此机械臂的运 动学特征及末端姿态。为了实现这项功能，Baxter 的一个机械臂由 ROS 的节点自动控制，当另一个机械臂运动时，此机械臂将同时运动以保持末端执行器的距离和方位保持不变。此功 能不是简单的复制机械臂关节的速度和位置，而是根据末端执行器姿态做相应运算。

### 4.1.4 下载与安装

BaxterSDK 版本应大于等于1.0。

```
$ cd ../src/
$ git clone –b baxter_iros_subm https://gitlab.com/figueredo/ros_dqrobotics.git
$ roscd
$ cd..
$ catkin_make  –pkg  dq_robotics
```

### 4.1.5 运行示例

执行该程序时，请让机器人处于使能状态：

```
$ rosrun  baxter_tools enable_robot.py  -e;
$ rosrun  baxter_interface  joint_trajectory_action_server.py 2>&1&
```

运行双臂协同及运动学跟踪程序：

```
$ rosrun  dq_robotics  baxter_dq_control
```

运行gripper_cuff_control 控制机械臂运动：

```
$ rosrun  baxter_examples  gripper_cuff_control.py
```

### 4.1.6 使用跟踪功能

使用跟踪控制器比较简单，首先指定双臂的初始姿态，末端执行器此后将保持此相对位置。

按一个机械臂上的 ITB 按钮，此机械臂将会被 ROS 节点自动控制以维持末端执行器相对姿态。此时，您移动一个机械臂是，另外一个将会进行运动学及末端姿态跟踪。

禁用跟踪模式只需在按一下上次的 ITB 按钮，双臂都会处于处于空闲的重力补偿状

态。

### 4.1.7 ROS 服务及主题

双臂协同及运动学跟踪也提供了一些特定的 ROS 任务以完成更高级的任务，基本来说，用户可以调用三种服务：空闲模式（机械臂进入重力补偿状态），跟踪模式（运动学和末端位置跟踪模式），点设置（为机械臂设置特定位置）。

在点设置模式下，用户可以通过下面的主题设置末端执行器的姿态：

```
$ rostopic  pub/baxter_dqcontrol/left(right)/control/pose(Type:geometry_msgs/Pose)
$ rostopic  pub/baxter_dqcontrol/left(right)/control/position(Type:geometry_msgs/Point)
$ rostopic  pub/baxter_dqcontrol/left(right)/control/add_translation(Type:geometry_msgs/Po int)
```

最后一个主题允许用户添加一个相对于当前姿态的平移量，可以让用户进行相对位置控制。 此外，用户也可以通过下面的主题获得每个臂的末端执行器的当前姿态和控制错误：

```
$ rostopic  echo/baxter_dqcontrol/left(right)/pose
$ rostopic  echo/baxter_dqcontrol/left(right)/error_info
```

## 4.2 视觉抓取乒乓球

此方案通过视觉处理算法，识别桌面上的乒乓球及放置位置，使用基本的规划方法选择一个球并规划出抓取方式。最后，使用逆运动学求解关节转角并控制机械臂运动。

此方案使用机器人左臂，安装电动夹持器（带橡胶垫的短的、窄的指端，安装在左臂夹抓位置）。



**图 4-4 视觉抓取乒乓球示意图**

### 4.2.1 相机与工作空间的关系

视觉处理时，球和托盘的位置必须比较精确，以方便机械臂抓取和放置。通过分析 Baxter 腕部相机的图像，可以得到相机到球和托盘的角度，如果同时知道手臂到桌面的距离，那么物体相对 Baxter 的位置就是已知的了。

一旦确定了球的位置，Baxter 就可以去抓取并放到托盘上。

这些工作空间坐标是相对于 Baxter 基座标系的，是笛卡尔坐标，以米为单位，用于 Baxter 正逆运动学求解。

### 4.2.2 相机标定参数

为了使 Baxter 的相机能够找到物体的位置，需要知道相机到物体的像素距离及单位像素所代表的实际距离，下面的标定将确定一个参数：单位像素所代表的实际距离（以米为单位）。

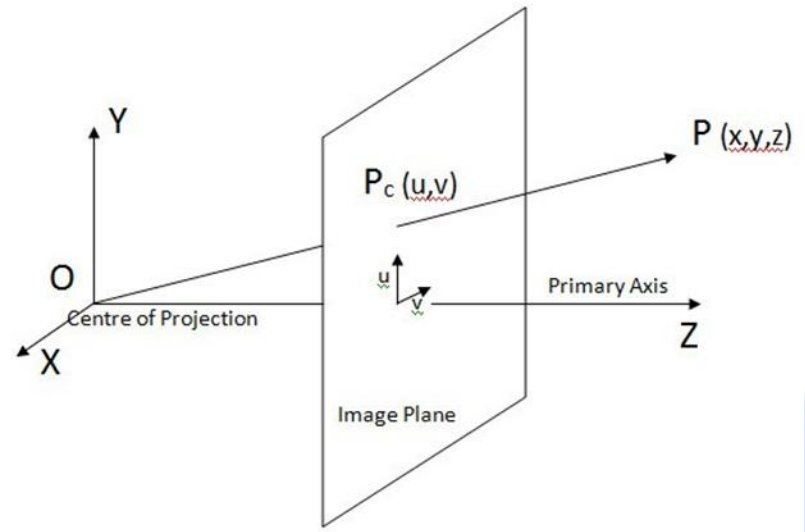



**图 4-5 坐标系示意图**

标定板由24 个间距为4cm 的2cm 的方块组成，6 行4 列分布，标定时，放在距相机不同的距离，分析图像找到每个方块的中心（以像素为单位）。

每帧图像都被转变成灰度图像。一个灰度阈值把灰度图像转变成黑白图像，其他接触图像的黑色区域被移除。一个区域阈值被用于移除图像中的小的黑色区域，使得图像中仅剩下目标方块。方块的中心被小黑点标出，如下图。

根据分析结果，相机标定结果是，在1m 距离时，每像素代表2.5mm。

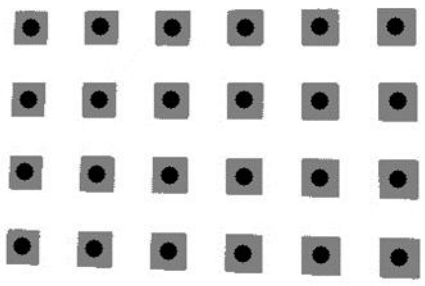


**图 4-6 标定板示意图**

### 4.2.3 像素坐标到工作空间坐标的转换

机械臂的运动应当垂直于桌面，这样把相机坐标转换到工作空间的坐标就比较容易

了。已知机械臂姿态，相机标定参数及距离桌面的高度，像素值可以转换到机械臂坐标系（相对于夹持器夹取点），公式如下：

**B=(Pp–Cp)*cc*d+Bp+Go**

B：物体在Baxter 坐标系中的坐标

Pp：物体在相机像素坐标系的坐标

Cp：相机坐标系中心点坐标

Bp：Baxter 姿态

Go：夹持器便宜

cc：相机标定参数

d：到桌面的距离

代码的第201 行-207 行把相机像素坐标转换到Baxter 工作空间的坐标。

第194 行-200 行进行逆转换，把Baxter 工作空间的坐标转换到像素坐标。

距离 d 通过手臂末端的红外距离传感器获得，一个标定程序被用于找到桌面到已知姿态的距离（机械臂末端垂直于桌面）。应当保持桌面干净，标定程序应当提前运行，如有需要可以自行编辑setup.bat。

### 4.2.4 确定托盘在工作空间中的位置

把机械臂移动到托盘上方时可以采集到托盘的图像，此托盘应该放在操作机械臂的那一侧（我们使用左臂，托盘放在左臂侧）。

分析该图像可以得到托盘相对于 Baxter 工作空间的坐标。

第 150 行-193 行是一些常规操作，比如参数设置，打开或关闭相机。

第 280 行-311 行监听相机图像。

在物体和背景对比度很高的情况下，使用 Canny 边缘检测算子可以很容易的找到物体的边缘，我们直接只用 OpenCV 中的 Canny 算法，Canny 图像如下图所示：



**图 4-7 Baxter 手臂摄像头视野**



**图 4-8 canny 算子处理图**

Canny 图像显示了边界化的区域，可以清楚的看到球和托盘。注意，夹持器手指的一部分也被识别为托盘了。canny_it 用于找到托盘的中心。



**图 4-9 托盘中心图**

### 4.2.5 确定托盘方向及插值放置位置

托盘的左下角位置由托盘区域的左最大像素给出，右下角位置由最小像素给出。这两个坐标及托盘中心的坐标被用于计算其他两个角的坐标及托盘的方向。

给定矩形的两个角C1、C2 及中心M，其他两个角可以由下面公式计算得出：

**C3=2*M–C1**

**C4=2*M–C2**

托盘可以容纳12 个球，以3x4 的方式排列，托盘的边长可以确定托盘的方向。

If**(C1–C2).(C1–C2)>(C2–C3).(C2–C3)**then**C1->C2**isthelongside.

If**(C1–C2).(C1–C2)<(C2–C3).(C2–C3)**then**C2->C3**isthelongside.

如果C1-C2 是长边，放置位置由下面公式给出：

**C1+(2*i–1)*V1+(2*j–1)*V2**

其中：

**V1=(C2–C1)/8i={1,2,3,4}**

**V2=(C3–C2)/6j={1,2,3}**

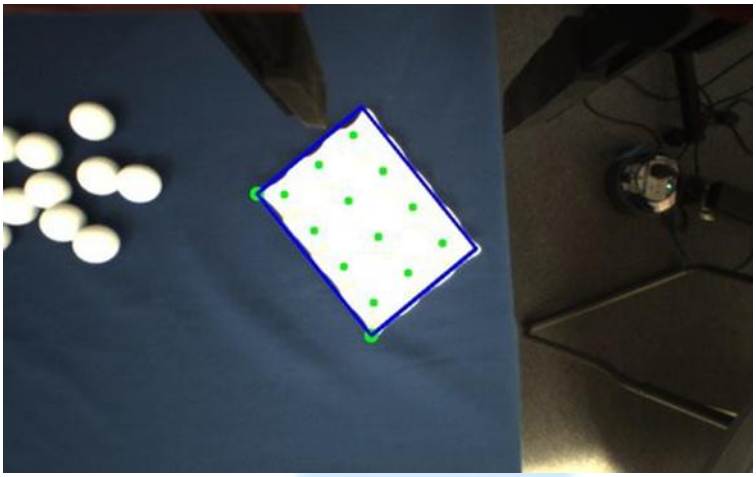


图 4-10 托盘分割图

### 4.2.6 确定球的位置

球是圆形的，可以用 Hough 算法定位球的位置。Hough 算法在灰度图像中找到潜在的圆形。对于图中的每一个像素及给定的半径，该算法考虑半径内的每一个像素，如果有足够多的像素落在圆周上，则认为找到圆形。

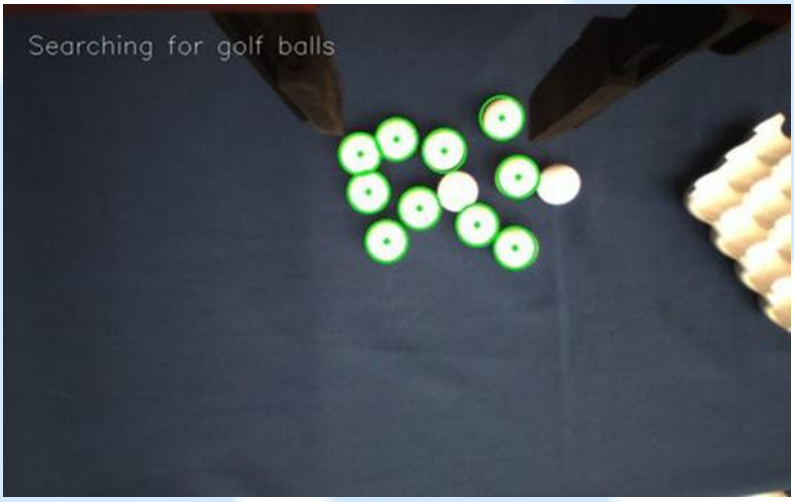



图 4-11 识别乒乓球图

如果Baxter 的机械臂放到球的上方，则可以采集到包含球的图像，再使用Hough 算法分割出球。同样根据球的中心像素坐标计算出球在 Baxter 工作坐标系中的位置，将该位置传给机械臂进行相应抓取操作。

### 4.2.7 下载和安装

通过地址 https://github.com/birlrobotics/birl_baxter_demos 可以下载；

将程序包放到工作空间的 src 目录之下，再在工作空间目录下运行 catkin_make 和 rospack profile 就可以了。

## 4.3 基于Kinect 的动作跟随

此案例展示的是 Baxter 模仿人手臂的动作。首先通过 Kinect 采集到深度图像，然后使用 openni_tracker 去获取人体双臂的坐标变换，将坐标变换转变成角度值，控制 Baxter 机器人相应关节运动到指定角度。



图 4-12 动作跟随图

动作跟随的详细代码如下：

```
#!/usr/bin/env python

import rospy
from std_msgs.msg import String
from std_msgs.msg import Float64

import rospy
import math
import tf
import geometry_msgs.msg
from std_msgs.msg  import Float64
import turtlesim.srv
import math
import baxter_interface
from baxter_interface import CHECK_VERSION
import operator

mirror = True
times = 20
data_set = {'left':[[0]*times for i in range(4)],'right':[[0]*times
for i in range(4)]}

pub_list = {}
pub_list_raw = {}

#求向量模
def vector_length(v):
    return math.sqrt(v[0]*v[0]+v[1]*v[1]+v[2]*v[2])

#获得单位向量
def get_unit_vector(v):
```

```
    length = vector_length(v)
    return (v[0]/length, v[1]/length, v[2]/length)

#求向量夹角
def angle_query(v1, v2):
    p = v1[0]*v2[0]+v1[1]*v2[1]+v1[2]*v2[2]
    m1 = vector_length(v1)
    m2 = vector_length(v2)
    cos  = p/(m1*m2)
    return math.acos(cos)

#向量叉乘
def vector_cross_product(a, b):
    r1 = a[1]*b[2]-b[1]*a[2]
    r2 = a[2]*b[0]-b[2]*a[0]
    r3 = a[0]*b[1]-b[0]*a[1]
    return (r1, r2, r3)

#根据坐标变换求解夹角
def tf_2_angles(shoulder, shoulder_x, elbow, hand,torso, head, base, listener, limb_name):
    try:
      (b_2_s, trash) = listener.lookupTransform(base, shoulder, rospy.Time(0))
      (b_2_s_x, trash) = listener.lookupTransform(base, shoulder_x, rospy.Time(0))
      (b_2_e, trash) = listener.lookupTransform(base, elbow, rospy.Time(0))
      (b_2_hand, trash) = listener.lookupTransform(base, hand, rospy.Time(0))
      (b_2_t, trash) = listener.lookupTransform(base, torso, rospy.Time(0))
      (b_2_head, trash) = listener.lookupTransform(base, head, rospy.Time(0))
    except (tf.LookupException, tf.ConnectivityException, tf.ExtrapolationException):
      print "tf exception"
      return {}

    b_2_s = list(b_2_s)
    b_2_s_x = list(b_2_s_x)
    b_2_e = list(b_2_e)
    b_2_hand = list(b_2_hand)
    b_2_t = list(b_2_t)
    b_2_head = list(b_2_head)



    s_2_s_x = map(operator.sub, b_2_s_x, b_2_s)
    s_2_t = map(operator.sub, b_2_t, b_2_s)
    s_2_e = map(operator.sub, b_2_e, b_2_s)

    e_2_hand = map(operator.sub, b_2_hand, b_2_e)
    head_2_t = map(operator.sub, b_2_t, b_2_head)
    my_v2 = head_2_t
```

```
    my_v3 = vector_cross_product(s_2_e, my_v2)
    my_theta_1 = angle_query(my_v3, s_2_s_x)

    if limb_name == 'right':
        radius_s0 = math.pi/4-my_theta_1
    else:
        radius_s0 = math.pi*3/4-my_theta_1

    if mirror:
        if limb_name == 'left':
            radius_s0 = -radius_s0+math.pi/2
        else:
            radius_s0 = -radius_s0-math.pi/2

    my_theta_2 = angle_query(my_v2, s_2_e)
    radius_s1 = math.pi/2-my_theta_2

    my_v4 = vector_cross_product(s_2_e, e_2_hand)
    my_theta_3 = angle_query(my_v3,my_v4)

    radius_e0 = my_theta_3
    if limb_name == 'left':
        radius_e0 = -radius_e0

    my_theta_4 = angle_query(s_2_e,e_2_hand)
    radius_e1 = my_theta_4

    return {"s0":radius_s0, "s1":radius_s1, "e0":radius_e0, "e1":radius_e1}

def track_one_arm(shoulder, shoulder_x, elbow, hand, torso, head, base, listener, limb_name):
    global pub_list_raw, pub_list


    d = tf_2_angles(shoulder, shoulder_x, elbow, hand, torso, head, base, listener, limb_name)

    if d == {}:
        print "tf listen failed."
        return

    radius_s0_raw = d["s0"]
    radius_s1_raw = d["s1"]
    radius_e0_raw = d["e0"]
    radius_e1_raw = d["e1"]

    print limb_name

    print
    for i in data_set[limb_name]:
```

```
            i.pop(0)

        data_set[limb_name][0].append(radius_s0_raw)
        data_set[limb_name][1].append(radius_s1_raw)
        data_set[limb_name][2].append(radius_e0_raw)
        data_set[limb_name][3].append(radius_e1_raw)

        command = [0.0, 0.0, 0.0, 0.0]
        for i in range(0, 4):
            sum = 0
            for j in range(0, times):
                sum = sum+data_set[limb_name][i][j]
            command[i] = sum/times

        radius_s0 = command[0]
        radius_s1 = command[1]
        radius_e0 = command[2]
        radius_e1 = command[3]

        print radius_s0, radius_s1, radius_e0, radius_e1

        print "haha"
        pub_list[limb_name+'_s0'].publish(radius_s0)
        print "hehe"

        pub_list[limb_name+'_s1'].publish(radius_s1)
        pub_list[limb_name+'_e0'].publish(radius_e0)
        pub_list[limb_name+'_e1'].publish(radius_e1)


def talker():
        global pub_list_raw, pub_list
        rospy.init_node('v8_xnode', anonymous=True)
        pub_left_s0 = rospy.Publisher('left_s0', Float64, queue_size=10)
        pub_left_s1 = rospy.Publisher('left_s1', Float64, queue_size=10)
        pub_left_e0 = rospy.Publisher('left_e0', Float64, queue_size=10)
        pub_left_e1 = rospy.Publisher('left_e1', Float64, queue_size=10)

        pub_right_s0 = rospy.Publisher('right_s0', Float64, queue_size=10)
        pub_right_s1 = rospy.Publisher('right_s1', Float64, queue_size=10)
        pub_right_e0 = rospy.Publisher('right_e0', Float64, queue_size=10)
        pub_right_e1 = rospy.Publisher('right_e1', Float64, queue_size=10)
        pub_list = {'left_s0':pub_left_s0,
                    'left_s1':pub_left_s1,
                    'left_e0':pub_left_e0,
                    'left_e1':pub_left_e1,
                   'right_s0':pub_right_s0,
                   'right_s1':pub_right_s1,
```

```
                'right_e0':pub_right_e0,
               'right_e1':pub_right_e1}

listener = tf.TransformListener()

user_index = '1'
i = 0
head_tf = None
last_head_tf = None

for_my_left_shoulder = 'right_shoulder_'+user_index
for_my_left_shoulder_x = 'left_shoulder_'+user_index
for_my_left_elbow = 'right_elbow_'+user_index
for_my_left_hand = 'right_hand_'+user_index

for_my_right_shoulder = 'left_shoulder_'+user_index
for_my_right_shoulder_x = 'right_shoulder_'+user_index
for_my_right_elbow = 'left_elbow_'+user_index
for_my_right_hand = 'left_hand_'+user_index

head = 'head_'+user_index
torso = 'torso_'+user_index
base = 'camera_depth_frame'

rospy.sleep(5)
rate = rospy.Rate(10)

while not rospy.is_shutdown():

    limb_1_name = 'left'
    limb_2_name = 'right'
    if mirror == True:
        limb_1_name = 'right'
        limb_2_name = 'left'

    track_one_arm(for_my_left_shoulder
                  ,for_my_left_shoulder_x
                  ,for_my_left_elbow
                  ,for_my_left_hand
                  ,torso
                  ,head
                  ,base
                  ,listener
                  ,limb_1_name)

    track_one_arm(for_my_right_shoulder
                  ,for_my_right_shoulder_x
                  ,for_my_right_elbow
```

```
                        ,for_my_right_hand
                        ,torso
                        ,head
                        ,base
                        ,listener
                        ,limb_2_name)

        rate.sleep()

if __name__ == '__main__':
    try:
        talker()
    except rospy.ROSInterruptException:
        pass
```

## 4.4 脸部识别与跟踪

此案例展示的是 Baxter 可以识别人脸，并实时跟踪人脸的方位，进而使 Baxter 头部跟随人脸运动。首先调用 Baxter 的头部摄像头获取图像，对图像进行分析，通过算法分析图像中是否含有人脸。

在你运行这个程序结点 (node) 前, 请确认你的电脑已经正确与 Baxter 建立了连接。然后请在终端中运行：

```
$ roslaunch  face_tracker  face_follower.launch
```

当你运行上面所说的 launch 文件时，它按顺序启动了四个程序结点 (node) 。

**第一个**被启动的程序结点 (node) 关闭了 Baxter 左手位置的摄像头 (*left_hand_camera*) 。

**第二个**被启动的程序结点 (node) 打开了 Baxter 头部位置的摄像头 (*head_camera*) 。这是因为基于 Baxter 内部带宽的限制， Baxter 身上同时开启的摄像头的数量最多只能达到 2 个。

**第三个**程序结点 (node) 运行了 rbx1 代码集中的 *face_tracker2* 。我们的代码需要该程序结点 (node) 进行人脸识别的计算。这意味着你需要先将 rbx1 的代码集下载到你本地的工作空间 (workspace) 中。

**第四个**程序结点 (node) 就是我们编写的用于跟踪脸部的，其中使用到了 PD 算法。

默认的头部位置摄像头 (*head_camera*) 获取的图像分辨率为 480x300。你可以在 launch 文件中对该参数进行更改。请在以下的分辨率中选择其中一个进行更改：1280x800, 960x600, 640x400, 480x300, 384x240, 320x200。选择的分辨率越大，图像处理的速度会越慢，但对脸部的识别会更加精确。

脸部识别与跟踪的相关代码如下：

```
import roslib
import rospy
from sensor_msgs.msg import RegionOfInterest, CameraInfo
from baxter_interface import head

class FaceFollower():
  def __init__(self):
    rospy.init_node("face_follower")
```

```
# Set the shutdown function
rospy.on_shutdown(self.shutdown)

# The tate of updating the robot's motion
self.rate = rospy.get_param("~rate", 10)
r = rospy.Rate(self.rate)

# The maximum rotation speed (speed range 0-100)
self.max_speed = rospy.get_param("~max_speed", 100)

# The minimum rotation speed (speed range 0-100)
self.min_speed = rospy.get_param("~min_speed", 5)

# Use PD Algorithm to control the robot
self.dist_gain = rospy.get_param("~dist_gain", 2.0)
self.speed_Kp = rospy.get_param("~speed_Kp", 2.0)
self.speed_Kd = rospy.get_param("~speed_Kd", 0)

# Only do some action if the displacement exceeds the threshold
# This can reduce shaking (in percentage format)
self.x_threshold = rospy.get_param("~x_threshold", 0.01)

# Intialize the head movement command
self.head_ = head.Head()
self.pan_distance_ = 0
self.pan_speed_ = 10
self.pre_pan_speed_ = 0
rospy.loginfo("Turning the head to the exactly middle...")
self.head_.set_pan(0, 10, 10.0)

# Get the image width and height from the camera_info topic
self.image_width = 0
self.image_height = 0

# Set flag to indicate when the ROI stops updating
self.target_visible = False

# Wait for the camera_info topic to become available
rospy.loginfo("Waiting for camera_info topic...")
rospy.wait_for_message("camera_info", CameraInfo)

# Subscribe the camera_info topic to get the image width and height
rospy.Subscriber("camera_info", CameraInfo, self.get_camera_info)


# Wait until we actually have the camera data
while self.image_width == 0 or self.image_height == 0:
```

```
        rospy.sleep(1)

    # Subscribe to the ROI topic and set the callback to update the robot's motion
    rospy.Subscriber('roi', RegionOfInterest, self.PD_cal)

    # Wait until ROI is detected
    rospy.wait_for_message('roi', RegionOfInterest)
    rospy.loginfo("ROI messages detected. Starting follower...")

    # Begin the following loop
    while not rospy.is_shutdown():
        # If the target is not visible, stop the robot
        if not self.target_visible:
            rospy.loginfo("No visible target!")
        else:
            # Reset the flag to False by default
            self.target_visible = False
            # Output data for debug
            rospy.loginfo("Following the targrt! Moving head to location %f in speed %d",
self.pan_distance_, self.pan_speed_)

        self.head_.set_pan(self.pan_distance_, self.pan_speed_, 10.0)

        # Sleep for 1/self.rate seconds
        r.sleep()

def PD_cal(self, msg):
    # If the ROI has a width or height of 0, we have lost the target
    if msg.width == 0 or msg.height == 0:
        return

    # If the ROI stops updating this, the robot will stay the same
    self.target_visible = True

    # Compute the displacement of the ROI from the center of the image
    target_offset_x = msg.x_offset + msg.width / 2 - self.image_width / 2

    try:
        percent_offset_x = float(target_offset_x) / (float(self.image_width) / 2.0)
    except:
        percent_offset_x = 0

    # Rotate the head only if the displacement of the target exceeds the threshold
    if abs(percent_offset_x) > self.x_threshold:
        # Calculate the pan distance by using P gain only
        self.pan_distance_ -= self.dist_gain * percent_offset_x
        # Calculate the pan speed by using PD Algorithm
        try:
```

```
            speed = self.speed_Kp * abs(percent_offset_x) - self.speed_Kd * (self.pan_speed_ -
self.pre_pan_speed_)
            if speed < 0:
                direction = -1
            else:
                direction = 1
            # Constrain the speed in range [min_speed, max_speed]
            self.pan_speed_ = abs(direction * max(self.min_speed, min(self.max_speed,
abs(speed))))
            # Store the speed as a previous one for next calculation
            self.pre_pan_speed_ = self.pan_speed_
        except:
            self.pan_speed_ = self.min_speed
            self.pre_pan_speed_ = self.min_speed
        #rospy.loginfo("percent_offset_x=%f,speed=%f,pre_pan_speed_=%d", percent_offset_x,
speed, self.pre_pan_speed_)
    else:
        # Otherwise stop the robot slowly. Here cannot set pan_distance_ to 0!
        self.pan_speed_ = 10

  def get_camera_info(self, msg):
    self.image_width = msg.width
    self.image_height = msg.height

  def shutdown(self):
    rospy.loginfo("Stopping the robot...")
    rospy.sleep(1)

if __name__ == '__main__':
  try:
    FaceFollower()
    rospy.spin()
  except rospy.ROSInterruptException:
    rospy.loginfo("Face following node terminated.")
```

## 4.5 Baxter 下四连棋游戏

此演示 connect_four 旨在展示 Baxter 在人工智能方面的应用，Baxter 不仅仅是一个可示教的机器人，采用人工智能技术，能使其拥有不逊于人类的大脑并具备可观测环境变化的眼睛。

将 cannect_four 演示示例作为教程，不仅能学习到棋盘类游戏最基本的 minimax 算法和人工智能领域重要的视觉处理算法，而且还能了解到 Baxter 本身的一些结构的 SDK 调用，比如正确开启手臂摄像头，如何记录手臂关节点位置，以及如何通过关节点位置使手臂作相应移动等。另外，因在该例程里用到了 ROS 系统，结合这些视觉、minimax 等人工智能算法以及 Baxter 本身的控制接口等，能通过该例程对如何控制和应用 Baxter 有一个比较系统的把握。

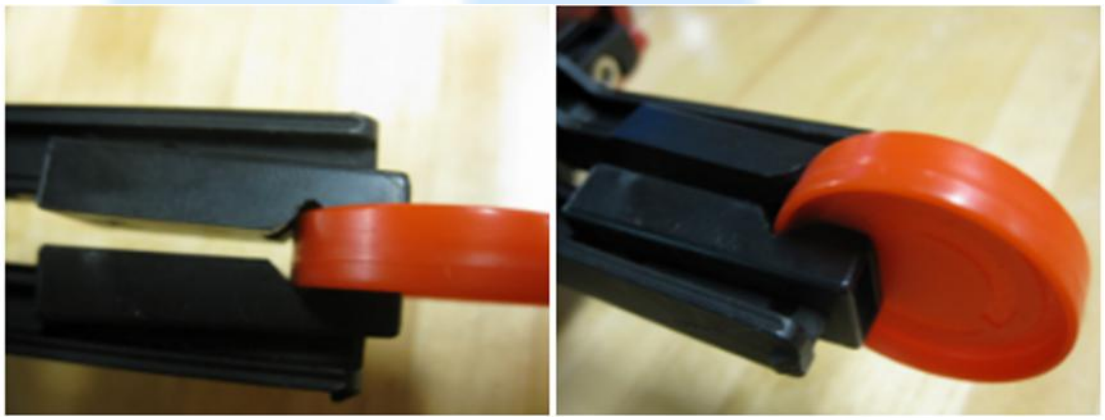
connect_four 就是采用屏风式的棋盘，宽为 7 个棋格，高为 6 个棋格，总共 42 个棋格，棋盘结构如图 4-13 所示。以宽方向的 7 个棋格为入口，两种颜色的棋子轮流下棋，一格格累加，直到棋盘在横、竖或者斜方向上出现同颜色四子相连的情况，则游戏结束，持

该颜色方则为获胜方。如若棋盘上为满格都还没有出现横或竖或斜方向上四子相连的情况，则为平局。



**图 4-13 四连棋棋盘**

在进行示例之前需要准备一些用具，比如四连棋棋盘；为了放置棋子掉落，需要设计夹取棋子的夹抓手指套，如图 4-14 所示；为了使 Baxter 设置一个固定的抓取棋子的位置，需要设计一个导槽，用于放置棋子，如图 4-14 所示，导槽的位置最好放置在棋盘平面的右前方或左前方；另外还需要一台装有 ubuntu 系统的电脑。



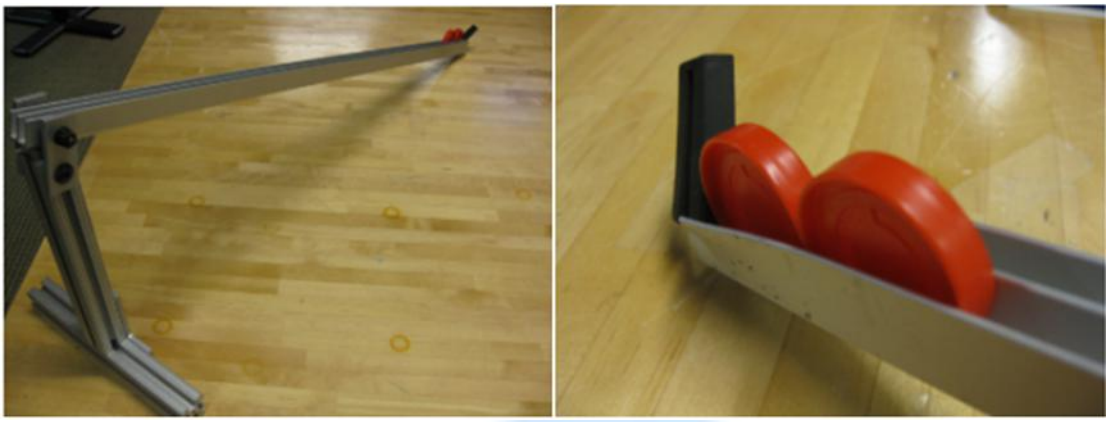


**图 4-14 夹取棋子的手抓和导槽示意图**

因 connect_four 在官网(sdk.rethinkrobotics.com)上已经有相应的演示样例，因此可参照 Baxter 的 sdk 官网的 Connect Four Demo 一步一步进行操作即可：

首先将 Connect Four 的程序包下载你的工作区间，此处假设工作区间为 ros_ws，输入命令：

```
$ cd  ~/ros_ws/src
$ git clone https://github.com/RethinkRobotics/baxter_sandbox.git
```

因为这个例子需要获取 Baxter 的关节点位置，以牵引 Baxter 的手臂移动，这些功能都是在 baxter_interface 库里提供：

```
$ cd ~/ros_ws/src/baxter_interface
$ git checkout development
$ git pull
```

然后在你的工作区间里运行 catkin_make 即可：

```
$ cd ~/ros_ws/
$ ./baxter.sh
$ catkin_make
```

将程序包下载完之后，可以看到工作区间多了一个 baxter_sandbox 的包，在这个包里有 connect_four 的文件夹，这便是 connect_four 的程序包。

在使用该程序包之前，需要修改下 vision.py 里的一些内容：将文件开头的 import cv 删除，将所有 cv.Xxx()（函数的第一个字母一般为大写）的函数都改为 cv2.xxx()（即函数的第一个字母都为小写），将所有 cv.CV_EVENT_XXX 的全局变量都改为 cv2.EVENT_XXX 即可。

当然，你们也可以直接使用我们提供的程序包，便可以不做任何修改。

确定程序包已经下载好之后，便可直接运行程序。

第一步、使能 Baxter，输入命令：

```
$ rosrun baxter_tools enable_robot.py -e
```

第二步、开启视觉功能：可以选择左臂或者右臂的摄像头来作视觉应用，建议一只手臂作为视觉应用，一只手臂作为抓取和放置的应用。此处选择左臂的摄像头作为视觉应用：

```
$ rosrun connect_four connect_vision.py -l left
```

其中-l 表示选择的手臂，left 为左臂，right 为右臂。这时会出现一个视觉图像显示界面，如图 4-15 所示：

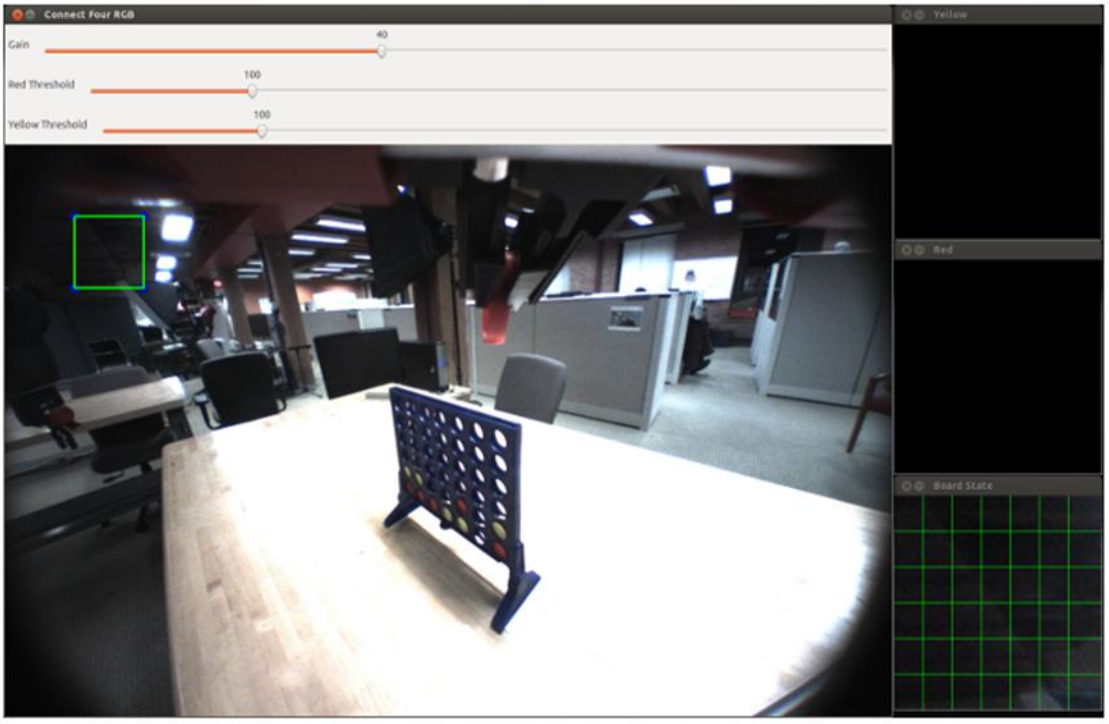



**图 4-15 视觉图像显示界面**

设置视觉的感兴趣区域(Region Of Interest, ROI)区域，即拖动绿色框的四个角点，使角点正好处于棋盘框的四个角点之上，如图 4-16 所示：

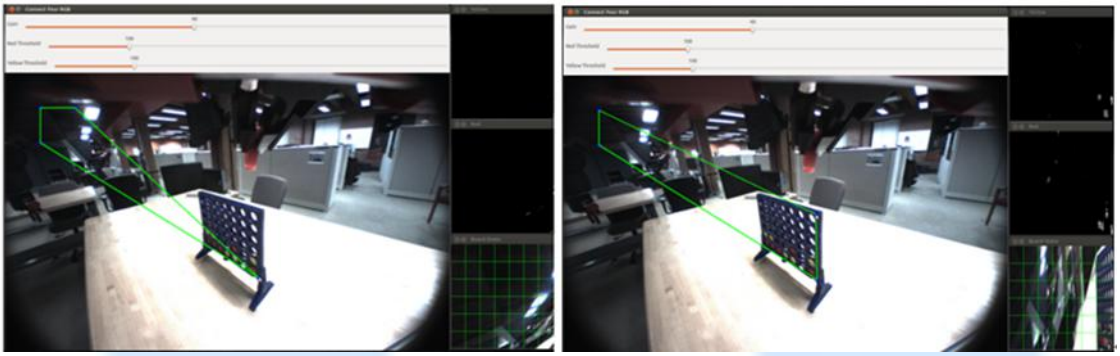


**图 4-16 视觉 ROI 角点拖动示意图**

设置成功之后的效果图如图 4-17 所示：

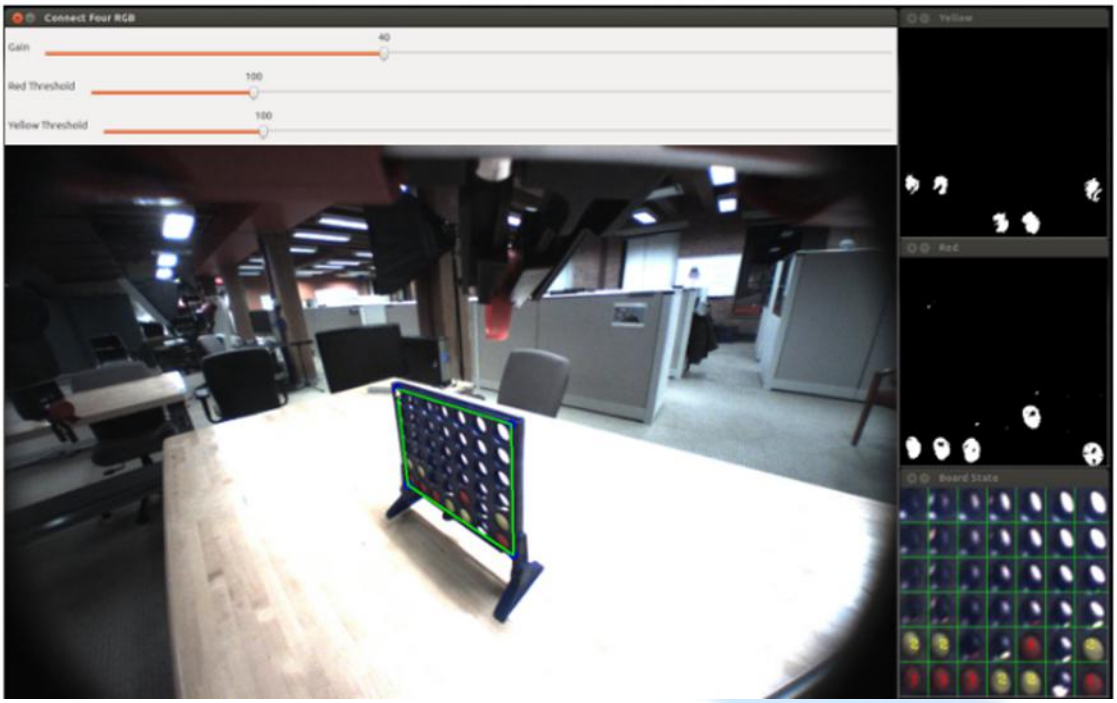



**图 4-17 视觉的 ROI 区域**

由图 4-17 可以看出，在图像的左侧上方有三个滑动条，分别是 Gain,Red Threshold 和 Yellow Threshold，移动这三个滑动条可以调节相应的阈值，使手臂摄像头更好的识别棋盘内的棋子的颜色。其中 Gain 调节的是环境的亮度，如果环境光线较暗，使棋子识别不准，则可以调大 Gain 的值来增加亮度，Gain 的值越大则越亮，一般的情况下 Gain 的值在 60 左右效果较好；Red Threshold 表示的是红色棋子的分割阈值，一般情况下为 160 左右；Yellow Threshold 为黄色棋子的分割阈值，一般情况下为 20 左右。如果这三个阈值都设置的较好，那么在图 4-17 右边的 3 个图像界面可以看到相应的分割效果图，图 4-17 右上所示为是黄色棋子的分割二值图，图 4-17 右中所示为红色棋子的分割二值图，图 4-17 右下所示为识别结果示意图，红色棋子上标为 1，黄色棋子上标为 2。

第三步、设置抓取和放置点：此处选择右臂作为抓取和放置的手臂。

```
$ rosrun connect_four connect_game.py -l right -c red
```

其中 -l 表示选择的手臂，-c 表示选择 Baxter 使用的棋子颜色，red 为红色，yellow 为黄色，如果 Baxter 选择红色棋子，则人类使用黄色棋子，反之亦然。运行该命令后，运行界面会出现提示词：

```
Would you like to use the previously found pick and place locations (y/n)?
```

输入 y 则表示使用之前设置好的那些点来进行操作，将不再做定点设置；输入 n 则表示需要重新定点设计。输入 n 后继续出现提示词：

```
Move right arm into a location allowing the camera to see the board - press Circle button to confirm
```

即需要你将手臂移动到摄像头可以看到棋盘的位置，因为此处我们使用了左臂的摄像头，则将该位置作为一个持平点（设计为高于棋盘的且位于抓取点的上方）的位置即可。值得注意的是，如果你采用了同一个手臂作为视觉应用和抓取应用，此处就要将该点放置在能看到棋盘的位置处。设置好该点后，按下手臂上的 Circle 按钮，就可以将该点记录下来，继续出现提示词：

```
Move Gripper pick location- press Circle button to record
```

即将手臂手抓移动到抓取棋子的位置，并按下 Circle 按钮记录好该位置。继续出现提示词：

| Move Gripper into left most drop slot - press Dash button to record |
|---|

即将手臂手抓移动到棋盘的最左端的入口，这个最左端是相对于 Baxter 本身而言的，并按下 Dash 按钮记录好该位置。继续出现提示词：

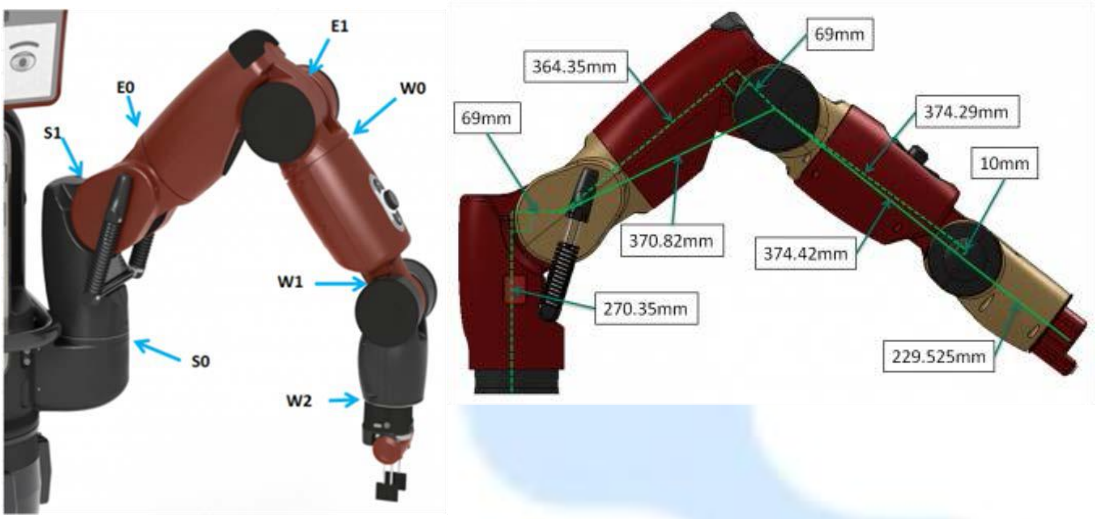
| Move Gripper into right most drop slot - press Dash button to record |
|---|

即将手臂移动到棋盘的最右端的棋盘入口，并按下 Dash 按钮记录好该位置。

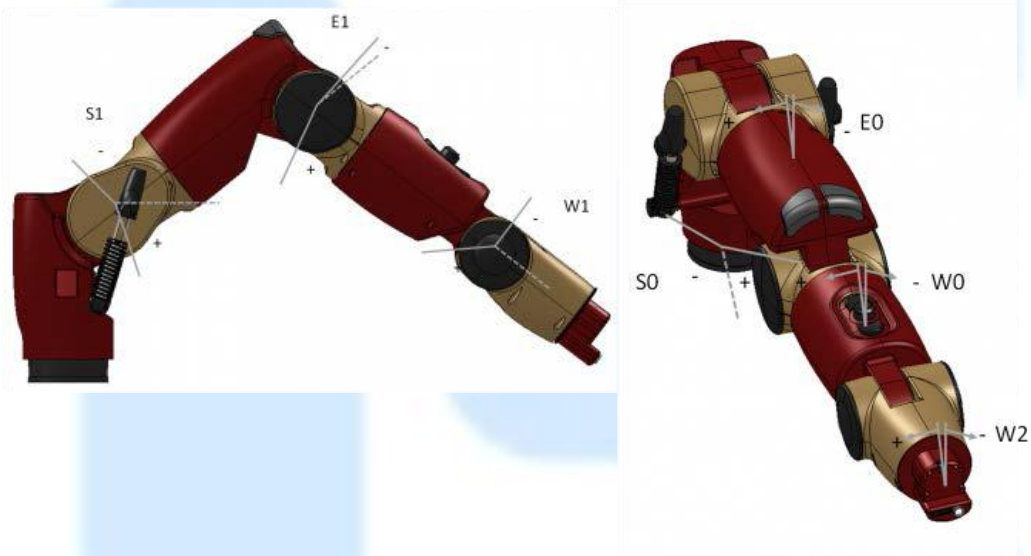
至此，右臂的相应的四个点都设置完成，Baxter 的右臂会接着移动到设置的第一个持平点的位置上。按提示按下 enter 键后需要再进行一些配置：首先是设置玩家名字，可随意输入；然后设计游戏等级，有 1-4 个等级，1 为最简单的，4 为最难的；接着设计下棋的次序，输入 baxter 则 Baxter 先下，输入 me 则人类先下。按要求配置好之后就可以进行游戏了。

# 附录一 Baxter 硬件参数

## 1）机械臂关节技术参数




图附-1 机械臂关节分布示意




图附-2 关节转动示意图

## 2）关节转动范围

表附-1 关节转动范围表格

| 关节 | 最小转角（°） | 最大转角 | 转动范围 | 最小转角（弧度） | 最大转角 | 转动范围 |
|---|---|---|---|---|---|---|
| S1 | -123 | +60 | 183 | -2.147 | +1.047 | 3.194 |
| E1 | -2.864 | +150 | 153 | -0.05 | +2.618 | 2.67 |
| W1 | -90 | +120 | 210 | -1.5707 | +2.094 | 3.6647 |
| S0 | -97.494 | +97.494 | 194.998 | -1.7016 | +1.7016 | 3.4033 |
| E0 | -174.987 | +174.987 | 349．979 | -3.0541 | +3.0541 | 6.1083 |
| W0 | -175.25 | +175.25 | 350.5 | -3.059 | +3.059 | 6.117 |

| | | | | | | |
|---|---|---|---|---|---|---|
| W2 | -175.25 | +175.25 | 350.5 | -3.059 | +3.059 | 6.117 |

**3）关节最大速度、弯曲刚度与峰值力矩**

**表附-2 关节最大速度、弯曲刚度与峰值力矩表**

| 关节 | 最大速度（弧度/秒） | 弯曲刚度 | 峰值力矩 |
|---|---|---|---|
| S0 | 2.0 | 50Nm时3.4度（约843Nm/rad） | 50Nm |
| S1 | 2.0 | 50Nm时3.4度（约843Nm/rad） | 50Nm |
| E0 | 2.0 | 50Nm时3.4度（约843Nm/rad） | 50Nm |
| E1 | 2.0 | 50Nm时3.4度（约843Nm/rad） | 50Nm |
| W0 | 4.0 | 15Nm时3.4度（约250Nm/rad） | 15Nm |
| W1 | 4.0 | 15Nm时3.4度（约250Nm/rad） | 15Nm |
| W2 | 4.0 | 15Nm时3.4度（约250Nm/rad） | 15Nm |

**4）其他硬件参数**

**表附-3 其他硬件参数表 •**

| | |
|---|---|
| **相机** | |
| 最大分辨率 | 1280x800像素 |
| 有效分辨率 | 640x480像素 |
| 帧速 | 30帧/秒 |
| 焦距 | 1.2mm |
| **内置主控** | |
| 处理器 | 第三代IntelCorei7-3770（8MB，3.4GHz） |
| 内存 | 4GB，1600MHZ，DDR3 |
| 硬盘 | 128GB固态硬盘 |
| **重量** | |
| 总重（带底座） | 135.2kg |
| 单臂 | 21.3kg |
| 躯干 | 31.8kg |
| 底座 | 60.8kg |
| 电气参数 | |
| 电池供电时 | 需要DC-120VAC逆变器（Baxter内部电脑不能通过 |
| 接口 | 标准120VAC电源，Baxter内部电源系统和电脑也 支 |
| 最大功耗 | 120VAC，6A，约720W |
| 电气效率 | 87%-92% |
| 内部电源 | 医用级直流开关电源 |
| 电压骤降 | 电压骤降至90V时，持续中断需要手动启动 |

| | |
|---|---|
| 电压闪变 | 滞留时间20ms |
| 电压不平衡 | 只需单相电供电 |
| **其他参数** | |
| 屏幕分辨率 | 1024x600像素 |
| 位置精度 | +/-5mm |
| 最大负载（含夹持器） | 2.2kg |
| 最大夹持力 | 35N |
| 红外探测距离 | 4-40cm |